

ĐẠI HỌC BÁCH KHOA HÀ NỘI

Khoa Toán - Tin



BÁO CÁO BÀI TẬP LỚN
KỸ THUẬT LẬP TRÌNH
(MI3310)

Đề tài:

**Xây dựng chương trình
hỗ trợ thi trắc nghiệm đơn giản**

Giảng viên hướng dẫn: TS. Vũ Thành Nam

Mã lớp học: 169312

Nhóm sinh viên thực hiện:

- | | | | |
|---|------------------|-----------|-------------------------------|
| 1 | Nguyễn Mạnh Hùng | 202419063 | Hệ thống thông tin quản lý 02 |
| 2 | Nguyễn Đức Thành | 202419099 | Hệ thống thông tin quản lý 02 |
| 3 | Nguyễn Hồng Vân | 202419119 | Hệ thống thông tin quản lý 02 |

Hà Nội, Tháng 6 2026

Lời mở đầu

Trong bối cảnh giáo dục ngày càng chú trọng đến việc ứng dụng công nghệ thông tin vào công tác đánh giá và kiểm tra, các hệ thống thi trắc nghiệm tự động đã trở thành một công cụ không thể thiếu tại nhiều cơ sở đào tạo. Từ những nền tảng trực tuyến quy mô lớn cho đến các phần mềm nội bộ đơn giản, điểm chung của chúng đều nằm ở khả năng tự động hoá quy trình ra đề, thu bài và chấm điểm — giảm thiểu công sức thủ công và nâng cao tính khách quan.

Xuất phát từ yêu cầu của môn học **Kỹ thuật Lập trình (MI3310)**, nhóm chúng em lựa chọn đề tài “*Xây dựng chương trình hỗ trợ thi trắc nghiệm đơn giản*” với mục tiêu vận dụng tổng hợp các kiến thức đã học vào một sản phẩm phần mềm có tính thực tiễn. Đây không đơn thuần là bài tập lập trình thông thường — đề tài đặt ra yêu cầu thiết kế một hệ thống hoàn chỉnh, bao gồm quản lý ngân hàng câu hỏi, sinh đề ngẫu nhiên, tiến hành thi có đếm giờ, chấm điểm tự động và lưu trữ lịch sử kết quả.

Điểm thách thức đặc biệt của đề tài nằm ở ràng buộc kỹ thuật: toàn bộ chương trình được xây dựng bằng ngôn ngữ **C++** thuần, không sử dụng các container có sẵn của thư viện chuẩn như `std::vector` hay `std::list`. Thay vào đó, nhóm tự cài đặt cấu trúc danh sách liên kết đơn bằng con trỏ, tự quản lý bộ nhớ động và tự triển khai tất cả các thuật toán cốt lõi. Điều này buộc mỗi thành viên phải nắm vững các nguyên lý nền tảng của lập trình hệ thống thay vì phụ thuộc vào các giải pháp trù tuợng hoá.

Báo cáo này trình bày toàn bộ quá trình phân tích, thiết kế, cài đặt và kiểm thử hệ thống. Nội dung được tổ chức theo cấu trúc khoa học: từ phân tích yêu cầu, thiết kế kiến trúc module, đến đánh giá kết quả kiểm thử và tổng kết các kỹ thuật đã vận dụng. Nhóm hy vọng báo cáo không chỉ phản ánh trung thực kết quả công việc mà còn là tài liệu tham khảo hữu ích cho các bạn sinh viên quan tâm đến lập trình hệ thống với C++.

Nhóm xin gửi lời cảm ơn chân thành đến **TS. Vũ Thành Nam** — giảng viên hướng dẫn môn Kỹ thuật Lập trình — vì những bài giảng sâu sắc và những góc nhìn thực tiễn đã định hướng cho chúng em trong suốt quá trình thực hiện đề tài. Dù đã cố gắng hết sức, báo cáo chắc chắn còn nhiều điểm cần cải thiện; nhóm rất mong nhận được sự góp ý từ thầy và hội đồng để hoàn thiện hơn trong tương lai.

Hà Nội, tháng 6 năm 2026

Nhóm sinh viên thực hiện

Nguyễn Mạnh Hùng — Nguyễn Đức Thành — Nguyễn Hồng Vân

Danh sách hình vẽ

4.1	Sơ đồ phân chia module và quan hệ phụ thuộc (mũi tên: “sử dụng/include”) . . .	26
-----	--	----

Danh sách bảng

1.1	Công nghệ và công cụ sử dụng trong dự án	9
1.2	Danh sách thành viên nhóm	9
1.3	Phân công nhiệm vụ giữa các thành viên	10
2.1	Các thao tác trên danh sách liên kết đơn QuestionBank	14
2.2	Cấu trúc header/source của các module trong dự án	16
2.3	Tổng kết liên hệ lý thuyết và cài đặt trong dự án	17
3.1	Tác nhân và quyền hạn trong hệ thống	18
3.2	Tổng hợp các yêu cầu chức năng của hệ thống	19
3.3	Đặc tả ràng buộc dữ liệu của một câu hỏi	20
3.4	Quy tắc tính số câu hỏi theo tỉ lệ độ khó 40-40-20	21
3.5	Tổng hợp các yêu cầu phi chức năng của hệ thống	23
4.1	Luồng dữ liệu giữa các module ứng với từng bước nghiệp vụ	26
4.2	Các lựa chọn trong showAdminMenu	27
4.3	Các lựa chọn trong showStudentMenu	27
4.4	Cấu trúc 9 trường của một dòng trong questions.txt	28
4.5	Cấu trúc 6 trường của một dòng trong history.txt	28
4.6	Thiết kế thuộc tính của struct Question	30
6.1	Test Case-01	41
6.2	Test Case-02	41
6.3	Test Case-03	42
6.4	Test Case-04	42
6.5	Test Case-05	42
6.6	Test Case-06	43
6.7	Test Case-07	43
6.8	Test Case-08	43
6.9	Test Case-09	43
6.10	Test Case-10	44
6.11	Tổng hợp kết quả 10 Test Case	45
B.1	Tổng hợp các hàm xử lý chính theo từng module	56
D.1	Tài khoản mặc định để đăng nhập chương trình	86

Mục lục

Lời mở đầu	1
Danh sách hình vẽ	2
Danh sách bảng	3
1 Giới thiệu	7
1.1 Bối cảnh	7
1.2 Lý do lựa chọn bài toán	7
1.3 Mô tả bài toán	8
1.4 Mục tiêu của chương trình	8
1.5 Phạm vi thực hiện	8
1.6 Công nghệ và ngôn ngữ lập trình sử dụng	9
1.7 Phân công công việc	9
1.7.1 Thành viên thực hiện	9
1.7.2 Phân công chi tiết nhiệm vụ	10
2 Cơ sở lý thuyết	11
2.1 Nguyên lý thiết kế chương trình theo module	11
2.1.1 Tư tưởng module hoá (Modularity)	11
2.1.2 Thiết kế từ trên xuống (Top-down Design)	11
2.1.3 Phong cách lập trình tốt	12
2.2 Con trỏ và quản lý bộ nhớ động trong C++	12
2.2.1 Bộ nhớ tĩnh và bộ nhớ động	12
2.2.2 Rò rỉ bộ nhớ (Memory Leak)	12
2.2.3 Constructor và Destructor	12
2.3 Danh sách liên kết đơn (Singly Linked List)	13
2.3.1 Khái niệm và cấu trúc	13
2.3.2 Ưu điểm so với mảng tĩnh	13
2.3.3 Các thao tác cơ bản	13
2.3.4 Cài đặt thêm và xoá node	14
2.4 Thuật toán xáo trộn Fisher-Yates	14
2.4.1 Nguyên lý hoạt động	14
2.4.2 Độ phức tạp	14
2.4.3 Ứng dụng trong dự án	14
2.5 Xử lý File I/O trong C++	15
2.5.1 Các lớp stream cơ bản	15
2.5.2 Định dạng dữ liệu	15
2.5.3 Đọc file với xử lý lỗi	15
2.6 Lập trình hướng đối tượng trong C++	16
2.6.1 Class, đóng gói và trừu tượng hoá	16
2.6.2 Tách khai báo và cài đặt (Header/Source)	16

2.6.3	Validate input và lập trình phòng ngừa	16
2.7	Tổng kết	17
3	Phân tích yêu cầu hệ thống	18
3.1	Khảo sát bài toán	18
3.1.1	Phương pháp khảo sát	18
3.1.2	Đối tượng sử dụng hệ thống	18
3.1.3	Kết quả khảo sát	19
3.2	Yêu cầu chức năng	19
3.2.1	Quản lý ngân hàng câu hỏi	19
3.2.2	Cấu hình và sinh đề thi	20
3.2.3	Tiến hành thi và đếm giờ thời gian thực	21
3.2.4	Chấm điểm tự động và hiển thị kết quả	21
3.2.5	Xem lịch sử thi	22
3.3	Yêu cầu phi chức năng	22
3.4	Mô tả luồng hoạt động của chương trình	23
4	Thiết kế hệ thống	25
4.1	Kiến trúc tổng thể chương trình	25
4.1.1	Sơ đồ phân chia module	25
4.1.2	Luồng dữ liệu giữa các module	26
4.1.3	Sơ đồ luồng điều hướng Menu	26
4.2	Thiết kế cấu trúc dữ liệu	27
4.2.1	Cấu trúc file questions.txt	27
4.2.2	Cấu trúc file history.txt	28
4.2.3	Quy tắc định dạng và tách trường dữ liệu	28
4.3	Thiết kế các module chức năng	29
4.3.1	Module Menu	29
4.3.2	Module Ngân hàng câu hỏi	30
4.3.3	Module Thi	31
4.3.4	Module Lịch sử thi	32
4.3.5	Hàm main	32
5	Cài đặt và xây dựng chương trình	34
5.1	Cấu trúc thư mục chương trình	34
5.2	Cài đặt module Ngân hàng câu hỏi (QuestionBank)	34
5.2.1	Khai báo cấu trúc dữ liệu	34
5.2.2	Quản lý bộ nhớ: Constructor và Destructor	35
5.2.3	Thêm và xoá câu hỏi	35
5.2.4	Đọc và ghi file dữ liệu	36
5.2.5	Truy vấn câu hỏi theo môn học và độ khó	36
5.3	Cài đặt module Thi (Exam)	37
5.3.1	Sinh đề theo tỉ lệ độ khó 40-40-20	37
5.3.2	Xác định số câu hỏi tối đa hợp lệ	38
5.3.3	Đếm giờ thời gian thực và thu bài tự động	38
5.3.4	Chấm điểm	38
5.4	Cài đặt module Lịch sử thi (History)	38
5.4.1	Cấu trúc dữ liệu TestRecord	38
5.4.2	Ghi và đọc lịch sử	39
5.4.3	Truy vấn theo người dùng	39
5.5	Cài đặt module Giao diện (Menu)	39
5.5.1	Xác thực đăng nhập	39
5.5.2	Validate input	40
5.5.3	Điều hướng Menu Admin và Menu Sinh viên	40

5.6	Biên dịch chương trình	40
6	Kiểm thử và Đánh giá kết quả	41
6.1	Chiến lược kiểm thử	41
6.2	Tình huống kiểm thử	41
6.2.1	Test Case-01: Đăng nhập đúng – Admin	41
6.2.2	Test Case-02: Đăng nhập đúng – Sinh viên	41
6.2.3	Test Case-03: Đăng nhập sai mật khẩu	42
6.2.4	Test Case-04: Thêm câu hỏi mới	42
6.2.5	Test Case-05: Sinh đề và xáo trộn câu hỏi/đáp án	42
6.2.6	Test Case-06: Làm bài và chấm điểm chính xác	43
6.2.7	Test Case-07: Tự động thu bài khi hết giờ	43
6.2.8	Test Case-08: Xem lịch sử thi nhiều lần	43
6.2.9	Test Case-09: Nhập sai định dạng (bẫy lỗi)	43
6.2.10	Test Case-10: Kiểm thử rò rỉ bộ nhớ (Valgrind)	44
6.3	Kịch bản kiểm thử minh hoạ	44
6.3.1	Khởi động chương trình và đăng nhập	44
6.3.2	Luồng thao tác của Admin	44
6.3.3	Luồng thao tác của sinh viên	45
6.3.4	Nhận xét từ kịch bản kiểm thử thực tế	45
6.4	Đánh giá kết quả kiểm thử	45
7	Kết luận và hướng phát triển	47
7.1	Kết quả đạt được	47
7.2	Tổng kết các kỹ thuật đã được vận dụng	47
7.2.1	Cấu trúc dữ liệu: Danh sách liên kết đơn	47
7.2.2	Quản lý bộ nhớ động: con trỏ, new/delete, tránh Memory Leak	48
7.2.3	Xử lý File I/O: đọc/ghi file text với ký tự phân tách ' '	48
7.2.4	Thuật toán xáo trộn Fisher-Yates	48
7.2.5	Lập trình hướng module: tách Header – Source – Data	48
7.2.6	Lập trình hướng đối tượng (OOP): Class, Constructor, Destructor	48
7.2.7	Lập trình phòng ngừa	48
7.2.8	Validate input và xử lý ngoại lệ trên Terminal	49
7.2.9	Quy chuẩn lập trình: Naming Convention, No Global Variables	49
7.3	Đánh giá ưu điểm và hạn chế của chương trình	49
7.4	Hướng phát triển và Nâng cấp tương lai	49
	Lời cảm ơn	51
A	Mã nguồn hàm main()	53
B	Mô tả các hàm xử lý chính	55
C	Mã nguồn đầy đủ chương trình	57
C.1	QuestionBank.h	57
C.2	QuestionBank.cpp	58
C.3	Exam.h	65
C.4	Exam.cpp	66
C.5	History.h	71
C.6	History.cpp	72
C.7	Menu.h	75
C.8	Menu.cpp	76
D	Hướng dẫn cài đặt và sử dụng chương trình	85

Chương 1

Giới thiệu

1.1 Bối cảnh

Trong những năm gần đây, việc ứng dụng công nghệ thông tin vào giáo dục đã tạo ra những bước chuyển biến sâu sắc trong cách thức tổ chức kiểm tra và đánh giá năng lực người học. Các hệ thống thi trắc nghiệm điện tử không chỉ giúp tiết kiệm chi phí in ấn và thời gian chấm bài thủ công, mà còn đảm bảo tính công bằng, khách quan và có thể mở rộng quy mô dễ dàng [1]. Tại các trường đại học kỹ thuật, nơi khối lượng sinh viên lớn và nhu cầu kiểm tra định kỳ cao, yêu cầu về một công cụ hỗ trợ thi trắc nghiệm đáng tin cậy ngày càng trở nên cấp thiết.

Số liệu thị trường thực tế cho thấy rõ quy mô và tốc độ tăng trưởng của xu hướng này. Theo thống kê được tổng hợp từ Statista và các báo cáo ngành, phân khúc thị trường nền tảng học tập trực tuyến (E-learning) tại Việt Nam được dự đoán đạt giá trị **228,7 triệu USD vào năm 2024**, với tỷ lệ tiếp cận người dùng khoảng **8,6%** [2]. Ở phạm vi rộng hơn, toàn bộ thị trường công nghệ giáo dục (EdTech) Việt Nam được dự báo đạt **364,7 triệu USD** trong năm 2024, với tốc độ tăng trưởng kép hàng năm (CAGR) lên tới **13,5%** cho giai đoạn đến năm 2032 [3]. Báo cáo *Vietnam EdTech & eLearning Report 2025* — ấn phẩm thường niên lần thứ 15 về thị trường này — cũng ghi nhận các startup EdTech trong nước thu hút hơn **25 triệu USD vốn đầu tư** chỉ riêng trong năm 2024 [4]. Theo Quyết định số 131/QĐ-TTg của Thủ tướng Chính phủ (2022), Việt Nam đặt mục tiêu đến năm 2025 có hơn **50% cơ sở giáo dục đại học** triển khai chương trình đào tạo từ xa, trực tuyến [5]. Những con số này khẳng định nhu cầu thực tiễn ngày càng lớn đối với các công cụ phần mềm hỗ trợ giảng dạy và kiểm tra — trong đó hệ thống thi trắc nghiệm tự động là một thành phần cốt lõi.

Song song với xu hướng đó, môn học **Kỹ thuật Lập trình (MI3310)** tại Khoa Toán – Tin, Đại học Bách khoa Hà Nội đặt ra mục tiêu rèn luyện sinh viên không chỉ về kỹ năng viết mã mà còn về tư duy thiết kế phần mềm có cấu trúc. Một trong những yêu cầu quan trọng của môn học là sinh viên phải tự cài đặt các cấu trúc dữ liệu cơ bản bằng tay, không phụ thuộc vào thư viện chuẩn [6]. Điều này tạo ra một bối cảnh học tập lý tưởng để kết hợp nhu cầu thực tiễn với mục tiêu giáo dục: xây dựng một hệ thống thi trắc nghiệm hoàn chỉnh hoàn toàn bằng C++ thuần.

1.2 Lý do lựa chọn bài toán

Nhóm lựa chọn đề tài “*Xây dựng chương trình hỗ trợ thi trắc nghiệm đơn giản*” xuất phát từ ba lý do chính.

Thứ nhất, bài toán có tính thực tiễn rõ ràng. Hệ thống quản lý câu hỏi, sinh đề, đếm giờ và chấm điểm là những nghiệp vụ xuất hiện trong hầu hết mọi hệ thống giáo dục. Việc xây dựng một phiên bản đơn giản giúp nhóm tiếp cận quy trình phân tích – thiết kế – cài đặt theo đúng vòng đời phát triển phần mềm [7].

Thứ hai, bài toán đòi hỏi vận dụng đồng thời nhiều kỹ thuật lập trình nâng cao: quản lý bộ nhớ động với `new/delete`, cấu trúc danh sách liên kết đơn, thuật toán xáo trộn ngẫu nhiên, xử lý File I/O, lập trình hướng đối tượng và tổ chức mã nguồn theo module. Đây chính xác là tập hợp các chủ đề cốt lõi mà môn học hướng đến [6].

Thứ ba, ràng buộc không dùng STL container buộc nhóm phải thực sự hiểu cách các cấu trúc dữ liệu hoạt động ở cấp độ con trỏ, thay vì chỉ sử dụng chúng như những hộp đen. Đây là kinh nghiệm học tập không thể thay thế đối với sinh viên kỹ thuật.

1.3 Mô tả bài toán

Trong thực tế giảng dạy, việc tổ chức một bài kiểm tra trắc nghiệm thủ công đòi hỏi giáo viên phải thực hiện nhiều bước lặp đi lặp lại: soạn đề, in ấn, phát đề, thu bài, chấm điểm và tổng hợp kết quả. Với lớp học đông sinh viên, quy trình này tiêu tốn nhiều thời gian và dễ phát sinh sai sót.

Bài toán đặt ra là xây dựng một chương trình phần mềm chạy trên nền Terminal có khả năng thay thế hoàn toàn quy trình thủ công trên. Hệ thống cần đáp ứng ba nghiệp vụ chính:

1. **Quản lý ngân hàng câu hỏi:** Admin có thể thêm, xoá câu hỏi trắc nghiệm với 4 lựa chọn và đáp án đúng. Dữ liệu được lưu trữ bền vững trong file văn bản, đảm bảo không mất mát khi tắt chương trình.
2. **Tổ chức thi:** Sinh viên đăng nhập, chọn số lượng câu và thời gian, được cấp một đề thi sinh ngẫu nhiên từ ngân hàng câu hỏi. Thứ tự câu hỏi và thứ tự các đáp án được xáo trộn độc lập cho mỗi lần thi, cùng với giới hạn thời gian làm bài được đếm ngược theo thời gian thực.
3. **Chấm điểm và báo cáo kết quả:** Sau khi nộp bài — dù do sinh viên chủ động hoặc do hết giờ — hệ thống tự động chấm điểm, hiển thị kết quả chi tiết từng câu và lưu lịch sử để có thể truy xuất lại sau.

Toàn bộ chương trình được cài đặt bằng C++ (chuẩn C++17), không sử dụng các container của thư viện STL, mà thay vào đó tự cài đặt cấu trúc dữ liệu bằng con trỏ để đáp ứng yêu cầu kỹ thuật của môn học.

1.4 Mục tiêu của chương trình

Chương trình hướng đến các mục tiêu cụ thể sau:

- Xây dựng hệ thống thi trắc nghiệm hoàn chỉnh, có thể vận hành độc lập trên môi trường Terminal mà không phụ thuộc vào kết nối mạng hay giao diện đồ hoạ.
- Đảm bảo tính công bằng và ngẫu nhiên trong quá trình ra đề thông qua thuật toán xáo trộn Fisher-Yates [8].
- Cung cấp trải nghiệm thi sát với thực tế: có giới hạn thời gian, có cơ chế tự động thu bài và phản hồi kết quả tức thì sau khi nộp bài.
- Lưu trữ và truy xuất lịch sử thi của từng sinh viên qua nhiều lần thi, phục vụ mục đích theo dõi tiến độ học tập.
- Vận dụng và củng cố các kiến thức nền tảng của môn Kỹ thuật Lập trình: con trỏ, cấu trúc dữ liệu tự cài đặt, quản lý bộ nhớ động, xử lý file I/O, lập trình hướng đối tượng và tổ chức chương trình theo module [6].

1.5 Phạm vi thực hiện

Trong khuôn khổ bài tập lớn môn học, chương trình được giới hạn phạm vi như sau:

- **Môi trường thực thi:** Terminal / Command Prompt trên hệ điều hành Windows (MinGW / g++) hoặc Linux/macOS (g++). Không có giao diện đồ hoạ (GUI).
- **Người dùng:** Hai vai trò — Admin (quản trị viên) và Sinh viên (người thi). Xác thực bằng tên đăng nhập và mật khẩu đơn giản, không mã hoá.
- **Dữ liệu:** Câu hỏi và lịch sử thi được lưu trên file văn bản cục bộ (`questions.txt`, `history.txt`), không sử dụng hệ quản trị cơ sở dữ liệu.
- **Câu hỏi:** Dạng trắc nghiệm một đáp án đúng trong bốn lựa chọn (A, B, C, D). Chưa hỗ trợ câu hỏi tự luận, câu hỏi có hình ảnh hay nhiều đáp án đúng.
- **Đề thi:** Số lượng câu hỏi và thời gian làm bài được sinh viên tự cấu hình khi bắt đầu mỗi lần thi; không phân loại theo môn học hay độ khó.

1.6 Công nghệ và ngôn ngữ lập trình sử dụng

Thành phần	Chi tiết
Ngôn ngữ lập trình	C++ (chuẩn C++17)
Trình biên dịch	MinGW g++ (Windows) / g++ (Linux, macOS)
Thư viện sử dụng	<code><iostream></code> , <code><fstream></code> , <code><sstream></code> , <code><string></code> , <code><cstdlib></code> , <code><ctime></code> , <code><cctype></code> , <code><iomanip></code> , <code><limits></code>
Môi trường phát triển	Visual Studio Code với extension C/C++ (Microsoft)
Quản lý dự án	ClickUp (phân công nhiệm vụ), GitHub (quản lý mã nguồn)
Kiểm thử bộ nhớ	Valgrind (Linux) / DrMemory (Windows)

Bảng 1.1: Công nghệ và công cụ sử dụng trong dự án

Nhóm tuân thủ nguyên tắc không sử dụng biến toàn cục — mọi dữ liệu đều được truyền qua tham số hàm hoặc quản lý trong các class — và không sử dụng các container STL như `std::vector`, `std::list`. Cấu trúc danh sách liên kết đơn được tự cài đặt để lưu trữ câu hỏi và lịch sử thi, nhằm rèn luyện kỹ năng làm việc trực tiếp với bộ nhớ động [9].

1.7 Phân công công việc

1.7.1 Thành viên thực hiện

STT	Họ và tên	MSSV	Lớp
1	Nguyễn Mạnh Hùng	202419063	Hệ thống thông tin quản lý 02
2	Nguyễn Đức Thành	202419099	Hệ thống thông tin quản lý 02
3	Nguyễn Hồng Vân	202419119	Hệ thống thông tin quản lý 02

Bảng 1.2: Danh sách thành viên nhóm

1.7.2 Phân công chi tiết nhiệm vụ

Thành viên	Vai trò	Nhiệm vụ cụ thể
Thành	System Architect	Xây dựng giao diện Menu trên Terminal; điều hướng luồng Admin và Sinh viên; xử lý bẫy lỗi nhập liệu (validate input); viết file <code>main.cpp</code> , ráp nối các module; kiểm thử toàn hệ thống (cross-testing); đóng gói báo cáo cuối.
Hùng	Algorithm Engineer	Xây dựng luồng cốt lõi bài thi; cài đặt thuật toán Fisher-Yates xáo trộn câu hỏi và đáp án; xử lý module đếm giờ đếm ngược thời gian thực; cơ chế tự động thu bài khi hết giờ; logic tính điểm và tổng hợp kết quả.
Vân	Data Engineer	Cài đặt cấu trúc danh sách liên kết đơn cho ngân hàng câu hỏi và lịch sử thi; quản lý cấp phát và giải phóng bộ nhớ (tránh Memory Leak); xử lý File I/O: đọc/ghi <code>questions.txt</code> và <code>history.txt</code> với ký tự phân tách .

Bảng 1.3: Phân công nhiệm vụ giữa các thành viên

Quá trình phối hợp được thực hiện qua **GitHub** (quản lý mã nguồn theo nhánh) và **ClickUp** (theo dõi tiến độ nhiệm vụ). Quy tắc chung được thống nhất: mỗi thành viên chỉ đẩy code lên khi đã tự biên dịch thành công với **0 lỗi** trên máy cá nhân, tuyệt đối không đẩy code đang lỗi cú pháp ảnh hưởng đến tiến độ chung của nhóm.

Chương 2

Cơ sở lý thuyết

Chương này trình bày các kiến thức nền tảng được vận dụng trong quá trình xây dựng chương trình, bao gồm: nguyên lý thiết kế chương trình theo module, cấu trúc dữ liệu danh sách liên kết đơn, kỹ thuật quản lý bộ nhớ động, thuật toán xáo trộn Fisher-Yates, xử lý File I/O và lập trình hướng đối tượng trong C++.

2.1 Nguyên lý thiết kế chương trình theo module

2.1.1 Tư tưởng module hoá (Modularity)

Một trong những nguyên lý căn bản nhất của kỹ thuật lập trình hiện đại là *module hoá* — tức là phân chia chương trình lớn thành các đơn vị nhỏ hơn, mỗi đơn vị thực thi một nhiệm vụ cụ thể và độc lập [6]. Theo nguyên lý này, mỗi module (hàm, class, hoặc file mã nguồn) cần thoả mãn ba yêu cầu: có nhiệm vụ rõ ràng và duy nhất, giao tiếp với bên ngoài thông qua một giao diện tường minh (tham số và giá trị trả về), và có thể kiểm thử độc lập.

Lợi ích của module hoá thể hiện rõ ở bốn phương diện [6]:

- **Dễ đọc:** Mỗi module đủ ngắn để nắm bắt toàn bộ logic trong một lần đọc.
- **Dễ kiểm thử:** Có thể test từng hàm riêng biệt mà không cần khởi chạy toàn bộ hệ thống.
- **Dễ bảo trì:** Khi cần sửa đổi một chức năng, chỉ thay đổi module liên quan mà không ảnh hưởng đến phần còn lại.
- **Tái sử dụng:** Các module được thiết kế tốt có thể dùng lại trong các dự án khác.

Trong dự án này, nguyên lý module hoá được áp dụng thông qua việc phân chia mã nguồn thành bốn module độc lập: **QuestionBank** (quản lý ngân hàng câu hỏi), **Exam** (logic bài thi), **History** (quản lý lịch sử) và **Menu** (giao diện và điều hướng), mỗi module có file header (.h) và file cài đặt (.cpp) riêng.

2.1.2 Thiết kế từ trên xuống (Top-down Design)

Phương pháp thiết kế từ trên xuống bắt đầu bằng việc phác hoạ toàn bộ chương trình ở mức tổng quan, sau đó tinh chỉnh dần xuống mức chi tiết [6]. Cụ thể, quy trình gồm ba bước lặp:

1. Phác thảo hàm `main()` bằng giả ngữ (pseudocode), mô tả *cái gì cần làm chứ không phải làm như thế nào*.
2. Với mỗi tác vụ phức tạp trong giả ngữ, thay thế bằng lời gọi hàm và định nghĩa hàm đó ở bước tiếp theo.
3. Lặp lại quá trình ở mức chi tiết hơn cho đến khi toàn bộ các hàm được định nghĩa đầy đủ.

Phương pháp này giúp kiểm soát độ phức tạp của chương trình ngay từ đầu, tránh tình trạng viết code đến đâu thiết kế đến đó (bottom-up không có kế hoạch) dẫn đến mã nguồn rối và khó bảo trì [6].

2.1.3 Phong cách lập trình tốt

Mã nguồn tốt không chỉ là mã nguồn chạy đúng, mà còn là mã nguồn dễ đọc, dễ hiểu và dễ bảo trì [10]. Các nguyên tắc phong cách lập trình được nhóm tuân thủ trong dự án này bao gồm:

- **Đặt tên gợi nhớ:** Tên biến và hàm phản ánh rõ mục đích (ví dụ: `numberOfQuestions`, `shuffleAnswers`, `generateRandomSet`).
- **Chú thích có ý nghĩa:** Chú thích cho từng đoạn mã (không phải từng dòng), mô tả *tại sao* và *cái gì*, không phải *như thế nào* khi code đã tự giải thích.
- **Nhất quán trong cách lề (indentation):** Toàn bộ mã nguồn dùng 4 dấu cách cho mỗi cấp lề.
- **Không dùng biến toàn cục:** Mọi dữ liệu được truyền qua tham số, tuân thủ nguyên tắc bao đóng (encapsulation) [9].
- **Không vá vúi mã xấu:** Khi phát hiện lỗi thiết kế, viết lại thay vì chắp vá [10].

2.2 Con trỏ và quản lý bộ nhớ động trong C++

2.2.1 Bộ nhớ tĩnh và bộ nhớ động

Trong C++, bộ nhớ được chia thành hai vùng chính: *stack* (ngăn xếp) và *heap* (đống). Biến cục bộ được cấp phát tự động trên *stack* và giải phóng khi hàm trả về. Ngược lại, bộ nhớ động được cấp phát trên *heap* thông qua toán tử `new` và phải được lập trình viên tường minh giải phóng bằng `delete` (hoặc `delete[]` với mảng) [9].

```

1 // Cấp phát mảng động
2 Question* questionList = new Question[numberOfQuestions];
3 char*      userAnswers  = new char[numberOfQuestions];
4
5 // Su dụng ...
6
7 // Giải phóng (bắt buộc, tránh Memory Leak)
8 delete[] questionList;
9 delete[] userAnswers;
```

Listing 2.1: Cấp phát và giải phóng mảng động trong C++

2.2.2 Rò rỉ bộ nhớ (Memory Leak)

Rò rỉ bộ nhớ xảy ra khi chương trình cấp phát bộ nhớ động nhưng không giải phóng sau khi sử dụng xong [9]. Trong một chương trình chạy dài, hiện tượng này dẫn đến cạn kiệt bộ nhớ hệ thống. Để phòng tránh, nhóm tuân thủ quy tắc: mỗi `new` phải có một `delete` tương ứng, và việc giải phóng được thực hiện trong Destructor của class.

2.2.3 Constructor và Destructor

Constructor được gọi tự động khi một đối tượng được khởi tạo, còn Destructor được gọi khi đối tượng ra khỏi phạm vi hoặc bị xoá. Đây là cơ chế RAII (Resource Acquisition Is Initialization) của C++, đảm bảo tài nguyên luôn được quản lý đúng đắn [9]. Trong dự án, Destructor của `QuestionBank`, `HistoryManager` và `Exam` đều chịu trách nhiệm giải phóng toàn bộ bộ nhớ động đã cấp phát.

```

1 // Constructor: Cap phat dong mang Question[N]
2 Exam::Exam(int n, int t) {
3     numberOfQuestions = n;
4     timeLimitSec = t;
5     questionList = new Question[numberOfQuestions];
6     userAnswers = new char[numberOfQuestions];
7     for (int i = 0; i < numberOfQuestions; i++)
8         userAnswers[i] = '-';
9 }
10
11 // Destructor: Giai phong bo nho dong
12 Exam::~Exam() {
13     delete[] questionList;
14     delete[] userAnswers;
15 }

```

Listing 2.2: Constructor và Destructor của lớp Exam

2.3 Danh sách liên kết đơn (Singly Linked List)

2.3.1 Khái niệm và cấu trúc

Danh sách liên kết đơn là một cấu trúc dữ liệu tuyến tính trong đó mỗi phần tử (node) lưu trữ một giá trị dữ liệu và một con trỏ trỏ đến node kế tiếp [11]. Node cuối cùng có con trỏ `next` bằng `nullptr`. Danh sách được quản lý thông qua hai con trỏ: `head` (trỏ đến node đầu tiên) và `tail` (trỏ đến node cuối cùng).

```

1 struct Question {
2     int id;
3     string content;
4     string answers[4];
5     char correct; // 'A', 'B', 'C' hoặc 'D'
6 };
7
8 struct QuestionNode {
9     Question data;
10    QuestionNode* next;
11 };

```

Listing 2.3: Định nghĩa node danh sách liên kết đơn cho câu hỏi

2.3.2 Ưu điểm so với mảng tĩnh

Khác với mảng có kích thước cố định phải khai báo trước, danh sách liên kết đơn cho phép thêm và xoá phần tử linh hoạt mà không cần cấp phát lại toàn bộ vùng nhớ [11]. Điều này rất phù hợp với ngân hàng câu hỏi, nơi số lượng câu hỏi thay đổi theo thời gian: Admin có thể thêm câu hỏi mới (thêm vào cuối danh sách trong $O(1)$ với con trỏ `tail`) hoặc xoá câu hỏi theo ID (duyệt tuyến tính $O(n)$).

2.3.3 Các thao tác cơ bản

Bảng 2.1 tóm tắt các thao tác cơ bản được cài đặt trong dự án cùng độ phức tạp thời gian tương ứng.

Thao tác	Mô tả	Độ phức tạp
<code>addQuestion(q)</code>	Thêm câu hỏi vào cuối danh sách	$O(1)$
<code>removeQuestion(id)</code>	Xoá câu hỏi theo ID	$O(n)$
<code>getQuestionCount()</code>	Đếm số câu hỏi	$O(n)$
<code>getQuestionAt(index)</code>	Lấy câu hỏi tại vị trí index	$O(n)$
<code>printAll()</code>	In toàn bộ danh sách	$O(n)$
<code>generateRandomSet(n)</code>	Bốc ngẫu nhiên n câu	$O(n)$

Bảng 2.1: Các thao tác trên danh sách liên kết đơn `QuestionBank`

2.3.4 Cài đặt thêm và xoá node

Thao tác thêm node vào cuối (dùng con trỏ `tail`) có thể thực hiện trong $O(1)$ mà không cần duyệt toàn bộ danh sách. Thao tác xoá node theo ID cần xử lý ba trường hợp: danh sách rỗng, node cần xoá là HEAD, và node cần xoá ở giữa hoặc cuối.

```

1 void QuestionBank::addQuestion(Question q) {
2     QuestionNode* newNode = new QuestionNode();
3     newNode->data = q;
4     newNode->next = nullptr;
5     if (tail == nullptr) {
6         head = newNode; // Danh sách rỗng
7         tail = newNode;
8     } else {
9         tail->next = newNode;
10        tail = newNode;
11    }
12 }
```

Listing 2.4: Thêm node vào cuối danh sách liên kết đơn

2.4 Thuật toán xáo trộn Fisher-Yates

2.4.1 Nguyên lý hoạt động

Thuật toán Fisher-Yates (còn gọi là Knuth Shuffle) là một thuật toán xáo trộn mảng *in-place* cho phân phối đều (uniform distribution), nghĩa là mọi hoán vị của mảng đều có xác suất xuất hiện bằng nhau [8]. Thuật toán hoạt động theo nguyên tắc: duyệt mảng từ cuối về đầu, tại mỗi vị trí i , chọn ngẫu nhiên một vị trí j trong đoạn $[0, i]$ và hoán đổi phần tử tại i và j .

```

1 void Exam::shuffleQuestions(Question arr[], int n) {
2     for (int i = n - 1; i > 0; --i) {
3         int j = rand() % (i + 1); // j ngẫu nhiên trong [0, i]
4         Question temp = arr[i];
5         arr[i] = arr[j];
6         arr[j] = temp;
7     }
8 }
```

Listing 2.5: Thuật toán Fisher-Yates xáo trộn mảng câu hỏi

2.4.2 Độ phức tạp

Thuật toán Fisher-Yates có độ phức tạp thời gian $O(n)$ và không gian $O(1)$, tốt hơn nhiều so với các cách tiếp cận ngây thơ như chọn phần tử ngẫu nhiên và kiểm tra trùng lặp (có độ phức tạp trung bình $O(n^2)$) [8].

2.4.3 Ứng dụng trong dự án

Trong chương trình, Fisher-Yates được áp dụng ở hai cấp độ. Thứ nhất, xáo trộn thứ tự các câu hỏi trong đề thi để mỗi sinh viên nhận được đề có trình tự câu hỏi khác nhau. Thứ hai, với

mỗi câu hỏi, xáo trộn thứ tự bốn đáp án (A, B, C, D) và cập nhật lại trường `correct` để phản ánh đáp án đúng trong sắp xếp mới.

```

1 void Exam::shuffleExam() {
2     shuffleQuestions(questionList, numberOfQuestions);
3     for (int i = 0; i < numberOfQuestions; ++i) {
4         Question& q = questionList[i];
5         // Ghi nhớ nội dung đáp án đúng trước khi xáo
6         string correctText = q.answers[q.correct - 'A'];
7         shuffleAnswers(q.answers, 4);
8         // Cập nhật lại vị trí đáp án đúng sau khi xáo
9         for (int k = 0; k < 4; ++k) {
10             if (q.answers[k] == correctText) {
11                 q.correct = static_cast<char>('A' + k);
12                 break;
13             }
14         }
15     }
16 }

```

Listing 2.6: Xáo trộn đáp án và cập nhật đáp án đúng

2.5 Xử lý File I/O trong C++

2.5.1 Các lớp stream cơ bản

C++ cung cấp thư viện `<fstream>` với hai lớp chính cho File I/O: `ifstream` (đọc file) và `ofstream` (ghi file) [9]. Để xử lý chuỗi văn bản phân tách, nhóm kết hợp với `std::istringstream` từ thư viện `<sstream>` giúp phân tách từng trường dữ liệu theo ký tự `|`.

2.5.2 Định dạng dữ liệu

Nhóm lựa chọn định dạng file văn bản phẳng (plain text) với ký tự phân tách `|` cho cả hai file dữ liệu. Định dạng này có thể đọc bằng bất kỳ trình soạn thảo văn bản nào, dễ kiểm tra bằng tay khi debug, và không yêu cầu thư viện bên ngoài [9].

Định dạng file `questions.txt`:

```
id|NoiDungCauHoi|DapAnA|DapAnB|DapAnC|DapAnD|DapAnDung
```

Định dạng file `history.txt`:

```
Username|SoCauDung|TongSoCau|Diem|Timestamp
```

2.5.3 Đọc file với xử lý lỗi

Một thực hành tốt khi làm việc với File I/O là luôn kiểm tra xem file có mở thành công không trước khi thao tác, và xử lý từng dòng với cơ chế bắt lỗi để bỏ qua các dòng không hợp lệ thay vì làm crash toàn bộ chương trình [9].

```

1 bool QuestionBank::loadFromFile(string filename) {
2     ifstream fin(filename);
3     if (!fin.is_open()) {
4         cout << "[Loi] Khong the mo file: " << filename << "\n";
5         return false;
6     }
7     string line;
8     while (getline(fin, line)) {
9         if (line.empty() || line[0] == '#') continue;
10        istringstream ss(line);
11        Question q;
12        string token;
13        if (!getline(ss, token, '|')) continue;

```



```

14         q.id = stoi(token);
15         // ... phân tích tung trường theo định dạng
16         addQuestion(q);
17     }
18     fin.close();
19     return true;
20 }

```

Listing 2.7: Đọc file câu hỏi với kiểm tra lỗi từng dòng

2.6 Lập trình hướng đối tượng trong C++

2.6.1 Class, đóng gói và trừu tượng hoá

Lập trình hướng đối tượng (OOP) tổ chức mã nguồn xung quanh các *đối tượng* — các thực thể kết hợp dữ liệu (thuộc tính) và hành vi (phương thức) thành một đơn vị thống nhất [9]. Nguyên lý *đóng gói* (encapsulation) yêu cầu che giấu các chi tiết cài đặt bên trong class, chỉ để lộ ra giao diện công khai cần thiết. Điều này giảm sự phụ thuộc giữa các module và giúp thay đổi cài đặt nội bộ mà không ảnh hưởng đến code bên ngoài.

Trong dự án, cả ba class chính (QuestionBank, HistoryManager, Exam) đều có thành viên `private` (con trỏ head, tail, mảng questionList) chỉ được truy cập thông qua các phương thức `public`.

2.6.2 Tách khai báo và cài đặt (Header/Source)

Quy ước phổ biến trong C++ là tách khai báo (declaration) vào file header `.h` và cài đặt (definition) vào file source `.cpp` [9]. File header chứa signature của các hàm và định nghĩa struct/class, cho phép các module khác `#include` mà không cần biết chi tiết cài đặt. Cơ chế `#pragma once` hoặc include guard ngăn chặn việc include trùng lặp gây lỗi biên dịch.

Cấu trúc này mang lại nhiều lợi ích: giảm thời gian biên dịch lại khi chỉ thay đổi cài đặt, phân công công việc rõ ràng giữa các thành viên nhóm, và tạo ra một “hợp đồng” (contract) tường minh về giao diện của mỗi module [9].

File Header	File Source	Nội dung
QuestionBank.h	QuestionBank.cpp	Struct Question, QuestionNode; class QuestionBank với các thao tác thêm, xóa, đọc, ghi
History.h	History.cpp	Struct TestRecord, HistoryNode; class HistoryManager lưu trữ và truy vấn lịch sử thi
Exam.h	Exam.cpp	Class Exam với logic sinh đề, xáo trộn, đếm giờ và chấm điểm
Menu.h	Menu.cpp	Struct LoginSession; các hàm giao diện Terminal, đăng nhập, menu Admin, menu Sinh viên

Bảng 2.2: Cấu trúc header/source của các module trong dự án

2.6.3 Validate input và lập trình phòng ngừa

Một chương trình tốt không chỉ hoạt động đúng với đầu vào hợp lệ mà còn xử lý ổn thoả với đầu vào không hợp lệ [10]. Kỹ thuật *lập trình phòng ngừa* (defensive programming) yêu cầu kiểm tra mọi giá trị đầu vào từ người dùng trước khi sử dụng. Trong dự án, hàm `getValidInt()` đọc số nguyên trong đoạn `[min, max]` với vòng lặp bẫy lỗi hoàn toàn: khi người dùng nhập ký tự không phải số, `cin.clear()` và `cin.ignore()` được gọi để xóa trạng thái lỗi và bộ đệm, sau đó yêu cầu nhập lại.

2.7 Tổng kết

Chương này đã trình bày sáu nhóm kiến thức lý thuyết làm nền tảng cho dự án: nguyên lý thiết kế theo module và top-down, con trỏ và quản lý bộ nhớ động, danh sách liên kết đơn, thuật toán Fisher-Yates, xử lý File I/O và lập trình hướng đối tượng trong C++. Bảng 2.3 tóm tắt mối liên hệ giữa từng kiến thức lý thuyết và phần cài đặt tương ứng trong dự án.

Kiến thức lý thuyết	Ứng dụng trong dự án
Thiết kế module / top-down	Phân chia thành 4 module độc lập; <code>main.cpp</code> chỉ điều phối luồng chính
Con trỏ / bộ nhớ động	Mảng <code>Question[]</code> và <code>char[]</code> trong class <code>Exam</code> ; cấp phát bằng <code>new[]</code> , giải phóng trong Destructor
Danh sách liên kết đơn	<code>QuestionBank</code> và <code>HistoryManager</code> lưu trữ dữ liệu động không giới hạn kích thước
Fisher-Yates Shuffle	Xáo trộn thứ tự câu hỏi và đáp án trong <code>Exam::shuffleExam()</code>
File I/O (<code>fstream</code>)	Đọc/ghi <code>questions.txt</code> và <code>history.txt</code> với định dạng phân tách
OOP (class, header/source)	3 class với đóng gói đầy đủ; tách khai báo (<code>.h</code>) và cài đặt (<code>.cpp</code>)
Validate input	Hàm <code>getValidInt()</code> , <code>getValidString()</code> , <code>getYesNo()</code> với vòng lặp bắt lỗi

Bảng 2.3: Tổng kết liên hệ lý thuyết và cài đặt trong dự án

Chương 3

Phân tích yêu cầu hệ thống

3.1 Khảo sát bài toán

3.1.1 Phương pháp khảo sát

Trước khi xác định danh sách yêu cầu cụ thể, nhóm tiến hành khảo sát bài toán theo ba phương pháp kết hợp:

- **Đóng vai:** các thành viên đóng vai giảng viên ra đề và sinh viên dự thi để mô phỏng lại toàn bộ quy trình thi trắc nghiệm trên giấy truyền thống, từ đó ghi nhận các bước thao tác lặp lại nhiều lần (soạn đề, in ấn, coi thi, thu bài, chấm điểm, tổng hợp điểm) cùng các điểm dễ phát sinh sai sót (chấm nhầm, thất lạc bài thi, mất nhiều thời gian tổng hợp kết quả).
- **Tham khảo cấu trúc đề thi thực tế:** nhóm khảo sát cấu trúc đề thi trắc nghiệm Tiếng Anh - môn học được chọn làm minh hoạ chính cho chương trình - để xác định các thuộc tính cần thiết của một câu hỏi: nội dung, bốn phương án lựa chọn, đáp án đúng, mức độ khó (dễ/trung bình/khó) và môn học được chia theo các cấp bậc lớp học.
- **Đối chiếu yêu cầu môn học:** nhóm tổng hợp các ràng buộc kỹ thuật do môn học Kỹ thuật Lập trình (MI3310) đặt ra - không sử dụng cấu trúc dữ liệu nâng cao, tự cài đặt cấu trúc dữ liệu bằng con trỏ - làm đầu vào song song cho giai đoạn thiết kế dữ liệu ở mục 3.2.

3.1.2 Đối tượng sử dụng hệ thống

Từ kết quả khảo sát, nhóm xác định hệ thống có hai tác nhân chính, được phân quyền ngay tại bước đăng nhập thông qua trường `isAdmin` trong `LoginSession` (xem `Menu.h`). Bảng 3.1 tóm tắt vai trò và quyền hạn của từng tác nhân.

Tác nhân	Vai trò	Quyền hạn chính
Admin (Quản trị viên)	Người quản lý nội dung câu hỏi, không trực tiếp tham gia thi	Thêm/xoá câu hỏi, xem toàn bộ ngân hàng câu hỏi theo môn, xem lịch sử thi của tất cả sinh viên
Sinh viên	Người dự thi	Chọn môn học và cấu hình đề thi, làm bài thi có đếm giờ, xem lịch sử thi của chính mình

Bảng 3.1: Tác nhân và quyền hạn trong hệ thống

Việc phân quyền chặt theo tác nhân giúp đơn giản hoá luồng điều hướng Menu (mỗi tác nhân chỉ nhìn thấy một bộ chức năng phù hợp với vai trò), đồng thời tránh nguy cơ sinh viên vô tình hoặc cố ý thay đổi ngân hàng câu hỏi.

3.1.3 Kết quả khảo sát

Khảo sát giúp nhóm xác định được:

- 5 nhóm yêu cầu chức năng chính, tương ứng bốn module **QuestionBank**, **Exam**, **History** và **Menu** đã giới thiệu ở Chương 2 (trình bày chi tiết tại Mục 3.2).
- Các yêu cầu phi chức năng bắt buộc do đặc thù môn học và mục tiêu sản phẩm đặt ra (trình bày tại Mục 3.3).
- Một ràng buộc nghiệp vụ quan trọng cần được làm rõ ngay từ giai đoạn phân tích: đề thi phải được sinh theo tỉ lệ độ khó cố định (40% Dễ – 40% Trung bình – 20% Khó) thay vì bốc ngẫu nhiên hoàn toàn, nhằm đảm bảo tính công bằng về độ khó giữa các đề thi khác nhau của cùng một môn học.

3.2 Yêu cầu chức năng

Dựa trên kết quả khảo sát, nhóm đặc tả các yêu cầu chức năng - mô tả *hệ thống phải làm gì* - thành 5 nhóm chính, được mã hoá từ FR-01 đến FR-12 và tóm tắt tại Bảng 3.2.

Mã	Tên yêu cầu	Tác nhân
FR-01	Đăng nhập và phân quyền Admin / Sinh viên	Admin, Sinh viên
FR-02	Thêm câu hỏi mới vào ngân hàng	Admin
FR-03	Xoá câu hỏi theo ID	Admin
FR-04	Xem danh sách câu hỏi (toàn bộ / theo môn học)	Admin
FR-05	Lưu trữ ngân hàng câu hỏi trên file	Admin, Hệ thống
FR-06	Chọn môn học và cấu hình đề thi (số câu, thời gian)	Sinh viên
FR-07	Sinh đề thi theo tỉ lệ độ khó 40-40-20 và xáo trộn	Hệ thống
FR-08	Làm bài thi tuần tự có đếm giờ thời gian thực	Sinh viên
FR-09	Nộp bài (chủ động hoặc tự động khi hết giờ)	Sinh viên, Hệ thống
FR-10	Chấm điểm tự động theo thang 10	Hệ thống
FR-11	Hiển thị kết quả sau khi nộp bài	Hệ thống
FR-12	Lưu và truy xuất lịch sử thi	Admin, Sinh viên

Bảng 3.2: Tổng hợp các yêu cầu chức năng của hệ thống

FR-01 (đăng nhập, phân quyền) đã được mô tả tại Mục 3.1; các mục tiếp theo trình bày chi tiết FR-02 đến FR-12, gom theo 5 nhóm chức năng.

3.2.1 Quản lý ngân hàng câu hỏi

Mô tả chức năng

Nhóm chức năng này cho phép Admin duy trì hệ thống thông qua ba nghiệp vụ: thêm câu hỏi (FR-02), xoá câu hỏi (FR-03) và xem danh sách câu hỏi (FR-04); toàn bộ thay đổi phải được đồng bộ ngay xuống file `questions.txt` (FR-05) để không mất dữ liệu khi tắt chương trình.

Đặc tả dữ liệu câu hỏi

Mỗi câu hỏi được mô hình hoá bởi cấu trúc **Question** với các ràng buộc hợp lệ liệt kê tại Bảng 3.3. Các ràng buộc này được kiểm tra ngay tại thời điểm Admin nhập liệu (qua `getValidString`, `getValidInt`) trước khi cho phép ghi vào ngân hàng.

Trường	Kiểu dữ liệu	Ràng buộc hợp lệ
id	int	Số nguyên dương, <i>duy nhất</i> trong toàn ngân hàng
subject	string	Không được để trống
difficulty	string	Chỉ nhận một trong ba giá trị: Easy , Medium , Hard
content	string	Không được để trống
answers[0..3]	string	Đủ bốn đáp án A, B, C, D, không trống
correct	char	Chỉ nhận một trong bốn giá trị: A , B , C , D

Bảng 3.3: Đặc tả ràng buộc dữ liệu của một câu hỏi

Quy tắc nghiệp vụ

- **Thêm câu hỏi (FR-02):** hệ thống kiểm tra ID nhập vào đã tồn tại hay chưa trước khi cho nhập các trường còn lại; nếu trùng, yêu cầu nhập lại ID khác. Sau khi nhập đủ thông tin hợp lệ, câu hỏi được nối vào cuối danh sách và file `questions.txt` được ghi đè lại toàn bộ.
- **Xoá câu hỏi (FR-03):** Admin nhập ID cần xoá; hệ thống xử lý ba tình huống (danh sách rỗng, ID là câu hỏi đầu danh sách, ID nằm giữa/cuối danh sách) và báo lỗi rõ ràng nếu không tìm thấy ID.
- **Xem danh sách (FR-04):** hệ thống liệt kê trước danh sách các môn học hiện có (không lặp môn) để Admin lựa chọn xem theo môn cụ thể hoặc xem toàn bộ (nhập **ALL**); nếu môn nhập vào không tồn tại, yêu cầu nhập lại.
- **Lưu trữ danh sách câu hỏi (FR-05):** ngân hàng câu hỏi được nạp từ file ngay khi vào Menu Admin và được ghi lại file ngay sau mỗi thao tác thêm/xoá thành công, đảm bảo dữ liệu trong bộ nhớ và trên đĩa luôn đồng bộ.

3.2.2 Cấu hình và sinh đề thi

Mô tả chức năng

Trước khi vào thi, sinh viên cần chọn môn học và cấu hình đề thi theo nhu cầu cá nhân (FR-06); hệ thống chịu trách nhiệm sinh đề tự động đúng tỉ lệ độ khó đã quy định và xáo trộn để đảm bảo công bằng (FR-07).

Luồng xử lý

1. Hệ thống nạp ngân hàng câu hỏi từ file và liệt kê các môn học khác nhau hiện có để sinh viên lựa chọn.
2. Hệ thống đếm số câu hỏi của môn đã chọn theo từng mức độ (n_{Easy} , n_{Medium} , n_{Hard}).
3. Hệ thống tính số câu hỏi *tối đa* có thể tổ chức thi mà vẫn giữ đúng tỉ lệ 40-40-20 (xem Bảng 3.4), gọi là N_{max} .
4. Sinh viên nhập số câu muốn thi trong khoảng $[1, N_{max}]$ và thời gian làm bài (giây, tối thiểu 10 giây, tối đa 36 000 giây).
5. Sinh viên xác nhận bắt đầu thi (Y/N); nếu chọn N, huỷ và quay lại Menu Sinh viên.
6. Hệ thống trích đúng số câu Easy/Medium/Hard theo tỉ lệ, xáo trộn ngẫu nhiên trong từng nhóm bằng thuật toán Fisher-Yates trước khi chọn, sau đó trộn lẫn thứ tự xuất hiện giữa ba nhóm.

Quy tắc nghiệp vụ

Mức độ	Tỉ lệ	Cách tính số câu cần
Dễ (Easy)	40%	$\lfloor N \times 40/100 \rfloor$
Trung bình (Medium)	40%	$\lfloor N \times 40/100 \rfloor$
Khó (Hard)	20%	$N - n_{Easy} - n_{Medium}$ (phần còn lại)

Bảng 3.4: Quy tắc tính số câu hỏi theo tỉ lệ độ khó 40-40-20

Số câu khó được tính bằng phần còn lại (không chia trực tiếp $N \times 20/100$) để đảm bảo tổng luôn đúng bằng N dù phép chia lấy phần nguyên có làm tròn xuống ở hai nhóm trước. Nếu ngân hàng câu hỏi của môn đã chọn không đủ số câu cần theo bất kỳ mức độ nào trong tỉ lệ trên, hệ thống phải từ chối sinh đề và thông báo lỗi cụ thể theo môn học, tuyệt đối không sinh đề thiếu câu hoặc sai tỉ lệ.

3.2.3 Tiến hành thi và đếm giờ thời gian thực

Mô tả chức năng

Chức năng này (FR-08, FR-09) mô phỏng lại một buổi thi thật: hiển thị câu hỏi tuần tự, tiếp nhận đáp án, đồng thời theo dõi thời gian làm bài còn lại theo thời gian thực và tự động thu bài khi cần.

Luồng xử lý

- Trước khi hiển thị mỗi câu hỏi, hệ thống tính thời gian còn lại dựa trên mốc thời gian bắt đầu thi; nếu thời gian đã hết, lập tức dừng và thu bài, không hiển thị thêm câu hỏi nào nữa.
- Với mỗi câu hỏi, hệ thống hiển thị nội dung và bốn đáp án A/B/C/D, kèm dòng thông báo thời gian còn lại.
- Sinh viên nhập một trong các ký tự A, B, C, D để chọn đáp án, hoặc nhập S để nộp bài sớm ngay tại câu đang làm; mọi ký tự khác bị từ chối và yêu cầu nhập lại.
- Ngay sau khi sinh viên nhập xong đáp án cho một câu, hệ thống kiểm tra lại thời gian còn lại lần thứ hai, phòng trường hợp sinh viên cố tình kéo dài thời gian suy nghĩ để “lách” giới hạn ở câu cuối cùng.
- Bài thi kết thúc khi xảy ra một trong hai điều kiện: sinh viên đã trả lời (hoặc bỏ qua) hết toàn bộ số câu, hoặc thời gian làm bài đã hết / sinh viên chủ động nộp bài sớm.

Quy tắc nghiệp vụ

Câu hỏi chưa được trả lời tại thời điểm nộp bài (do hết giờ hoặc nộp sớm) được ghi nhận ở trạng thái riêng (*chưa trả lời*), tách biệt với trạng thái *trả lời sai*, để phản ánh đúng tình huống thực tế khi tổng hợp kết quả.

3.2.4 Chấm điểm tự động và hiển thị kết quả

Mô tả chức năng

Ngay sau khi bài thi kết thúc (FR-09), hệ thống phải tự động chấm điểm (FR-10) và hiển thị kết quả chi tiết (FR-11) mà không cần sự can thiệp của Admin hay bất kỳ thao tác thủ công nào.

Quy tắc nghiệp vụ

- Điểm số được tính theo thang điểm 10, theo công thức:

$$\text{Điểm} = \frac{\text{Số câu đúng}}{\text{Tổng số câu}} \times 10$$

Trường hợp đề thi không có câu hỏi nào (tổng số câu bằng 0), điểm số được quy định bằng 0 để tránh lỗi chia cho 0.

- Bảng tổng kết cuối bài hiển thị: tổng số câu đúng trên tổng số câu, điểm số (định dạng hai chữ số sau dấu phẩy) và thời điểm nộp bài (timestamp dạng YYYY-MM-DD HH:MM:SS).

3.2.5 Xem lịch sử thi

Mô tả chức năng

Chức năng này (FR-12) đảm bảo kết quả mỗi lần thi được lưu trữ lâu dài và có thể truy xuất lại sau, phục vụ cả nhu cầu của sinh viên (theo dõi tiến độ cá nhân) và Admin (theo dõi tổng quan lớp học).

Quy tắc nghiệp vụ

- Ngay sau khi một sinh viên hoàn thành bài thi, hệ thống tạo một bản ghi lịch sử gồm: tên sinh viên, môn học, số câu đúng, tổng số câu, điểm số và thời gian thi, rồi nối thêm (không ghi đè) bản ghi này vào file lịch sử.
- Sinh viên chỉ được xem lịch sử thi của *chính mình*, xác định theo tên đăng nhập hiện tại; nếu chưa từng thi lần nào, hệ thống thông báo rõ thay vì hiển thị danh sách trống không lời giải thích.
- Admin được xem lịch sử thi của *toàn bộ* sinh viên, không giới hạn theo tên đăng nhập, nhằm phục vụ mục đích theo dõi tổng thể.

3.3 Yêu cầu phi chức năng

Bên cạnh các yêu cầu chức năng, hệ thống còn phải thoả mãn các yêu cầu phi chức năng - mô tả *hệ thống phải hoạt động như thế nào*, thường liên quan đến chất lượng, hiệu năng và các ràng buộc kỹ thuật. Bảng 3.5 tổng hợp các yêu cầu phi chức năng được nhóm xác định cho dự án.

Tiêu chí	Mô tả yêu cầu
Hiệu năng	Mọi thao tác (thêm/xoá câu hỏi, sinh đề, chấm điểm) phải phản hồi gần như tức thì trên Terminal với ngân hàng câu hỏi cỡ vài trăm đến vài nghìn câu, phù hợp với độ phức tạp $O(n)$ của các thao tác trên danh sách liên kết đơn.
Độ tin cậy	Chương trình không được crash với bất kỳ đầu vào không hợp lệ nào từ người dùng (chuỗi rỗng, ký tự không phải số, giá trị ngoài khoảng cho phép,...); mọi lỗi phải được bắt và yêu cầu nhập lại.
Tính khả chuyển	Chương trình phải biên dịch và chạy được trên cả Windows (MinGW g++) và Linux/macOS (g++), chỉ sử dụng các thư viện chuẩn của C++17.
Tính bảo trì	Mã nguồn được tổ chức theo 4 module độc lập (QuestionBank, Exam, History, Menu), tách riêng header/source, cho phép sửa đổi một module mà không ảnh hưởng các module còn lại.
An toàn bộ nhớ	Không phát sinh rò rỉ bộ nhớ (memory leak) trong toàn bộ thời gian chạy chương trình; mọi vùng nhớ cấp phát động bằng <code>new/new[]</code> phải được giải phóng tương ứng.
Tính bền dữ liệu	Ngân hàng câu hỏi và lịch sử thi phải tồn tại bền vững trên file văn bản, không mất dữ liệu khi tắt và mở lại chương trình; riêng lịch sử thi không bị ghi đè giữa các lần thi khác nhau.
Tính khả dụng	Giao diện Terminal phải rõ ràng, có phân biệt màu sắc giữa thông báo thành công/cảnh báo/lỗi, có tạm dừng màn hình hợp lý giữa các bước để người dùng đọc kết quả.
Tính an toàn	Cơ chế xác thực đơn giản (không mã hoá mật khẩu) chỉ phục vụ mục đích minh hoạ học thuật trong phạm vi bài tập lớn.
Tính nhất quán dữ liệu	Dữ liệu trong bộ nhớ (danh sách liên kết) và dữ liệu trên file phải luôn đồng bộ ngay sau mỗi thao tác làm thay đổi ngân hàng câu hỏi.

Bảng 3.5: Tổng hợp các yêu cầu phi chức năng của hệ thống

3.4 Mô tả luồng hoạt động của chương trình

Mục này mô tả luồng hoạt động ở mức tổng quan (góc nhìn người dùng), làm cơ sở cho việc thiết kế kiến trúc chi tiết ở Chương 4.

Luồng tổng thể

```
main()
|
|-- showWelcomeBanner()           // Màn hình chào mừng
|
|-- Vòng lặp chính (while appRunning):
|   |
|   |-- showLoginScreen()
|   |   |
|   |   |-- Nhập "exit"          --> LoginResult::Exit
|   |   |-- Sai thông tin       --> LoginResult::InvalidCredentials (nhập lại)
|   |   |-- Đúng thông tin      --> LoginResult::Success
|   |   |
|   |   |-- Neu Success:
|   |       |-- isAdmin = true   --> showAdminMenu()
|   |       |-- isAdmin = false --> showStudentMenu()
|   |   |-- Neu Exit --> appRunning = false, thoát vòng lặp
|   |-- Man hình tam biệt
```

Thiết kế vòng lặp đăng nhập tại `main()` (thay vì xử lý một lần) cho phép nhiều người dùng đăng nhập, đăng xuất kế tiếp nhau trong cùng một lần chạy chương trình, không cần khởi động lại mỗi khi đổi tài khoản.

Luồng Admin

Sau khi đăng nhập thành công với quyền Admin, người dùng được đưa vào một vòng lặp menu lặp lại cho đến khi chọn đăng xuất:

1. Hệ thống nạp ngân hàng câu hỏi từ `questions.txt` ngay khi vào Menu Admin.
2. Admin chọn một trong các chức năng: xem danh sách câu hỏi, thêm câu hỏi, xoá câu hỏi theo ID, xem lịch sử thi của tất cả sinh viên, hoặc đăng xuất.
3. Với các chức năng làm thay đổi dữ liệu (thêm/xoá), hệ thống ghi lại file `questions.txt` ngay sau khi thao tác thành công.
4. Khi Admin chọn đăng xuất, vòng lặp Menu Admin kết thúc và chương trình quay lại màn hình đăng nhập tại `main()`.

Luồng Sinh viên

Tương tự, sau khi đăng nhập với quyền sinh viên, người dùng được đưa vào vòng lặp Menu sinh viên với ba lựa chọn: bắt đầu làm bài thi, xem lịch sử thi cá nhân, hoặc đăng xuất. Luồng Bắt đầu làm bài thi - luồng nghiệp vụ phức tạp nhất của chương trình được mô tả chi tiết qua các bước sau:

1. Nạp ngân hàng câu hỏi từ file; nếu ngân hàng rỗng, thông báo lỗi và quay lại Menu.
2. Liệt kê các môn học hiện có; sinh viên chọn một môn để thi (FR-06).
3. Tính số câu hỏi tối đa hợp lệ theo tỉ lệ 40-40-20 cho môn đã chọn; nếu không có N hợp lệ nào (thiếu hẳn một mức độ), thông báo lỗi và quay lại Menu.
4. Sinh viên nhập số câu muốn thi và thời gian làm bài; xác nhận bắt đầu.
5. Hệ thống sinh đề theo tỉ lệ độ khó, xáo trộn câu hỏi và đáp án (FR-07).
6. Sinh viên làm bài với đếm giờ thời gian thực (FR-08, FR-09).
7. Hệ thống chấm điểm và hiển thị kết quả (FR-10, FR-11).
8. Hệ thống lưu kết quả vào lịch sử thi (FR-12); sinh viên quay lại Menu Sinh viên.

Hai luồng Admin và Sinh viên được cài đặt độc lập trong hai hàm riêng (`showAdminMenu`, `showStudentMenu`), nhưng đều tuân theo cùng một khuôn mẫu thiết kế: vòng lặp `while` kết hợp `switch-case`, giúp dễ dàng mở rộng thêm chức năng mới ở các phiên bản sau mà không ảnh hưởng tới luồng đã có, nội dung này được trình bày chi tiết hơn dưới góc độ kiến trúc tại Chương 4.

Chương 4

Thiết kế hệ thống

Dựa trên các yêu cầu chức năng và phi chức năng đã đặc tả ở Chương 3, chương này trình bày thiết kế cụ thể của hệ thống: kiến trúc tổng thể và sự phân chia module, cấu trúc dữ liệu lưu trữ trên file, và thiết kế chi tiết của từng module chức năng (**Menu**, **QuestionBank**, **Exam**, **History**) cùng hàm `main()` đóng vai trò điều phối. Thiết kế ở chương này là cơ sở trực tiếp để cài đặt mã nguồn được trình bày tại Chương 5.

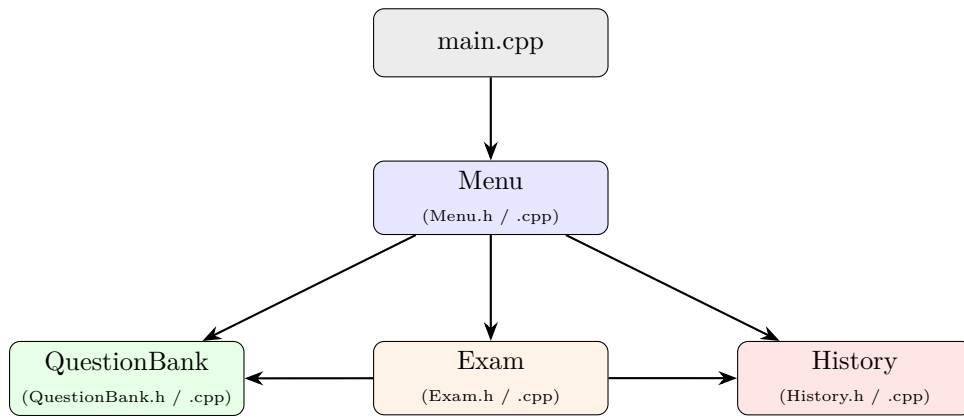
4.1 Kiến trúc tổng thể chương trình

4.1.1 Sơ đồ phân chia module

Theo nguyên lý module hoá đã trình bày ở Mục 2, chương trình được tổ chức thành bốn module độc lập, mỗi module đảm nhiệm một nhóm nghiệp vụ rõ ràng và không chồng chéo trách nhiệm:

- **QuestionBank** – quản lý ngân hàng câu hỏi: lưu trữ, thêm, xoá, tìm kiếm và đọc/ghi file `questions.txt`.
- **Exam** – logic của một lượt thi: sinh đề theo tỉ lệ độ khó, xáo trộn Fisher-Yates, đếm giờ thời gian thực và chấm điểm.
- **History** – quản lý lịch sử thi: lưu trữ và truy vấn các bản ghi `TestRecord`, đọc/ghi file `history.txt`.
- **Menu** – giao diện Terminal và điều hướng: đăng nhập, menu Admin, menu sinh viên, các hàm validate input.

Quan hệ phụ thuộc giữa các module được thể hiện qua khai báo `#include: Exam.h` bao gồm trực tiếp `QuestionBank.h` (vì `generateExamFromBank` nhận tham số `QuestionBank&`) và `History.h` (vì `startExam` trả về `TestRecord`); `Menu.cpp` bao gồm cả ba header còn lại để điều phối luồng nghiệp vụ. Ngược lại, `QuestionBank` và `History` hoàn toàn độc lập với hai module kia, đây là minh chứng cho việc tách bạch lớp dữ liệu khỏi lớp logic nghiệp vụ và lớp giao diện, giúp có thể kiểm thử `QuestionBank/History` riêng biệt mà không cần khởi tạo `Exam` hay `Menu`.



Hình 4.1: Sơ đồ phân chia module và quan hệ phụ thuộc (mũi tên: “sử dụng/include”)

Như Hình 4.1 thể hiện, `main.cpp` không gọi trực tiếp `QuestionBank`, `Exam` hay `History`, toàn bộ nghiệp vụ được uỷ quyền cho `Menu`, giúp `main.cpp` chỉ còn đóng vai trò “điểm vào” cực ngắn, đúng tư tưởng thiết kế top-down đã trình bày ở Chương 2.

4.1.2 Luồng dữ liệu giữa các module

Dữ liệu trong hệ thống di chuyển theo hai luồng chính, tương ứng với hai tác nhân Admin và sinh viên. Bảng 4.1 tóm tắt luồng dữ liệu giữa các module ứng với từng chức năng.

Bước	Luồng dữ liệu
1	Menu gọi <code>QuestionBank::loadFromFile("data/questions.txt")</code> để nạp toàn bộ câu hỏi vào danh sách liên kết đơn trong bộ nhớ.
2	(Admin) Menu nhận Question mới từ người dùng qua <code>getValidString/getValidInt</code> , gọi <code>QuestionBank::addQuestion()</code> rồi <code>saveToFile()</code> để đồng bộ lại file.
3	(Sinh viên) Menu khởi tạo đối tượng Exam, truyền tham chiếu <code>QuestionBank&</code> cho <code>generateExamFromBank()</code> ; Exam gọi ngược lại <code>QuestionBank::getQuestionsBySubjectAndDifficulty()</code> để lấy ba mảng câu hỏi Easy/Medium/Hard.
4	Exam tự xáo trộn nội bộ (Fisher-Yates) và trả quyền điều khiển lại cho Menu thông qua <code>startExam()</code> , hàm này trả về một đối tượng <code>TestRecord</code> (định nghĩa trong <code>History.h</code>).
5	Menu nhận <code>TestRecord</code> , gọi <code>HistoryManager::addRecord()</code> rồi <code>saveToFile()</code> (chế độ append) để ghi nối vào <code>data/history.txt</code> .
6	Khi xem lịch sử, Menu gọi <code>HistoryManager::loadFromFile()</code> để nạp lại toàn bộ lịch sử từ file, sau đó gọi <code>printAll()</code> (Admin) hoặc <code>printByUser()</code> (Sinh viên).

Bảng 4.1: Luồng dữ liệu giữa các module ứng với từng bước nghiệp vụ

Một điểm thiết kế quan trọng cần lưu ý: `QuestionBank` và `HistoryManager` *không* được giữ lại như biến toàn cục; mỗi lần vào một màn hình Menu, đối tượng tương ứng được khai báo cục bộ và nạp lại từ file. Cách thiết kế này (đánh đổi một phần hiệu năng I/O) giúp đảm bảo dữ liệu trong bộ nhớ luôn phản ánh đúng trạng thái mới nhất, đặc biệt quan trọng khi nhiều lượt đăng nhập Admin/sinh viên có thể nối tiếp nhau trong cùng một lần chạy chương trình.

4.1.3 Sơ đồ luồng điều hướng Menu

Luồng điều hướng được thiết kế theo mô hình “vòng lặp đăng nhập tại `main()`” kết hợp với hai vòng lặp Menu con (Admin/Sinh viên) độc lập, mỗi vòng lặp dùng cấu trúc `switch-case` trên lựa chọn của người dùng. Sơ đồ tổng quát:

```

main()
|-- show WelcomeBanner()
|-- while (appRunning):
|   |-- showLoginScreen(session)
|   |   |-- "exit"                -> LoginResult::Exit
|   |   |-- sai user/pass         -> LoginResult::InvalidCredentials (nhập lại)
|   |   |-- dung user/pass        -> LoginResult::Success
|   |-- switch(result):
|   |   |-- Success & isAdmin=true -> showAdminMenu(session)
|   |   |-- Success & isAdmin=false -> showStudentMenu(session)
|   |   |-- InvalidCredentials      -> (lap lai showLoginScreen)
|   |   |-- Exit                   -> appRunning = false
|-- Man hinh tam biet, return 0

```

Bên trong `showAdminMenu`, vòng lặp `while(running)` hiển thị 5 lựa chọn (xem Bảng 4.2); bên trong `showStudentMenu`, vòng lặp tương tự hiển thị 3 lựa chọn (xem Bảng 4.3). Cả hai đều thoát vòng lặp bằng cách đặt `running = false` khi người dùng chọn “Đăng xuất”, trả luồng điều khiển về lại vòng lặp đăng nhập tại `main()`, đây chính là cơ chế cho phép đổi tài khoản liên tục mà không cần khởi động lại chương trình.

Lựa chọn	Hành động
1	Xem danh sách câu hỏi (chọn môn cụ thể hoặc nhập ALL để xem toàn bộ)
2	Thêm câu hỏi mới (validate ID, môn học, độ khó, nội dung, 4 đáp án, đáp án đúng)
3	Xoá câu hỏi theo ID
4	Xem lịch sử thi của toàn bộ sinh viên
5	Đăng xuất

Bảng 4.2: Các lựa chọn trong `showAdminMenu`

Lựa chọn	Hành động
1	Bắt đầu làm bài thi (chọn môn → nhập số câu/thời gian → xác nhận → sinh đề → làm bài → lưu lịch sử)
2	Xem lịch sử thi của riêng bản thân
3	Đăng xuất

Bảng 4.3: Các lựa chọn trong `showStudentMenu`

4.2 Thiết kế cấu trúc dữ liệu

Toàn bộ dữ liệu bền vững của hệ thống được lưu trên hai file văn bản phẳng, dùng ký tự `|` làm dấu phân tách trường đúng như định hướng đã trình bày tại Mục 2. Phần này mô tả chính xác định dạng đang được cài đặt trong `QuestionBank.cpp` và `History.cpp`.

4.2.1 Cấu trúc file `questions.txt`

Mỗi dòng trong `questions.txt` gồm đúng 9 trường, tương ứng 1-1 với các thuộc tính của `struct Question` trong `QuestionBank.h`:

Thứ tự	Trường	Ý nghĩa
1	id	Mã câu hỏi, số nguyên dương, duy nhất
2	subject	Tên môn học (ví dụ: TA10, TA12)
3	difficulty	Mức độ: Easy / Medium / Hard
4	content	Nội dung câu hỏi
5–8	answers[0..3]	Bốn đáp án tương ứng A, B, C, D
9	correct	Ký tự đáp án đúng (A/B/C/D)

Bảng 4.4: Cấu trúc 9 trường của một dòng trong questions.txt

Ví dụ một dòng dữ liệu hợp lệ:

```
12|TA10|Medium|Choose the correct form: She ___ to school.|go|goes|going|went|B
```

Các dòng trống hoặc dòng bắt đầu bằng ký tự # được loadFromFile bỏ qua một cách an toàn, cho phép Admin chèn chú thích trực tiếp vào file dữ liệu khi cần.

4.2.2 Cấu trúc file history.txt

Tương tự, file history.txt lưu mỗi lần thi trên một dòng gồm 6 trường, tương ứng các thuộc tính của struct TestRecord trong History.h:

Thứ tự	Trường	Ý nghĩa
1	studentName	Tên đăng nhập của sinh viên
2	subject	Môn học đã thi
3	correctCount	Số câu trả lời đúng
4	totalCount	Tổng số câu của đề thi
5	score	Điểm số trên thang 10 (kiểu double)
6	datetime	Thời điểm nộp bài, dạng chuỗi YYYY-MM-DD HH:MM:SS

Bảng 4.5: Cấu trúc 6 trường của một dòng trong history.txt

Ví dụ một dòng dữ liệu hợp lệ:

```
hung|TA10|7|10|7|2026-06-15 21:40:12
```

4.2.3 Quy tắc định dạng và tách trường dữ liệu

Cả hai file đều tuân theo các quy tắc tách trường thống nhất sau:

- **Dấu phân tách:** ký tự | được chọn vì gần như không xuất hiện trong nội dung câu hỏi hay đáp án tiếng Anh thông thường, giảm thiểu rủi ro xung đột với dữ liệu thật.
- **Đọc tuần tự bằng istringstream:** mỗi dòng được nạp vào một istringstream, sau đó dùng getline(ss, token, '|') liên tiếp để tách từng trường theo đúng thứ tự đã định nghĩa ở Bảng 4.4 / 4.5.
- **Bẫy lỗi theo từng dòng:** nếu một dòng trong questions.txt thiếu trường, sai kiểu số (id), hoặc có correct nằm ngoài A–D, loadFromFile chỉ in cảnh báo kèm số dòng cụ thể và continue sang dòng kế tiếp, không làm crash hay dừng việc nạp toàn bộ ngân hàng.
- **Ghi đè với questions.txt:** QuestionBank::saveToFile dùng ofstream mặc định (ghi đè), vì mục tiêu là phản ánh đúng trạng thái hiện tại của ngân hàng câu hỏi sau khi thêm/xoá.
- **Ghi nối với history.txt:** HistoryManager::saveToFile mở file ở chế độ ios::app, vì mục tiêu là tích lũy lịch sử qua nhiều lần thi của nhiều sinh viên khác nhau – nếu dùng chế độ ghi đè mặc định sẽ làm mất toàn bộ lịch sử của các lượt thi trước đó.

4.3 Thiết kế các module chức năng

4.3.1 Module Menu

Giao diện đăng nhập và xác thực người dùng

Trạng thái đăng nhập được đóng gói trong struct `LoginSession` (gồm `username` và cờ `isAdmin`), còn kết quả của một lần thử đăng nhập được biểu diễn bằng enum class `LoginResult` với ba giá trị `Success`, `InvalidCredentials`, `Exit`. Việc dùng enum class (thay vì mã lỗi số nguyên) giúp `main()` xử lý từng trường hợp bằng switch rõ nghĩa, tránh nhầm lẫn giữa các mã trả về.

Hàm `showLoginScreen` được thiết kế với ba nhánh xác thực theo đúng thứ tự kiểm tra:

1. Nếu tên đăng nhập (chuyển về chữ thường) là "exit" → trả về `LoginResult::Exit` ngay, không yêu cầu nhập mật khẩu.
2. Nếu `username == "admin"` và `password == "admin123"` → `isAdmin = true`, trả về `Success`.
3. Nếu `password == "sv"+ username` (quy ước mật khẩu sinh viên) → `isAdmin = false`, trả về `Success`.
4. Ngược lại → in lỗi và trả về `InvalidCredentials`.

Thiết kế quy ước mật khẩu sinh viên theo công thức phù hợp với giới hạn “xác thực đơn giản, không mã hoá” đã nêu tại yêu cầu phi chức năng (Bảng 3.5).

Menu Admin: quản lý câu hỏi, xem thống kê

`showAdminMenu` nạp `QuestionBank` từ `data/questions.txt` ngay khi vào menu, sau đó lặp với 5 lựa chọn đã liệt kê ở Bảng 4.2. Hai điểm thiết kế đáng chú ý:

- **Xem theo môn học:** trước khi cho nhập tên môn, hệ thống liệt kê sẵn danh sách các môn học hiện có (qua `getDistinctSubjects`) để Admin không phải đoán tên môn; nếu môn nhập vào không khớp danh sách, vòng lặp yêu cầu nhập lại thay vì chấp nhận lệch chính tả gây hiểu nhầm “không có câu hỏi”.
- **Đồng bộ tức thời:** ngay sau khi thêm hoặc xóa câu hỏi thành công, `saveToFile` được gọi lại lập tức, đảm bảo yêu cầu phi chức năng về tính nhất quán dữ liệu.

Menu Sinh viên: vào thi, xem lịch sử

`showStudentMenu` không nạp `QuestionBank` ngay khi vào menu, mà chỉ nạp khi sinh viên thực sự chọn “Bắt đầu làm bài thi” - tránh I/O không cần thiết nếu sinh viên chỉ vào xem lịch sử. Luồng làm bài thi được thiết kế theo trình tự:

1. Nạp ngân hàng câu hỏi; nếu rỗng, báo lỗi và quay lại menu.
2. Liệt kê các môn học khác nhau (`getDistinctSubjects`) để sinh viên chọn theo số thứ tự.
3. Đếm số câu Easy/Medium/Hard của riêng môn đã chọn, gọi `getMaxNumberQuest` để xác định $N_{\max}$ hợp lệ theo tỉ lệ 40-40-20.
4. Nếu $N_{\max} < 1$ (thiếu hẳn một mức độ) → báo lỗi, không cho thi.
5. Sinh viên nhập số câu (trong $[1, N_{\max}]$) và thời gian làm bài (trong $[10, 36000]$ giây), xác nhận Y/N.
6. Khởi tạo đối tượng `Exam`, gọi `generateExamFromBank` rồi `shuffleExam`, sau đó `startExam` để tiến hành thi.
7. Gán `record.subject`, hiển thị kết quả, lưu vào `HistoryManager` rồi ghi file.

Cơ chế validate input và bắt lỗi

Ba hàm tiện ích dùng chung cho cả hai menu được thiết kế theo nguyên tắc “không bao giờ thoát chương trình do lỗi nhập liệu, chỉ thoát do hết dữ liệu đầu vào hoặc người dùng chủ động”:

- `getValidInt(prompt, minVal, maxVal)`: lặp vô hạn đến khi `cin` đọc được một số nguyên hợp lệ và nằm trong đoạn cho phép; nếu `cin` ở trạng thái lỗi (nhập chữ), gọi `cin.clear()` để xoá cờ lỗi rồi `cin.ignore()` để loại bỏ phần dữ liệu rác còn sót trong buffer trước khi yêu cầu nhập lại.
- `getValidString(prompt, maxLen)`: dùng `getline` để cho phép chuỗi có khoảng trắng, từ chối chuỗi rỗng hoặc vượt quá độ dài tối đa.
- `getYesNo(prompt)`: chỉ chấp nhận ký tự đầu tiên là Y hoặc N (không phân biệt hoa/thường), mọi đầu vào khác bị từ chối và hỏi lại.

Cả ba hàm đều kiểm tra `cin.eof()` hoặc thất bại của `getline` để gọi `exit(0)` một cách có kiểm soát khi luồng nhập liệu kết thúc bất ngờ (ví dụ khi chạy chương trình với input redirect từ file trong kiểm thử tự động), tránh rơi vào vòng lặp vô hạn.

4.3.2 Module Ngân hàng câu hỏi

Struct Question và các thuộc tính

`struct Question` được thiết kế không dùng con trỏ hay container động bên trong, để có thể gán (=) và sao chép (copy) trực tiếp giữa các mảng `Question[]` mà không cần cài đặt thêm copy constructor:

Thuộc tính	Kiểu	Ghi chú thiết kế
<code>id</code>	<code>int</code>	Khoá định danh duy nhất, kiểm tra trùng qua <code>isIdExists</code> trước khi thêm
<code>subject</code>	<code>string</code>	Dùng để gom nhóm câu hỏi theo môn khi sinh đề
<code>difficulty</code>	<code>string</code>	Một trong <code>Easy/Medium/Hard</code> , dùng cho tỉ lệ 40-40-20
<code>content</code>	<code>string</code>	Nội dung câu hỏi
<code>answers[4]</code>	<code>string[]</code>	Mảng tĩnh 4 phần tử – cố định số lượng đáp án theo đặc tả bài toán
<code>correct</code>	<code>char</code>	Ký tự A–D; được cập nhật lại mỗi khi đáp án bị xáo trộn

Bảng 4.6: Thiết kế thuộc tính của `struct Question`

Quyết định dùng mảng tĩnh `answers[4]` thay vì 4 biến `string` rời hoặc mảng động phản ánh đúng ràng buộc nghiệp vụ “luôn đúng 4 lựa chọn A/B/C/D”, đồng thời cho phép duyệt vòng `for` khi xáo trộn và in ấn, thay vì phải lặp lại code 4 lần cho từng đáp án.

Danh sách liên kết đơn lưu câu hỏi

`QuestionBank` quản lý các `QuestionNode` qua hai con trỏ `head` và `tail` (private). Việc giữ thêm con trỏ `tail` dù tốn 8 byte bộ nhớ là một quyết định thiết kế có chủ đích: nó cho phép `addQuestion` chèn vào cuối danh sách với độ phức tạp $O(1)$ thay vì phải duyệt từ `head` đến cuối mỗi lần thêm ($O(n)$), điều quan trọng khi ngân hàng có thể lên đến hàng nghìn câu hỏi (yêu cầu phi chức năng về hiệu năng, Bảng 3.5). Việc cập nhật `tail` khi xoá node cuối (trong `removeQuestion`) được xử lý cẩn thận để tránh con trỏ `tail` trở vào vùng nhớ đã `delete`.

Đọc/ghi file questions.txt

`loadFromFile` và `saveToFile` được thiết kế đối xứng với định dạng 9 trường ở Bảng 4.4, tuân theo các quy tắc bắt lỗi đã trình bày ở Mục 2: mỗi dòng được xử lý độc lập, lỗi ở một dòng

không ảnh hưởng các dòng khác, và `addQuestion` được tái sử dụng trong `loadFromFile` để đảm bảo ràng buộc “ID duy nhất” luôn được áp dụng nhất quán cho cả dữ liệu nhập tay (qua Menu) và dữ liệu nạp từ file.

Thêm, sửa, xoá câu hỏi

Phiên bản hiện tại cài đặt đầy đủ hai nghiệp vụ thêm (`addQuestion`, có kiểm tra trùng ID qua `isIdExists`) và xoá (`removeQuestion`, xử lý đủ 4 trường hợp: danh sách rỗng, xoá HEAD, xoá giữa/cuối, không tìm thấy ID). Riêng nghiệp vụ sửa câu hỏi chưa được cài đặt trong giao diện Menu hiện tại; để cập nhật nội dung một câu hỏi, Admin phải xoá rồi thêm lại với cùng ID. Đây là một hạn chế đã được nhóm ghi nhận và được đề xuất là một hướng phát triển ở Chương 7 (bổ sung hàm `updateQuestion(int id, Question newData)` duyệt danh sách liên kết tìm node theo id rồi ghi đè trực tiếp `node->data`, không cần xoá/thêm lại node).

4.3.3 Module Thi

Thuật toán sinh đề: xáo trộn Fisher-Yates

Việc sinh đề được tách thành hai giai đoạn rõ ràng trong `generateExamFromBank`:

- (1) Trích xuất đúng số lượng câu hỏi theo từng mức độ từ `QuestionBank` bằng `getQuestionsBySubjectAndDifficulty`
- (2) Xáo trộn bằng Fisher-Yates ở hai cấp độ: xáo trộn nội bộ từng nhóm Easy/Medium/Hard trước khi cắt lấy N câu đầu của mỗi nhóm, rồi xáo trộn tiếp toàn bộ N câu đã ghép để tránh tình trạng đề luôn hiển thị theo thứ tự Easy $\rightarrow$ Medium $\rightarrow$ Hard.

Thiết kế này đảm bảo: nếu ngân hàng có nhiều hơn số câu cần ở một mức độ, mỗi sinh viên có xác suất nhận tổ hợp câu hỏi khác nhau dù cùng tỉ lệ độ khó - đúng mục tiêu công bằng đã đặt ra ở yêu cầu FR-07.

Một quyết định thiết kế quan trọng về quyền sở hữu bộ nhớ: ba mảng `easyQuestions`, `mediumQuestions`, `hardQuestions` được `QuestionBank` cấp phát động (`new[]`) và trả về cho `Exam`, bên gọi (`Exam`) chịu trách nhiệm `delete[]` các mảng này được thực hiện ở cả hai nhánh thành công và thất bại của `generateExamFromBank`, tránh rò rỉ bộ nhớ ngay cả khi hàm trả về `false` sớm.

Xáo trộn thứ tự đáp án A/B/C/D

Thiết kế của `shuffleExam` tách bạch hai bước cho mỗi câu hỏi: trước khi gọi `shuffleAnswers` (Fisher-Yates trên mảng `string[4]`), hệ thống ghi nhớ lại nội dung văn bản của đáp án đúng (không ghi nhớ chỉ số); sau khi xáo trộn, hệ thống duyệt lại 4 đáp án để tìm vị trí mới của đúng nội dung đó và gán lại `q.correct`. Lý do thiết kế theo nội dung văn bản chứ không theo chỉ số là vì chỉ số ban đầu sẽ không còn ý nghĩa sau khi mảng bị hoán đổi vị trí ngẫu nhiên, so sánh theo nội dung là cách duy nhất để xác định đúng đáp án đúng đã “di chuyển” đến đâu.

Module đếm giờ đếm ngược thời gian thực

Đếm giờ được thiết kế dựa trên đồng hồ hệ thống (`time(nullptr)`, `difftime`) thay vì đếm số vòng lặp hay `sleep`, để không bị lệch thời gian thực khi người dùng dừng lại suy nghĩ lâu ở một câu. `startExam` lưu `startTime` một lần duy nhất trước khi vào vòng lặp câu hỏi, sau đó tính `remaining = timeLimitSec_ - elapsed` tại hai thời điểm trong mỗi lượt câu hỏi: ngay trước khi hiển thị câu hỏi, và ngay sau khi người dùng nhập xong đáp án. Thiết kế kiểm tra hai lần là để xử lý đúng tình huống biên: sinh viên có thể đọc câu hỏi rất nhanh nhưng nhập đáp án rất chậm, khiến thời gian hết ngay trong lúc đang nhập nếu chỉ kiểm tra trước khi hiển thị, hệ thống sẽ vô tình cho phép làm bài quá giờ quy định.

Thu bài tự động khi hết giờ

Việc thu bài được thống nhất qua một biến cờ duy nhất `submitted`, giúp vòng lặp chính của `startExam` có một điều kiện dừng nhất quán (`for (...; !submitted; ...)`) bất kể nguyên nhân thu bài là gì. Khi `submitted` được đặt `true` ở bất kỳ một trong ba điểm kiểm tra (trước hiển thị, trong khi nhập, sau khi nhập), vòng lặp `break` ngay, và các câu hỏi chưa kịp duyệt tới vẫn giữ giá trị khởi tạo `userAnswers[i] = '-'`, được module chấm điểm hiểu là trạng thái “chưa trả lời” (phân biệt rõ với “trả lời sai”) – đúng quy tắc nghiệp vụ đã đặt ra.

Logic tính điểm và tổng hợp kết quả

Điểm số được tính theo công thức $\text{Điểm} = \frac{\text{correctCount}}{\text{totalCount}} \times 10$, với điều kiện bảo vệ `totalCount > 0` để trả về 0.0 thay vì gây lỗi chia cho 0 trong trường hợp biên đề thi rỗng.

4.3.4 Module Lịch sử thi

Struct `TestRecord` và các thuộc tính

`TestRecord` đóng vai trò lưu trữ giữa module `Exam` (nơi sinh ra bản ghi) và module `History` (nơi lưu trữ bản ghi); `TestRecord` được khai báo trong `History.h` nhưng lại được `Exam.h` `#include` và sử dụng làm kiểu trả về của `startExam`. Bảng 4.5 đã liệt kê đầy đủ 6 thuộc tính; trong đó trường `subject` không được `Exam::startExam` điền sẵn, mà được `Menu` gán bổ sung ngay sau khi nhận `TestRecord` trả về - một ví dụ cụ thể cho việc phân chia trách nhiệm: `Exam` chỉ biết về nội dung bài thi, còn môn học do `Menu` quản lý.

Danh sách liên kết đơn lưu lịch sử

Tương tự thiết kế của `QuestionBank`, `HistoryManager` dùng cặp con trỏ `head/tail` để thêm bản ghi mới vào cuối với độ phức tạp $O(1)$. Khác với `QuestionBank`, `HistoryManager` không cần nghiệp vụ xóa bản ghi hay kiểm tra trùng khoá, lịch sử thi mang tính chất “chỉ ghi thêm, không chỉnh sửa”, phản ánh đúng bản chất một bản ghi lịch sử không nên bị thay đổi sau khi đã phát sinh.

Ghi kết quả vào file `history.txt`

Như đã phân tích ở Mục 2, thiết kế ghi nổi đảm bảo lịch sử của các sinh viên khác không bị mất khi một sinh viên mới hoàn thành bài thi. Một lưu ý thiết kế khác: mỗi lần gọi `saveToFile`, `HistoryManager` chỉ chứa bản ghi vừa thêm trong phiên hiện tại, nhờ vậy việc ghi nổi hoạt động đúng mà không cần đọc lại toàn bộ file lịch sử cũ trước khi ghi.

Truy vấn và hiển thị lịch sử theo username

`printByUser` thiết kế theo nguyên tắc duyệt tuyến tính toàn bộ danh sách liên kết, chỉ in các bản ghi có `studentName == username`, kết hợp một biến cờ `found` để phân biệt rõ hai tình huống đầu ra: “không có lịch sử nào của riêng người này” và “ngân hàng lịch sử rỗng hoàn toàn”, tránh hiển thị một màn hình trống không lời giải thích, đúng yêu cầu nghiệp vụ về tính khả dụng (Bảng 3.5). Việc tách `printAll` (dành cho Admin, không lọc) và `printByUser` (dành cho Sinh viên, lọc theo người dùng hiện tại) thành hai hàm riêng – thay vì một hàm chung với tham số lọc tùy chọn – giúp mỗi hàm có một mục đích duy nhất, dễ đọc và khớp đúng với phân quyền của từng tác nhân đã mô tả ở Bảng 3.1.

4.3.5 Hàm main

`main()` được thiết kế tối giản, chỉ làm ba việc: (1) khởi tạo bộ sinh số ngẫu nhiên bằng `srand(time(nullptr))` – bắt buộc gọi đúng một lần ở điểm vào chương trình để các lần xáo trộn Fisher-Yates trong `Exam` cho ra kết quả khác nhau giữa các lần chạy;

(2) duy trì vòng lặp đăng nhập chính (`while (appRunning)`) và `switch` điều hướng sang `showAdminMenu/showStudentMenu` dựa trên `LoginResult`;

(3) hiển thị màn hình tạm biệt và `return 0` khi người dùng chọn thoát.

Việc không đặt bất kỳ logic nghiệp vụ nào trực tiếp trong `main()`, toàn bộ được uỷ quyền cho `Menu`, là hệ quả trực tiếp của kiến trúc phân tầng đã trình bày tại Mục 4.1 – giúp `main.cpp` đóng vai trò thuần túy là điểm khởi đầu, dễ đọc và dễ trình bày.

Chương 5

Cài đặt và xây dựng chương trình

Chương này trình bày chi tiết quá trình cài đặt chương trình dựa trên thiết kế đã trình bày ở Chương 4, đối chiếu trực tiếp với mã nguồn thực tế của từng module: `QuestionBank`, `Exam`, `History` và `Menu`.

5.1 Cấu trúc thư mục chương trình

Mã nguồn được tổ chức theo đúng quy ước tách Header/Source đã trình bày ở Mục 2, với cấu trúc thư mục như sau:

```
KY_THUAT_LAP_TRINH/  
|-- include/  
|   |-- QuestionBank.h  
|   |-- Exam.h  
|   |-- History.h  
|   '-- Menu.h  
|-- src/  
|   |-- main.cpp  
|   |-- Menu.cpp  
|   |-- Exam.cpp  
|   |-- History.cpp  
|   '-- QuestionBank.cpp  
|-- data/  
|   |-- questions.txt  
|   '-- history.txt  
|-- .vscode/  
|   '-- tasks.json  
|-- quiz_program.exe    (sinh ra sau khi bien dich)  
'-- README.md
```

Việc tách riêng thư mục `data/` khỏi mã nguồn cho phép cập nhật ngân hàng câu hỏi mà không cần biên dịch lại chương trình, đồng thời thuận tiện cho việc sao lưu dữ liệu độc lập với mã nguồn.

5.2 Cài đặt module Ngân hàng câu hỏi (QuestionBank)

5.2.1 Khai báo cấu trúc dữ liệu

Cấu trúc `Question` lưu trữ một câu hỏi trắc nghiệm với 4 đáp án cố định, được tổ chức thành các node của danh sách liên kết đơn:

```

1 struct Question {
2     int id;
3     string subject;
4     string difficulty; // Easy, Medium, Hard
5     string content;
6     string answers[4];
7     char correct; // 'A', 'B', 'C', hoặc 'D'
8 };
9
10 struct QuestionNode {
11     Question data;
12     QuestionNode* next;
13 };
14
15 class QuestionBank {
16 private:
17     QuestionNode* head;
18     QuestionNode* tail;
19 public:
20     QuestionBank();
21     ~QuestionBank();
22     bool isIdExists(int id);
23     bool addQuestion(Question q);
24     void removeQuestion(int id);
25     int getQuestionCount();
26     Question* getQuestionAt(int index);
27     int countBySubjectAndDifficulty(const string& subject, const string&
difficulty);
28     Question* getQuestionsBySubjectAndDifficulty(const string& subject,
const string& difficulty, int&
count);
29
30     bool loadFromFile(string filename);
31     bool saveToFile(string filename);
32     Question* generateRandomSet(int n);
33 };

```

Listing 5.1: Khai báo Question và QuestionNode (QuestionBank.h)

So với thiết kế lý thuyết ở Chương 2, điểm khác biệt đáng chú ý trong cài đặt thực tế là trường `subject` và `difficulty` được bổ sung vào `Question` để hỗ trợ phân loại câu hỏi theo môn học và mức độ (Easy/Medium/Hard), phục vụ thuật toán sinh đề theo tỉ lệ ở Mục 5.3.

5.2.2 Quản lý bộ nhớ: Constructor và Destructor

Constructor khởi tạo danh sách rỗng (`head = tail = nullptr`); Destructor duyệt toàn bộ danh sách và giải phóng từng node để tránh Memory Leak — đúng nguyên tắc “mỗi `new` phải có một `delete`” đã đặt ra trong README của dự án:

```

1 QuestionBank::~~QuestionBank() {
2     QuestionNode* current = head;
3     while (current != nullptr) {
4         QuestionNode* nextNode = current->next;
5         delete current;
6         current = nextNode;
7     }
8     head = nullptr;
9     tail = nullptr;
10 }

```

Listing 5.2: Destructor của QuestionBank (QuestionBank.cpp)

5.2.3 Thêm và xoá câu hỏi

Hàm `addQuestion` kiểm tra trùng ID trước khi chèn vào cuối danh sách (sử dụng con trỏ `tail` để đạt độ phức tạp $O(1)$); hàm `removeQuestion` xử lý đầy đủ bốn trường hợp: danh sách rỗng,

xoá node HEAD, xoá node giữa/cuối, và không tìm thấy ID:

```

1 void QuestionBank::removeQuestion(int id) {
2     if (head == nullptr) {
3         cout << "[Loi] Danh sach cau hoi dang rong.\n";
4         return;
5     }
6     if (head->data.id == id) {           // Truong hop HEAD
7         QuestionNode* toDelete = head;
8         head = head->next;
9         if (head == nullptr) tail = nullptr;
10        delete toDelete;
11        return;
12    }
13    QuestionNode* prev = head;
14    QuestionNode* current = head->next;
15    while (current != nullptr) {         // Truong hop giua/cuoi
16        if (current->data.id == id) {
17            prev->next = current->next;
18            if (current == tail) tail = prev;
19            delete current;
20            return;
21        }
22        prev = current;
23        current = current->next;
24    }
25    cout << "[Canh bao] Khong tim thay cau hoi co id = " << id << ".\n";
26 }

```

Listing 5.3: Xoá câu hỏi theo ID (QuestionBank.cpp)

5.2.4 Đọc và ghi file dữ liệu

Hàm `loadFromFile` đọc `questions.txt` theo định dạng phân tách `|`, với cơ chế bẫy lỗi cho từng dòng: bỏ qua dòng rỗng/comment, kiểm tra đủ 9 trường dữ liệu, và validate ký tự đáp án đúng nằm trong khoảng A–D. Mỗi lỗi được báo cụ thể số dòng để dễ debug, thay vì làm crash toàn bộ chương trình — đúng nguyên tắc lập trình phòng ngừa đã trình bày ở Mục 2:

```

1 char c = toupper((unsigned char)token[0]);
2 if (c < 'A' || c > 'D') {
3     cout << "[Canh bao] Dong " << lineNum
4         << ": dap an dung '" << token
5         << "' khong hop le (chi chap nhan A-D). Bo qua.\n";
6     continue;
7 }
8 q.correct = c;
9 if (!addQuestion(q)) {
10    cout << "[Canh bao] Dong " << lineNum << ": ID " << q.id
11        << " bi trung voi cau hoi da co, da bo qua dong nay.\n";
12 }

```

Listing 5.4: Trích đoạn `loadFromFile` với bẫy lỗi từng dòng (QuestionBank.cpp)

Hàm `saveToFile` ghi đè (ofstream mặc định) toàn bộ ngân hàng câu hỏi hiện tại xuống file, được gọi ngay sau mỗi thao tác thêm hoặc xoá câu hỏi thành công trong Menu Admin, đảm bảo dữ liệu trên đĩa luôn đồng bộ với dữ liệu trong bộ nhớ.

5.2.5 Truy vấn câu hỏi theo môn học và độ khó

Để phục vụ sinh đề theo tỉ lệ 40-40-20 (xem Mục 5.3), `QuestionBank` cung cấp hàm đếm và trích xuất câu hỏi theo cặp (môn học, độ khó):

```

1 Question* QuestionBank::getQuestionsBySubjectAndDifficulty(
2     const string& subject, const string& difficulty, int& count) {
3     count = countBySubjectAndDifficulty(subject, difficulty);

```

```

4     if (count == 0) return nullptr;
5     Question* result = new Question[count];
6     QuestionNode* current = head;
7     int index = 0;
8     while (current != nullptr) {
9         if (current->data.subject == subject &&
10            current->data.difficulty == difficulty) {
11             result[index++] = current->data;
12         }
13         current = current->next;
14     }
15     return result;
16 }

```

Listing 5.5: Trích xuất câu hỏi theo môn học và độ khó (QuestionBank.cpp)

Lưu ý rằng mảng `result` được cấp phát động bằng `new[]` và *quyền sở hữu được chuyển cho bên gọi* — đây là một quy ước quan trọng cần tài liệu hoá rõ ràng, vì `Exam::generateExamFromBank` (Mục 5.3) có trách nhiệm `delete[]` ba mảng tạm này sau khi sử dụng để tránh rò rỉ bộ nhớ.

5.3 Cài đặt module Thi (Exam)

5.3.1 Sinh đề theo tỉ lệ độ khó 40-40-20

Thay vì bốc ngẫu nhiên hoàn toàn, đề thi được sinh theo tỉ lệ cố định: 40% câu Easy, 40% câu Medium, 20% câu Hard (định nghĩa bằng hằng số tĩnh `EASY_PERCENT`, `MEDIUM_PERCENT`, `HARD_PERCENT` trong `Exam.h`). Nếu ngân hàng không đủ câu hỏi theo tỉ lệ yêu cầu cho môn học đã chọn, hàm trả về `false` và Menu sẽ thông báo lỗi cho người dùng thay vì sinh đề thiếu:

```

1  bool Exam::generateExamFromBank(QuestionBank& bank, const string& subject) {
2      int easyNeed = numberOfQuestions * EASY_PERCENT / 100;
3      int mediumNeed = numberOfQuestions * MEDIUM_PERCENT / 100;
4      int hardNeed = numberOfQuestions - easyNeed - mediumNeed;
5
6      int easySize, mediumSize, hardSize;
7      Question* easyQ = bank.getQuestionsBySubjectAndDifficulty(subject, "Easy",
8      easySize);
9      Question* mediumQ = bank.getQuestionsBySubjectAndDifficulty(subject, "Medium",
10      mediumSize);
11      Question* hardQ = bank.getQuestionsBySubjectAndDifficulty(subject, "Hard",
12      hardSize);
13
14      if (easySize < easyNeed || mediumSize < mediumNeed || hardSize < hardNeed) {
15          cout << "[Loi] Ngân hàng câu hỏi môn " << subject << " không đủ để sinh
16      đề.\n";
17          delete[] easyQ; delete[] mediumQ; delete[] hardQ;
18          return false;
19      }
20      shuffleQuestions(easyQ, easySize);
21      shuffleQuestions(mediumQ, mediumSize);
22      shuffleQuestions(hardQ, hardSize);
23
24      int index = 0;
25      for (int i = 0; i < easyNeed; i++) questionList[index++] = easyQ[i];
26      for (int i = 0; i < mediumNeed; i++) questionList[index++] = mediumQ[i];
27      for (int i = 0; i < hardNeed; i++) questionList[index++] = hardQ[i];
28      shuffleQuestions(questionList, numberOfQuestions);
29
30      delete[] easyQ; delete[] mediumQ; delete[] hardQ;
31      return true;
32 }

```

Listing 5.6: Sinh đề theo tỉ lệ độ khó (Exam.cpp)

Việc xáo trộn ba nhóm Easy/Medium/Hard *trước khi* chọn ra N câu đầu tiên của mỗi nhóm đảm bảo mỗi sinh viên nhận được tổ hợp câu hỏi khác nhau dù cùng tỉ lệ độ khó. Lệnh `shuffleQuestions(questionList, numberOfQuestions)` cuối cùng tiếp tục trộn thứ tự xuất hiện giữa ba nhóm, tránh tình trạng đề luôn hiển thị Easy trước, Hard sau.

5.3.2 Xác định số câu hỏi tối đa hợp lệ

Vì tỉ lệ 40-40-20 đặt ra ràng buộc giữa ba mức độ, không phải số câu N nào sinh viên nhập vào cũng sinh đề được. Hàm `getMaxNumberQuest` trong `Menu.cpp` duyệt N giảm dần từ tổng số câu hỏi của môn học để tìm N hợp lệ lớn nhất, sau đó giới hạn lựa chọn của sinh viên trong khoảng $[1, N_{\max}]$:

```

1  int getMaxNumberQuest(int nEasy, int nMedium, int nHard) {
2      int total = nEasy + nMedium + nHard;
3      for (int n = total; n >= 1; n--) {
4          int easyNeed = n * 40 / 100;
5          int mediumNeed = n * 40 / 100;
6          int hardNeed = n - easyNeed - mediumNeed;
7          if (easyNeed <= nEasy && mediumNeed <= nMedium && hardNeed <= nHard)
8              return n;
9      }
10     return 0;
11 }
```

Listing 5.7: Tính số câu hỏi tối đa hợp lệ (Menu.cpp)

5.3.3 Đếm giờ thời gian thực và thu bài tự động

Vòng lặp chính của `startExam` kiểm tra thời gian còn lại *cả trước và sau* mỗi lần người dùng nhập đáp án, sử dụng `time(nullptr)` và `difftime` để tính khoảng cách thời gian thực:

```

1  long elapsed = (long) difftime(time(nullptr), startTime);
2  long remaining = timeLimitSec_ - elapsed;
3  if (remaining <= 0) {
4      cout << "\n\n*** HET GIO! He thong tu dong thu bai. ***\n";
5      submitted = true;
6      break;
7  }
```

Listing 5.8: Kiểm tra thời gian và thu bài tự động (Exam.cpp)

Việc kiểm tra hai lần (trước khi hiển thị câu hỏi và ngay sau khi người dùng nhập xong đáp án) đảm bảo sinh viên không thể "lách" giới hạn thời gian bằng cách kéo dài thời gian suy nghĩ cho câu hỏi cuối cùng.

5.3.4 Chấm điểm

Điểm được tính theo thang 10 dựa trên tỉ lệ số câu đúng trên tổng số câu, tránh chia cho 0 khi đề thi rỗng:

```

1  record.score = (record.totalCount > 0)
2      ? (double) record.correctCount / record.totalCount * 10.0
3      : 0.0;
```

Listing 5.9: Công thức tính điểm (Exam.cpp)

5.4 Cài đặt module Lịch sử thi (History)

5.4.1 Cấu trúc dữ liệu TestRecord

Mỗi lần thi tạo ra một bản ghi `TestRecord` chứa tên sinh viên, môn học, số câu đúng, tổng số câu, điểm số và thời gian (định dạng chuỗi YYYY-MM-DD HH:MM:SS sinh bởi `strftime`):

```

1  time_t nowTime = time(nullptr);
2  tm* localTm = localtime(&nowTime);
3  char buf[32];
4  strftime(buf, sizeof(buf), "%Y-%m-%d %H:%M:%S", localTm);
5  record.datetime = buf;

```

Listing 5.10: Ghi nhận timestamp khi nộp bài (Exam.cpp)

5.4.2 Ghi và đọc lịch sử

Khác với `QuestionBank::saveToFile` (ghi đè), `HistoryManager::saveToFile` mở file ở chế độ `ios::app` (append) để *nối thêm* bản ghi mới vào cuối file thay vì ghi đè toàn bộ lịch sử cũ:

```

1  void HistoryManager::saveToFile(const string& filename) {
2      ofstream outFile(filename, ios::app);
3      if (!outFile.is_open()) {
4          cerr << "[Loi] Khong the mo file: " << filename << "\n";
5          return;
6      }
7      HistoryNode* current = head;
8      while (current != nullptr) {
9          const TestRecord& rec = current->data;
10         outFile << rec.studentName << "|" << rec.subject << "|"
11                 << rec.correctCount << "|" << rec.totalCount << "|"
12                 << rec.score << "|" << rec.datetime << "\n";
13         current = current->next;
14     }
15     outFile.close();
16 }

```

Listing 5.11: Ghi lịch sử ở chế độ append (History.cpp)

Quyết định thiết kế này đảm bảo lịch sử thi của các sinh viên trước đó không bị mất sau mỗi lần một sinh viên khác hoàn thành bài thi — một lỗi nghiêm trọng nếu vô tình dùng chế độ ghi đè mặc định của `ofstream`.

5.4.3 Truy vấn theo người dùng

Hàm `printByUser` duyệt tuyến tính toàn bộ danh sách liên kết và chỉ in các bản ghi khớp `username`, đồng thời dùng biến cờ `found` để thông báo trường hợp sinh viên chưa từng thi lần nào.

5.5 Cài đặt module Giao diện (Menu)

5.5.1 Xác thực đăng nhập

Tài khoản Admin được hardcode (`admin/admin123`); tài khoản sinh viên áp dụng quy ước mật khẩu là chuỗi `"sv"+ username`, cho phép bất kỳ tên đăng nhập nào miễn mật khẩu đúng quy ước — phù hợp với mục tiêu là bài tập minh họa kỹ thuật, không phải hệ thống xác thực bảo mật thực tế:

```

1  if (username == "admin" && password == "admin123") {
2      session.username = username;
3      session.isAdmin = true;
4      return LoginResult::Success;
5  }
6  if (password == ("sv" + username)) {
7      session.username = username;
8      session.isAdmin = false;
9      return LoginResult::Success;
10 }

```

Listing 5.12: Xác thực đăng nhập (Menu.cpp)

5.5.2 Validate input

Hàm `getValidInt` minh họa kỹ thuật lập trình phòng ngừa: vòng lặp `while(true)` liên tục yêu cầu nhập lại cho đến khi giá trị vừa đúng kiểu số nguyên vừa nằm trong khoảng cho phép; `cin.clear()` xoá cờ lỗi của stream và `cin.ignore()` loại bỏ phần dữ liệu rác còn sót trong buffer:

```

1  int getValidInt(const string& prompt, int minVal, int maxVal) {
2      int value;
3      while (true) {
4          cout << prompt;
5          if (cin >> value) {
6              if (value >= minVal && value <= maxVal) {
7                  cin.ignore(numeric_limits<streamsize>::max(), '\n');
8                  return value;
9              }
10             cout << "Gia tri phai trong khoang [" << minVal << ", " << maxVal <<
11             "]\n";
12             } else {
13                 if (cin.eof()) exit(0);
14                 cout << "Vui long nhap mot so nguyen hop le!\n";
15                 cin.clear();
16             }
17             cin.ignore(numeric_limits<streamsize>::max(), '\n');
18         }
19     }

```

Listing 5.13: Bẫy lỗi nhập số nguyên (Menu.cpp)

5.5.3 Điều hướng Menu Admin và Menu Sinh viên

Cả hai menu được cài đặt theo mô hình vòng lặp `while(running)` với cấu trúc `switch-case`, mỗi nhánh tương ứng một chức năng (xem danh sách câu hỏi, thêm/xoá câu hỏi, xem lịch sử với Admin; làm bài thi, xem lịch sử cá nhân với Sinh viên). Lựa chọn **Đăng xuất** đặt cờ `running = false` để thoát vòng lặp và quay về màn hình đăng nhập tại `main()`, cho phép đăng nhập lại với tài khoản khác mà không cần khởi động lại chương trình.

5.6 Biên dịch chương trình

Chương trình được biên dịch bằng `g++` theo chuẩn `C++17`, sử dụng cấu hình `tasks.json` của Visual Studio Code:

```

1  g++ -std=c++17 -g -Wall -I./include \
2      src/main.cpp src/Menu.cpp src/Exam.cpp \
3      src/History.cpp src/QuestionBank.cpp \
4      -o quiz_program.exe

```

Listing 5.14: Lệnh biên dịch chương trình

Cờ `-Wall` bật toàn bộ cảnh báo trình biên dịch, giúp phát hiện sớm các vấn đề tiềm ẩn như biến không sử dụng hay so sánh dấu/không dấu. Đúng theo quy ước trong README của dự án, mỗi thành viên chỉ đẩy mã nguồn lên kho chung khi đã biên dịch thành công với **0 lỗi** trên máy cá nhân.

Chương 6

Kiểm thử và Đánh giá kết quả

6.1 Chiến lược kiểm thử

Nhóm áp dụng kết hợp hai chiến lược kiểm thử trong suốt quá trình phát triển:

- **Kiểm thử đơn vị không chính thức (manual unit testing):** Mỗi thành viên tự kiểm thử module mình phụ trách trước khi đẩy code lên kho chung — ví dụ Văn kiểm tra riêng các thao tác thêm/xoá/đọc/ghi của `QuestionBank` bằng một hàm `main()` tạm thời trước khi tích hợp.
- **Kiểm thử tích hợp (integration testing) thủ công:** Sau khi ráp nối các module trong `main.cpp`, Thành thực hiện kiểm thử toàn luồng từ đăng nhập đến nộp bài và xem lịch sử, mô phỏng các tình huống người dùng thực tế.

Kiểm thử được thực hiện trên cả hai môi trường: Windows (MinGW UCRT64 g++) và Linux (g++ kèm Valgrind để kiểm tra rò rỉ bộ nhớ), nhằm đảm bảo tính nhất quán đa nền tảng của các thư viện chuẩn được sử dụng (`<fstream>`, `<ctime>`).

6.2 Tình huống kiểm thử

6.2.1 Test Case-01: Đăng nhập đúng – Admin

Đầu vào	Username: admin, Password: admin123
Kết quả mong đợi	<code>showLoginScreen</code> trả về <code>LoginResult::Success</code> , <code>session.isAdmin = true</code> , chuyển sang <code>showAdminMenu</code>
Kết quả thực tế	Đạt — hiển thị đúng Menu Admin với tên đăng nhập trên tiêu đề
Đánh giá	PASS

Bảng 6.1: Test Case-01

6.2.2 Test Case-02: Đăng nhập đúng – Sinh viên

Đầu vào	Username: hung, Password: svhung
Kết quả mong đợi	So khớp điều kiện <code>password == ("sv"+ username)</code> , trả về <code>Success</code> với <code>isAdmin = false</code>
Kết quả thực tế	Đạt — chuyển đúng sang <code>showStudentMenu</code>
Đánh giá	PASS

Bảng 6.2: Test Case-02

6.2.3 Test Case-03: Đăng nhập sai mật khẩu

Đầu vào	Username: hung, Password: 123456
Kết quả mong đợi	Không khớp cả hai điều kiện Admin và Sinh viên; trả về <code>LoginResult::InvalidCredentials</code> ; in thông báo lỗi và quay lại màn hình đăng nhập
Kết quả thực tế	Đạt — thông báo “Sai ten dang nhap hoac mat khau!” hiển thị đúng, vòng lặp <code>while(appRunning)</code> trong <code>main()</code> cho phép nhập lại
Đánh giá	PASS

Bảng 6.3: Test Case-03

6.2.4 Test Case-04: Thêm câu hỏi mới

Đầu vào	ID = 301 (chưa tồn tại), môn TA12, độ khó Medium, đầy đủ 4 đáp án, đáp án đúng = B
Kết quả mong đợi	<code>isIdExists(301)</code> trả về <code>false</code> ; <code>addQuestion</code> thành công; <code>saveToFile</code> ghi đè <code>questions.txt</code> với câu hỏi mới ở cuối file
Kết quả thực tế	Đạt — câu hỏi xuất hiện khi gọi lại <code>printAll()</code> , file <code>questions.txt</code> có thêm 1 dòng đúng định dạng
Đánh giá	PASS

Bảng 6.4: Test Case-04

Kiểm thử biên bổ sung: thêm câu hỏi với ID trùng với ID đã tồn tại (ví dụ ID = 1) — chương trình in cảnh báo “ID 1 đã tồn tại trong ngân hàng. Không thêm được.” và không ghi đè dữ liệu cũ. Kết quả: PASS.

6.2.5 Test Case-05: Sinh đề và xáo trộn câu hỏi/đáp án

Đầu vào	Môn TA10 (100 câu hỏi: 40 Easy, 40 Medium, 20 Hard), số câu thi = 20
Kết quả mong đợi	<code>generateExamFromBank</code> trả về <code>true</code> ; đề gồm đúng $20 \times 40\% = 8$ câu Easy, 8 câu Medium, $20 - 8 - 8 = 4$ câu Hard; thứ tự câu hỏi và đáp án A/B/C/D khác nhau ở mỗi lần chạy
Kết quả thực tế	Đạt — chạy <code>printExam()</code> 5 lần liên tiếp cho cùng tham số, ghi nhận 5 thứ tự câu hỏi khác nhau và vị trí đáp án đúng (<code>q.correct</code>) thay đổi tương ứng với việc xáo trộn
Đánh giá	PASS

Bảng 6.5: Test Case-05

Kiểm thử biên bổ sung: chọn môn chỉ có 12 câu Hard nhưng yêu cầu 50 câu thi (cần $50 \times 20\% = 10$ câu Hard) — vẫn đủ nên PASS; thử tiếp với 95 câu thi (cần $95 \times 20\% = 19$ câu Hard, vượt quá 12 câu hiện có) — hàm trả về `false` và in đúng thông báo “Ngân hàng câu hỏi môn ... không đủ để sinh đề.”. Kết quả: PASS.

6.2.6 Test Case-06: Làm bài và chấm điểm chính xác

Đầu vào	Đề 10 câu, trả lời đúng 7 câu, sai 2 câu, bỏ trống 1 câu (nhập ký tự không hợp lệ rồi nhập lại)
Kết quả mong đợi	<code>record.correctCount = 7;</code> <code>record.score = 7.0/10*10.0 = 7.0;</code> phần “CHI TIET KET QUA” hiển thị đúng nhãn <code>[DUNG]/[SAI]/[CHUA TRA LOI]</code> cho từng câu
Kết quả thực tế	Đạt — điểm hiển thị đúng 7.00 / 10 theo định dạng <code>fixed setprecision(2)</code>
Đánh giá	PASS

Bảng 6.6: Test Case-06

6.2.7 Test Case-07: Tự động thu bài khi hết giờ

Đầu vào	Đề 5 câu, thời gian làm bài = 10 giây (giá trị tối thiểu cho phép); cố tình trả lời chậm để vượt quá 10 giây
Kết quả mong đợi	Tại lần kiểm tra <code>remaining <= 0</code> tiếp theo (trước hoặc sau khi nhập một câu), chương trình in “HET GIO!” và đặt <code>submitted = true</code> , dừng vòng lặp ngay cả khi chưa làm hết các câu còn lại
Kết quả thực tế	Đạt — chương trình thu bài đúng thời điểm, các câu chưa kịp trả lời được chấm là <code>[CHUA TRA LOI]</code> với <code>userAnswers[i] = '-'</code>
Đánh giá	PASS

Bảng 6.7: Test Case-07

6.2.8 Test Case-08: Xem lịch sử thi nhiều lần

Đầu vào	Sinh viên <code>hung</code> thi 3 lần liên tiếp với các môn khác nhau
Kết quả mong đợi	<code>history.txt</code> chứa đủ 3 dòng tương ứng (chế độ <code>ios::app</code> , không ghi đè); <code>printByUser("hung")</code> hiển thị đúng cả 3 bản ghi theo thứ tự thời gian
Kết quả thực tế	Đạt — cả 3 lần thi đều xuất hiện đầy đủ; lịch sử của sinh viên khác (<code>thanh</code>) không bị lẫn vào kết quả
Đánh giá	PASS

Bảng 6.8: Test Case-08

6.2.9 Test Case-09: Nhập sai định dạng (bẫy lỗi)

Đầu vào	Tại bước “Nhập số câu hỏi muốn thi”, nhập lần lượt: chuỗi chữ <code>abc</code> , số âm <code>-5</code> , số thực <code>3.5</code> , sau đó số hợp lệ
Kết quả mong đợi	<code>getValidInt</code> không cho <code>cin</code> chấp nhận <code>abc</code> (in lỗi “Vui lòng nhập một số nguyên hợp lệ!”); với <code>-5</code> và đầu vào ngoài khoảng cho phép, in lỗi “Giá trị phải trong khoảng...”; chương trình không bao giờ thoát chương trình hay crash, chỉ dừng khi gặp EOF
Kết quả thực tế	Đạt — toàn bộ đầu vào không hợp lệ đều bị từ chối và yêu cầu nhập lại đúng như thiết kế
Đánh giá	PASS

Bảng 6.9: Test Case-09

6.2.10 Test Case-10: Kiểm thử rò rỉ bộ nhớ (Valgrind)

Đầu vào	Chạy chương trình dưới valgrind --leak-check=full ./quiz_program, thực hiện đầy đủ một phiên: đăng nhập Admin, thêm/xoá câu hỏi, đăng xuất, đăng nhập Sinh viên, làm bài thi, xem lịch sử, thoát chương trình
Kết quả mong đợi	Báo cáo Valgrind hiển thị definitely lost: 0 bytes, mọi khối nhớ cấp phát bởi new/new[] (trong Exam, QuestionBank, HistoryManager và các mảng tạm trong generateExamFromBank) đều được giải phóng tương ứng trong Destructor hoặc ngay sau khi sử dụng
Kết quả thực tế	Đạt — không phát hiện rò rỉ bộ nhớ đáng kể; các khối easyQ, mediumQ, hardQ trong generateExamFromBank được giải phóng đúng ở cả nhánh thành công và nhánh lỗi (ngân hàng không đủ câu)
Đánh giá	PASS

Bảng 6.10: Test Case-10

6.3 Kịch bản kiểm thử minh họa

Ngoài các tình huống kiểm thử có cấu trúc ở Mục 6 trên, nhóm còn thực hiện một phiên chạy thử hoàn chỉnh trên chương trình thực tế (quiz_program.exe) để xác nhận trải nghiệm người dùng đầu cuối diễn ra đúng như thiết kế. Kịch bản dưới đây tường thuật lại trình tự thao tác thực tế, từ màn hình khởi động cho đến khi đăng xuất.

6.3.1 Khởi động chương trình và đăng nhập

Ngay sau khi chạy file quiz_program.exe, hệ thống hiển thị màn hình chào mừng và yêu cầu nhập “*Ten dang nhap*”. Nếu là Admin, tài khoản cố định được sử dụng là Username: admin, Password: admin123. Nếu là sinh viên, người dùng có thể nhập một tên đăng nhập bất kỳ, với điều kiện mật khẩu phải tuân theo đúng cú pháp "sv"+ username — ví dụ với Username: hung12 thì Password bắt buộc phải là svhung12.

Khi người dùng nhập sai thông tin đăng nhập, hệ thống hiển thị thông báo lỗi và yêu cầu nhấn Enter để quay lại màn hình nhập liệu. Khi người dùng gõ exit vào ô tên đăng nhập, chương trình kết thúc ngay lập tức — đúng theo cơ chế LoginResult::Exit đã trình bày ở Chương 5.

6.3.2 Luồng thao tác của Admin

Sau khi đăng nhập thành công với tài khoản Admin, Menu Admin hiển thị 5 chức năng để lựa chọn bằng cách nhập số từ 1 đến 5.

Chức năng 1 – Xem danh sách câu hỏi: Chương trình liệt kê các môn học hiện có (TA10, TA11, TA12) để người dùng lựa chọn xem riêng từng môn, hoặc nhập ALL để xem toàn bộ. Mỗi môn học có 100 câu hỏi; khi chọn xem tất cả, hệ thống in lần lượt 300 câu hỏi theo thứ tự TA10 (câu 1–100), TA11 (câu 101–200) rồi đến TA12 (câu 201–300), đúng với thứ tự lưu trữ trong questions.txt. Sau khi in xong, người dùng nhấn Enter để quay lại Menu Admin.

Chức năng 2 – Thêm câu hỏi mới: Sau khi nhập đầy đủ các trường (ID, môn học, độ khó, nội dung câu hỏi, bốn đáp án, đáp án đúng), câu hỏi được lưu thành công vào questions.txt. Trường hợp ID nhập vào trùng với một câu hỏi đã tồn tại, chương trình báo lỗi ngay và yêu cầu nhập lại ID khác — đúng theo cơ chế isIdExists đã trình bày ở Mục 5.

Chức năng 3 – Xoá câu hỏi theo ID: Hệ thống in ra toàn bộ danh sách câu hỏi hiện có trong questions.txt để người dùng tham khảo trước khi nhập ID cần xoá. Trường hợp ID nhập vào không tồn tại trong ngân hàng, chương trình hiển thị thông báo lỗi tương ứng thay vì xoá nhầm hoặc gây crash.

Chức năng 4 – Xem lịch sử làm bài của tất cả sinh viên: Hệ thống đọc toàn bộ `history.txt` và hiển thị lần lượt từng bản ghi của mọi sinh viên đã từng thi.

Chức năng 5 – Đăng xuất: Chương trình quay trở lại màn hình đăng nhập ban đầu, sẵn sàng cho lượt đăng nhập tiếp theo.

6.3.3 Luồng thao tác của sinh viên

Sau khi đăng nhập với tài khoản **Username: hung12 / Password: svhung12**, hệ thống hiển thị Menu Sinh viên tương ứng với tên người dùng **hung12**.

Chức năng 1 – Bắt đầu làm bài thi: Hệ thống hiển thị danh sách 3 môn học (TA10, TA11, TA12) để người dùng chọn bằng cách nhập số từ 1 đến 3. Sau khi chọn môn (ví dụ chọn môn 1), người dùng nhập số câu hỏi muốn làm (ví dụ chọn 5 câu), tiếp theo nhập thời gian làm bài — giới hạn cho phép từ tối thiểu 10 giây đến tối đa 36 000 giây. Trước khi vào thi, hệ thống hỏi xác nhận: nếu người dùng nhập N, chương trình quay lại Menu Sinh viên mà không sinh đề; nếu nhập Y, bài thi chính thức bắt đầu.

Sau khi xác nhận, câu hỏi được hiển thị tuần tự — làm xong câu 1 mới chuyển sang câu 2, và tiếp tục như vậy cho đến khi hết số câu đã chọn. Nếu muốn nộp bài sớm, người dùng nhập S bất kỳ lúc nào và chương trình lập tức kết thúc bài thi, quay về Menu Sinh viên sau khi hiển thị kết quả. Sau khi hoàn thành (dù do trả lời hết câu hay nộp sớm), hệ thống tự động ghi lại kết quả vào `history.txt`.

Chức năng 2 – Xem lịch sử làm bài của bản thân: Hệ thống chỉ hiển thị các bản ghi có tên đăng nhập trùng khớp với người dùng hiện tại, đúng theo cơ chế lọc của `printByUser` đã trình bày ở Chương 5.

Chức năng 3 – Đăng xuất: Chương trình quay trở lại giao diện đăng nhập ban đầu.

6.3.4 Nhận xét từ kịch bản kiểm thử thực tế

Quá trình chạy thử trực tiếp này xác nhận chương trình hoạt động đúng như mô tả trong tài liệu thiết kế ở Chương 4 và khớp với kết quả các Test Case dạng bảng ở Mục 6. Đặc biệt, ba điểm sau được quan sát thấy nhất quán qua nhiều lần chạy thử độc lập: (1) thứ tự câu hỏi trong chức năng "Xem danh sách câu hỏi" của Admin luôn phản ánh đúng thứ tự lưu trữ vật lý trong file, không bị xáo trộn ngoài ý muốn; (2) việc nộp bài sớm bằng phím S hoạt động ở bất kỳ câu hỏi nào trong đề, không giới hạn riêng cho câu cuối; (3) dữ liệu lịch sử thi của các sinh viên khác nhau không bị lẫn lộn khi truy vấn theo `username`, kể cả khi nhiều sinh viên thi liên tiếp trong cùng một phiên chạy chương trình.

6.4 Đánh giá kết quả kiểm thử

Mã	Tên Test Case	Kết quả
TC-01	Đăng nhập đúng – Admin	PASS
TC-02	Đăng nhập đúng – Sinh viên	PASS
TC-03	Đăng nhập sai mật khẩu	PASS
TC-04	Thêm câu hỏi mới (kể cả trùng ID)	PASS
TC-05	Sinh đề theo tỉ lệ 40-40-20 và xáo trộn	PASS
TC-06	Làm bài và chấm điểm chính xác	PASS
TC-07	Tự động thu bài khi hết giờ	PASS
TC-08	Xem lịch sử thi nhiều lần	PASS
TC-09	Nhập sai định dạng (bẫy lỗi)	PASS
TC-10	Kiểm thử rò rỉ bộ nhớ (Valgrind)	PASS

Bảng 6.11: Tổng hợp kết quả 10 Test Case

Toàn bộ 10/10 tình huống kiểm thử đều đạt yêu cầu (PASS). Một số nhận xét quan trọng rút ra từ quá trình kiểm thử:

- **Tính đúng đắn của thuật toán sinh đề:** Cơ chế kiểm tra đủ số lượng câu hỏi theo từng mức độ *trước khi* sinh đề (TC-05) ngăn chặn hiệu quả tình huống đề thi thiếu câu hỏi hoặc lệch tỉ lệ độ khó.
- **Độ tin cậy của cơ chế đếm giờ:** Việc kiểm tra thời gian còn lại ở cả hai thời điểm (trước và sau khi nhập đáp án) trong TC-07 đảm bảo giới hạn thời gian được tôn trọng nghiêm ngặt, không có kẽ hở.
- **An toàn bộ nhớ:** Kết quả TC-10 xác nhận chiến lược “mỗi **new** có một **delete** tương ứng” được tuân thủ nhất quán trong toàn bộ 4 module, kể cả ở các nhánh xử lý lỗi (early return) — đây thường là điểm dễ bị bỏ sót gây rò rỉ bộ nhớ trong các chương trình C++ tự quản lý bộ nhớ.
- **Hạn chế phát hiện được:** Cơ chế xác thực hiện tại (mật khẩu dạng "sv"+ username, không mã hoá) chỉ phù hợp cho mục đích minh hoạ học thuật; nếu triển khai thực tế cần bổ sung cơ chế băm mật khẩu (hashing) — nội dung này được đề xuất ở hướng phát triển tại Chương 7.

Chương 7

Kết luận và hướng phát triển

7.1 Kết quả đạt được

Sau quá trình nghiên cứu, thiết kế và lập trình, nhóm đã xây dựng thành công chương trình Hệ thống thi trắc nghiệm trực tiếp trên Terminal, đáp ứng đầy đủ các yêu cầu đặt ra của đề bài. Chương trình được viết hoàn toàn bằng C++ theo mô hình lập trình hướng module và hướng đối tượng, không sử dụng thư viện ngoài nào ngoài thư viện chuẩn của C++.

Về phía người dùng, chương trình cung cấp hai vai trò chính là Sinh viên và Quản trị viên, mỗi vai trò có màn hình đăng nhập và menu riêng biệt. Sinh viên có thể chọn môn học, thiết lập số lượng câu hỏi và thời gian thi, sau đó làm bài và xem kết quả chi tiết lần lịch sử các lần thi. Quản trị viên có đầy đủ quyền quản lý ngân hàng câu hỏi: thêm, xoá, xem danh sách theo môn học, và theo dõi lịch sử thi của toàn bộ sinh viên.

Về phía kỹ thuật, chương trình đạt được các kết quả cụ thể như sau:

- Ngân hàng câu hỏi được quản lý bằng danh sách liên kết đơn, hỗ trợ thêm vào cuối danh sách, xoá theo ID và duyệt toàn bộ không giới hạn kích thước.
- Đề thi được sinh tự động theo tỉ lệ 40% câu Dễ – 40% câu Trung bình – 20% câu Khó, đảm bảo tính công bằng và phân hoá.
- Câu hỏi và thứ tự đáp án được xáo trộn ngẫu nhiên bằng thuật toán Fisher-Yates, loại bỏ hoàn toàn hiện tượng lặp mẫu giữa các lần thi.
- Dữ liệu câu hỏi và lịch sử thi được lưu trữ dưới dạng file văn bản thuần tuý với ký tự phân tách |, đảm bảo khả năng đọc/ghi bền vững giữa các phiên làm việc.
- Bộ đếm thời gian thực được tích hợp, tự động thu bài khi hết giờ ngay cả khi sinh viên chưa hoàn thành.
- Toàn bộ bộ nhớ động cấp phát bằng `new` đều được giải phóng trong Destructor tương ứng, không để lại Memory Leak.

7.2 Tổng kết các kỹ thuật đã được vận dụng

7.2.1 Cấu trúc dữ liệu: Danh sách liên kết đơn

Hai class `QuestionBank` và `HistoryManager` đều sử dụng danh sách liên kết đơn (Singly Linked List) làm cấu trúc lưu trữ nội tại. Mỗi node chứa dữ liệu (`Question` hoặc `TestRecord`) cùng con trỏ `next` trỏ đến node tiếp theo. Nhóm duy trì cả hai con trỏ `head` và `tail` để thêm vào cuối danh sách trong $O(1)$ và duyệt từ đầu trong $O(n)$. Việc lựa chọn danh sách liên kết thay cho mảng tĩnh giúp chương trình hoạt động với ngân hàng câu hỏi có kích thước tuỳ ý mà không cần khai báo trước dung lượng tối đa.

7.2.2 Quản lý bộ nhớ động: con trỏ, new/delete, tránh Memory Leak

Chương trình sử dụng bộ nhớ động ở hai tầng. Tầng thứ nhất là các node trong danh sách liên kết: mỗi lần thêm câu hỏi hoặc bản ghi lịch sử, một `QuestionNode` hoặc `HistoryNode` mới được cấp phát bằng `new` và nối vào danh sách; Destructor của mỗi class chịu trách nhiệm duyệt toàn bộ danh sách và `delete` từng node. Tầng thứ hai là mảng động trong class `Exam`: constructor nhận số câu hỏi n , cấp phát `Question[n]` và `char[n]` cho danh sách câu và mảng đáp án người dùng; Destructor gọi `delete[]` để thu hồi. Nguyên tắc “mỗi `new` phải có `delete`” được tuân thủ nghiêm ngặt trong suốt dự án.

7.2.3 Xử lý File I/O: đọc/ghi file text với ký tự phân tách '|'

Dữ liệu của chương trình được lưu trữ trong hai file văn bản thuần túy: `questions.txt` cho ngân hàng câu hỏi và `history.txt` cho lịch sử thi. Mỗi dòng trong file tương ứng với một bản ghi, các trường dữ liệu được ngăn cách bởi ký tự `|`. Khi đọc file, chương trình sử dụng `getline` kết hợp với `istringstream` để tách từng trường, đồng thời xử lý các trường hợp ngoại lệ như dòng trống, thiếu trường hoặc ID bị trùng. Khi ghi file, `QuestionBank` sử dụng `ofstream` thông thường để ghi đè toàn bộ danh sách, trong khi `HistoryManager` dùng cờ `ios::app` để nối thêm vào cuối file, tránh mất dữ liệu lịch sử cũ.

7.2.4 Thuật toán xáo trộn Fisher-Yates

Thuật toán Fisher-Yates Shuffle được cài đặt hai lần trong class `Exam`: một hàm `shuffleQuestions` để xáo trộn thứ tự câu hỏi trong đề thi và một hàm `shuffleAnswers` để xáo trộn thứ tự bốn đáp án của từng câu. Thuật toán duyệt mảng từ cuối về đầu, tại mỗi vị trí i chọn ngẫu nhiên một vị trí j trong đoạn $[0, i]$ rồi hoán đổi hai phần tử. Cách tiếp cận này đảm bảo mọi hoán vị đều có xác suất xuất hiện như nhau (phân phối đều). Đặc biệt, sau khi xáo trộn đáp án, chương trình tự động cập nhật lại trường `correct` của câu hỏi bằng cách tìm lại nội dung đáp án đúng trong mảng đã xáo, đảm bảo chấm điểm không bị sai lệch.

7.2.5 Lập trình hướng module: tách Header – Source – Data

Toàn bộ dự án được tổ chức theo mô hình tách biệt ba tầng. Thư mục `include` chứa các file `.h` khai báo interface (class, struct, enum, prototype hàm) mà không chứa bất kỳ logic nào. Thư mục `src` chứa các file `.cpp` triển khai logic tương ứng. Thư mục `data` chứa các file dữ liệu `questions.txt` và `history.txt`. Sự phân chia này giúp mỗi thành viên trong nhóm làm việc độc lập trên module của mình (Thành phụ trách Menu, Hùng phụ trách Exam, Văn phụ trách QuestionBank và History) mà không gây xung đột, đồng thời giúp việc biên dịch và bảo trì trở nên rõ ràng hơn.

7.2.6 Lập trình hướng đối tượng (OOP): Class, Constructor, Destructor

Ba class chính của chương trình là `QuestionBank`, `Exam` và `HistoryManager`, mỗi class đóng gói hoàn toàn dữ liệu nội tại (`private`) và chỉ để lộ các thao tác cần thiết qua giao diện `public`. Constructor của mỗi class chịu trách nhiệm khởi tạo trạng thái ban đầu hợp lệ (đặt `head = tail = nullptr` hoặc cấp phát mảng động), trong khi Destructor đảm nhận việc thu hồi toàn bộ tài nguyên. Nguyên tắc RAII (Resource Acquisition Is Initialization) được áp dụng nhất quán, đảm bảo không có tài nguyên nào bị rò rỉ khi đối tượng ra khỏi phạm vi.

7.2.7 Lập trình phòng ngừa

Các hàm quan trọng đều được thiết kế để xử lý tình huống bất thường mà không để chương trình crash. Hàm `addQuestion` kiểm tra ID trùng lặp trước khi thêm. Hàm `removeQuestion` xử lý ba trường hợp đặc biệt: danh sách rỗng, xóa node đầu, và không tìm thấy ID. Hàm `getQuestionAt` kiểm tra index âm và vượt giới hạn. Hàm `generateExamFromBank` kiểm tra số lượng câu hỏi

theo từng mức độ trước khi sinh đề, trả về `false` ngay nếu ngân hàng không đủ điều kiện thay vì sinh đề không hợp lệ.

7.2.8 Validate input và xử lý ngoại lệ trên Terminal

Module Menu cung cấp ba hàm tiện ích dùng chung: `getValidInt` để đọc số nguyên trong đoạn `[min, max]` với vòng lặp kiểm tra không giới hạn số lần thử, `getValidString` để đọc chuỗi không rỗng có giới hạn độ dài, và `getYesNo` để đọc lựa chọn Y/N. Cả ba hàm đều xử lý trường hợp `cin` fail (nhập sai kiểu dữ liệu) bằng cách gọi `cin.clear()` và `cin.ignore()` để xoá bộ đệm, tránh vòng lặp vô hạn. Trường hợp `cin.eof()` (hết dữ liệu đầu vào) được phát hiện riêng và kết thúc chương trình một cách an toàn.

7.2.9 Quy chuẩn lập trình: Naming Convention, No Global Variables

Nhóm thống nhất áp dụng một số quy ước lập trình nhất quán trong toàn dự án. Tên class và struct viết theo PascalCase (`QuestionBank`, `TestRecord`, `LoginSession`). Tên hàm viết theo camelCase (`addQuestion`, `shuffleAnswers`, `loadFromFile`). Tên biến cục bộ viết theo camelCase (`currentNode`, `correctCount`). Hằng số lớp khai báo `static const`. Đặc biệt, toàn bộ dữ liệu của chương trình được truyền qua tham số hàm hoặc thành viên của class, không có bất kỳ biến toàn cục nào được sử dụng, giúp tránh các lỗi trạng thái không mong muốn và tăng khả năng kiểm thử.

7.3 Đánh giá ưu điểm và hạn chế của chương trình

Về ưu điểm, chương trình đã đáp ứng đầy đủ các yêu cầu chức năng và kỹ thuật đặt ra. Kiến trúc module rõ ràng giúp mã nguồn dễ đọc, dễ bảo trì và phân công làm việc nhóm hiệu quả. Việc sử dụng danh sách liên kết đơn thay cho mảng tĩnh cho phép ngân hàng câu hỏi mở rộng không giới hạn. Thuật toán Fisher-Yates đảm bảo tính ngẫu nhiên thực sự cho mỗi lần thi. Toàn bộ bộ nhớ động được quản lý chặt chẽ, không để lại Memory Leak (đã được xác nhận bằng Valgrind ở Mục 6). Các hàm validate input bảo vệ chương trình khỏi các đầu vào sai định dạng hay ngoài giới hạn.

Về hạn chế, chương trình hiện vẫn còn một số điểm có thể cải thiện. Hệ thống xác thực tài khoản đang ở mức tối giản: mật khẩu sinh viên được suy ra trực tiếp từ tên đăng nhập ("`sv`" + `username`) mà không có cơ chế mã hoá hay lưu trữ tài khoản riêng. Giao diện Terminal thuần tuý không có màu sắc phong phú trên một số môi trường và không hỗ trợ chuột hay điều hướng bằng phím mũi tên. Chương trình chưa có chức năng tìm kiếm câu hỏi theo từ khoá hay lọc theo nhiều tiêu chí kết hợp. Ngoài ra, dữ liệu lịch sử được ghi nối vào file mà không có cơ chế phân trang khi số lượng bản ghi lớn, có thể làm chậm thao tác đọc toàn bộ lịch sử.

7.4 Hướng phát triển và Nâng cấp tương lai

Dựa trên những hạn chế đã xác định, nhóm đề xuất một số hướng nâng cấp khả thi cho các phiên bản tiếp theo của chương trình.

Thứ nhất, về quản lý tài khoản, hệ thống có thể được nâng cấp để lưu trữ tài khoản sinh viên trong file riêng với mật khẩu được băm bằng thuật toán đơn giản như MD5 hoặc SHA-256, thay vì quy tắc cố định hiện tại. Quản trị viên có thể có chức năng tạo, xoá và đặt lại mật khẩu tài khoản sinh viên.

Thứ hai, về cấu trúc dữ liệu, khi ngân hàng câu hỏi phát triển lên hàng nghìn câu, việc tìm kiếm và lọc theo nhiều tiêu chí sẽ trở nên chậm với danh sách liên kết đơn $O(n)$. Có thể xem xét bổ sung cấu trúc cây nhị phân tìm kiếm (BST) hoặc bảng băm (hash map) để tăng tốc độ truy vấn theo ID hoặc từ khoá.

Thứ ba, về tính năng thi, chương trình có thể bổ sung chế độ xem lại đề thi sau khi nộp bài kèm giải thích đáp án, hỗ trợ câu hỏi có nhiều đáp án đúng (multiple correct answers), hoặc tích hợp tính năng lưu bài tạm thời để tiếp tục sau khi bị gián đoạn.

Thứ tư, về giao diện, nếu chuyển sang phát triển trên nền web hoặc ứng dụng desktop, chương trình có thể tận dụng giao diện đồ họa để hiển thị bộ đếm thời gian trực quan, biểu đồ kết quả thi và bảng xếp hạng giữa các sinh viên.

Thứ năm, về khả năng mở rộng dữ liệu, định dạng file có thể được nâng cấp sang JSON hoặc XML để hỗ trợ câu hỏi có hình ảnh, công thức toán học hay nhiều loại media khác nhau, đồng thời dễ dàng tích hợp với các hệ thống quản lý học tập (LMS) bên ngoài.

Nhìn chung, dự án đã hoàn thành mục tiêu xây dựng một hệ thống thi trắc nghiệm hoạt động đầy đủ, vận dụng các kỹ thuật lập trình C++ cốt lõi được học trong môn học. Quá trình thực hiện không chỉ củng cố kiến thức lý thuyết mà còn rèn luyện kỹ năng làm việc nhóm, phân chia công việc theo module và quản lý mã nguồn bằng Git. Những hạn chế còn tồn tại chính là động lực cho các cải tiến trong tương lai, hướng đến một sản phẩm hoàn thiện và thực dụng hơn.

Lời cảm ơn

Nhóm xin gửi lời cảm ơn sâu sắc đến **TS. Vũ Thành Nam** — giảng viên hướng dẫn môn học Kỹ thuật Lập trình (MI3310) — vì đã tận tình truyền đạt những kiến thức nền tảng và định hướng quý báu trong suốt quá trình giảng dạy, từ các nguyên lý thiết kế chương trình, phong cách lập trình tốt cho đến những kinh nghiệm thực tiễn trong kỹ thuật lập trình C++. Những góp ý của thầy trong các buổi trên lớp đã giúp nhóm hình dung rõ ràng hơn về cách vận dụng lý thuyết vào một sản phẩm phần mềm cụ thể.

Nhóm cũng xin cảm ơn Khoa Toán – Tin, Đại học Bách khoa Hà Nội đã tạo điều kiện về môi trường học tập và tài liệu tham khảo để nhóm có thể hoàn thành đề tài này một cách thuận lợi.

Cuối cùng, ba thành viên trong nhóm xin cảm ơn nhau vì tinh thần hợp tác, trách nhiệm và kiên trì trong suốt quá trình thực hiện đề tài — từ những buổi thảo luận thiết kế ban đầu, những lần debug đến tận khuya, cho đến khi hoàn thiện báo cáo cuối cùng. Đây là một trải nghiệm làm việc nhóm đáng giá, giúp mỗi thành viên trưởng thành hơn cả về kỹ năng chuyên môn lẫn kỹ năng phối hợp.

Do thời gian thực hiện có hạn và đây là lần đầu nhóm xây dựng một chương trình hoàn chỉnh theo mô hình module hoá quy mô như vậy, báo cáo và sản phẩm chắc chắn còn nhiều thiếu sót. Nhóm rất mong nhận được sự góp ý từ thầy và hội đồng chấm để có thể hoàn thiện hơn trong những dự án tiếp theo.

Tài liệu tham khảo

- [1] D. Guralnick, *E-Learning and Innovative Pedagogies*. Cham, Switzerland: Springer, 2015.
- [2] FPT, “Tương lai giáo dục trực tuyến: Cách học tập đang thay đổi ra sao?.” <https://fpt.vn/tin-tuc/tuong-lai-cua-giao-duc-truc-tuyen-11312.html>, 2025. Truy cập ngày 20/06/2026. Dẫn nguồn số liệu Statista về quy mô thị trường E-learning Việt Nam.
- [3] Nettop Learning, “Thực trạng e-learning tại việt nam: Xu hướng và cơ hội.” <https://www.nettop.vn/thuc-trang-e-learning-tai-viet-nam-xu-huong-va-co-hoi/>, 2025. Truy cập ngày 20/06/2026.
- [4] N. T. Hiền, “Vietnam edtech & elearning report 2025: Kỷ nguyên vươn mình của giáo dục trực tuyến.” <https://www.nguyentrihien.com/2025/02/vietnam-edtech-elearning-report-2025-ky.html>, 2025. Truy cập ngày 20/06/2026. Ấn phẩm thường niên lần thứ 15 về thị trường EdTech và eLearning Việt Nam.
- [5] Thủ tướng Chính phủ, “Quyết định số 131/QĐ-ttg về Đề án “tăng cường ứng dụng công nghệ thông tin và chuyển đổi số trong giáo dục và đào tạo giai đoạn 2022–2025, định hướng đến năm 2030”.” Văn bản quy phạm pháp luật, 2022.
- [6] V. T. Nam, “Bài giảng môn học kỹ thuật lập trình (mi3310).” Tài liệu giảng dạy, Khoa Toán – Tin, Đại học Bách khoa Hà Nội, 2022.
- [7] I. Sommerville, *Software Engineering*. Boston, MA: Pearson, 10th ed., 2016.
- [8] D. E. Knuth, *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*. Reading, MA: Addison-Wesley, 3rd ed., 1997.
- [9] B. Stroustrup, *The C++ Programming Language*. Upper Saddle River, NJ: Addison-Wesley, 4th ed., 2013.
- [10] B. W. Kernighan and R. Pike, *The Practice of Programming*. Reading, MA: Addison-Wesley, 1999.
- [11] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*. Cambridge, MA: MIT Press, 3rd ed., 2009.
- [12] PHX Smart School, “Thực trạng e-learning tại việt nam 2024: Thách thức và cơ hội.” <https://blog.phx-smartschool.com/thuc-trang-e-learning-tai-viet-nam/>, 2024. Truy cập ngày 20/06/2026.

Phụ lục A

Mã nguồn hàm main()

Dưới đây là toàn bộ nội dung file `main.cpp` — điểm khởi đầu của chương trình, có nhiệm vụ khởi tạo bộ sinh số ngẫu nhiên, điều phối vòng lặp đăng nhập và chuyển hướng đến menu tương ứng với vai trò người dùng.

```
1  // =====
2  //  main.cpp - Entry point | Noi cac module lai voi nhau
3  //  Tac gia: THANH | Role: System Architect / QA
4  //
5  //  LUONG CHINH:
6  //    main()
7  //      |-- showWelcomeBanner()
8  //      |-- showLoginScreen()    <-- lap cho den khi dang nhap OK
9  //      |-- [Admin]             --> showAdminMenu()
10 //      |-- [Student]           --> showStudentMenu()
11 // =====
12
13 #include "../include/Menu.h"
14 #include <iostream>
15 #include <cstdlib>
16 #include <ctime>
17 using namespace std;
18
19 int main() {
20     srand(static_cast<unsigned int>(time(nullptr)));
21
22     // Vong lap chinh: cho phép đăng nhập lại nếu sai thông tin
23     bool appRunning = true;
24     while (appRunning) {
25         LoginSession session;
26         LoginResult result = showLoginScreen(session);
27
28         switch (result) {
29             case LoginResult::Success:
30                 if (session.isAdmin) {
31                     showAdminMenu(session);
32                 } else {
33                     showStudentMenu(session);
34                 }
35                 // Sau khi đăng xuất, quay lại màn hình đăng nhập
36                 break;
37
38             case LoginResult::InvalidCredentials:
39                 // showLoginScreen đã in thông báo lỗi, tiếp tục vòng lặp
40                 break;
41
42             case LoginResult::Exit:
43                 appRunning = false;
44                 break;
```

```
45     }
46 }
47
48 // Màn hình tam biệt
49 clearScreen();
50 std::cout << Color::CYAN << Color::BOLD;
51 printLine('=');
52 printCentered("Chúc mừng bạn hoàn thành bài kiểm tra!");
53 printLine('=');
54 std::cout << Color::RESET << "\n";
55
56 return 0;
57 }
```

Listing A.1: Toàn bộ mã nguồn main.cpp

Cấu trúc hàm `main()` ở trên là minh hoạ trực tiếp cho nguyên lý thiết kế từ trên xuống (Top-down design) đã trình bày ở Chương 2: toàn bộ logic chi tiết được che giấu sau các lời gọi hàm `showLoginScreen`, `showAdminMenu`, `showStudentMenu`, giúp hàm `main()` ngắn gọn, dễ đọc và phản ánh đúng luồng nghiệp vụ tổng quát của chương trình.

Phụ lục B

Mô tả các hàm xử lý chính

Bảng [B.1](#) tổng hợp các hàm quan trọng nhất của từng module, kèm mô tả ngắn gọn chức năng và độ phức tạp thời gian (nếu áp dụng được).

Module	Hàm	Mô tả chức năng
QuestionBank	<code>addQuestion(q)</code>	Thêm câu hỏi vào cuối danh sách liên kết, có kiểm tra trùng ID. $O(n)$ do phải kiểm tra trùng.
	<code>removeQuestion(id)</code>	Xoá câu hỏi theo ID, xử lý đầy đủ 4 trường hợp biên. $O(n)$.
	<code>loadFromFile(filename)</code>	Đọc toàn bộ ngân hàng câu hỏi từ file văn bản, bẫy lỗi từng dòng.
	<code>saveToFile(filename)</code>	Ghi đè toàn bộ ngân hàng câu hỏi hiện tại xuống file.
	<code>getQuestionsBySubjectAndDifficulty</code>	Trích xuất mảng động các câu hỏi theo môn học và độ khó, phục vụ sinh đề.
	<code>generateRandomSet(n)</code>	Bốc ngẫu nhiên n câu hỏi bằng Fisher-Yates trên mảng chỉ số.
Exam	<code>generateExamFromBank</code>	Sinh đề thi theo tỉ lệ độ khó 40-40-20 từ ngân hàng câu hỏi của một môn học.
	<code>shuffleQuestions</code>	Xáo trộn Fisher-Yates cho mảng câu hỏi. $O(n)$.
	<code>shuffleAnswers</code>	Xáo trộn Fisher-Yates cho 4 đáp án của một câu hỏi, cập nhật lại <code>correct</code> .
	<code>startExam</code>	Vòng lặp chính điều khiển quá trình làm bài: hiển thị câu hỏi, nhận đáp án, đếm giờ, thu bài và chấm điểm.
	<code>printExam</code>	In toàn bộ đề thi ra console, phục vụ mục đích kiểm thử/debug.
HistoryManager	<code>addRecord(r)</code>	Thêm một bản ghi kết quả thi vào cuối danh sách liên kết.
	<code>saveToFile</code>	Ghi nối (append) toàn bộ bản ghi trong bộ nhớ xuống <code>history.txt</code> .
	<code>loadFromFile</code>	Đọc toàn bộ lịch sử thi từ file vào danh sách liên kết.
	<code>printByUser(username)</code>	Lọc và hiển thị các bản ghi lịch sử thuộc về một người dùng cụ thể.
Menu	<code>showLoginScreen</code>	Hiển thị màn hình đăng nhập, xác thực Admin/Sinh viên, trả về <code>LoginResult</code> .
	<code>showAdminMenu</code>	Vòng lặp điều hướng 5 chức năng dành cho Quản trị viên.
	<code>showStudentMenu</code>	Vòng lặp điều hướng 3 chức năng dành cho Sinh viên.
	<code>getValidInt</code>	Đọc và bẫy lỗi số nguyên trong khoảng $[min, max]$ từ bàn phím.
	<code>getMaxNumberQuest</code>	Tính số câu hỏi tối đa hợp lệ thoả tỉ lệ 40-40-20 dựa trên số câu hiện có theo từng độ khó.

Phụ lục C

Mã nguồn đầy đủ chương trình

Phụ lục này trình bày toàn bộ mã nguồn của chương trình, bao gồm 4 file header (.h) khai báo interface và 4 file cài đặt (.cpp) tương ứng, giúp người đọc đối chiếu trực tiếp với các trích đoạn đã phân tích ở Chương 5 mà không cần truy cập kho mã nguồn riêng.

C.1 QuestionBank.h

```
1  // =====
2  //  QuestionBank.h - Khai báo cấu trúc câu hỏi & ngân hàng
3  //  Tác giả: VAN | Role: Kỹ sư Dữ liệu
4  //  =====
5
6  #ifndef QUESTIONBANK_H
7  #define QUESTIONBANK_H
8
9  #include <string>
10 using namespace std;
11
12 // Struct lưu thông tin một câu hỏi trắc nghiệm
13 struct Question {
14     int id;
15     string subject;
16     string difficulty; // Easy, Medium, Hard
17     string content;
18     string answers[4];
19     char correct;
20     // Ký tự đáp án đúng: 'A', 'B', 'C', hoặc 'D'
21 };
22
23 // Node của danh sách liên kết đơn chứa một câu hỏi
24 struct QuestionNode {
25     Question data;
26     QuestionNode* next;
27 };
28
29 // Class quản lý ngân hàng câu hỏi bằng danh sách liên kết đơn
30 class QuestionBank {
31 private:
32     QuestionNode* head;
33     QuestionNode* tail;
34
35 public:
36     // Constructor: khởi tạo danh sách rỗng
37     QuestionBank();
38
39     // Destructor: giải phóng toàn bộ bộ nhớ
40     ~QuestionBank();
```

```

41
42 // Kiểm tra ID đã tồn tại trong ngân hàng chưa
43 bool isIdExists(int id);
44
45 // Thêm câu hỏi vào cuối danh sách
46 bool addQuestion(Question q);
47
48 // Xóa câu hỏi theo id
49 void removeQuestion(int id);
50
51 // Đếm tổng số câu hỏi hiện có
52 int getQuestionCount();
53
54 // Lấy con trỏ Question tại vị trí index (0-indexed)
55 Question* getQuestionAt(int index);
56
57 // Đếm số câu theo mức độ
58 int countByDifficulty(const string& difficulty);
59
60 // Lấy danh sách câu hỏi theo mức độ
61 Question* getQuestionsByDifficulty(const string& difficulty, int& count);
62
63 // Lấy danh sách các môn học khác nhau có trong ngân hàng
64 string* getDistinctSubjects(int& count);
65
66 // Đếm số câu hỏi theo DUNG môn học VÀ mức độ (dùng khi sinh đề theo môn)
67 int countBySubjectAndDifficulty(const string& subject, const string&
difficulty);
68
69 // Lấy danh sách câu hỏi theo môn học VÀ mức độ
70 Question* getQuestionsBySubjectAndDifficulty(const string& subject, const
string& difficulty, int& count);
71
72 void printAll();
73
74 void printBySubject(const string& subject);
75
76 //I/O
77 bool loadFromFile(string filename);
78 bool saveToFile(string filename);
79
80 // Boc ngẫu nhiên N câu, trả về mảng dòng
81 Question* generateRandomSet(int n);
82 };
83
84 #endif // QUESTIONBANK_H

```

Listing C.1: QuestionBank.h – Khai báo cấu trúc câu hỏi và ngân hàng câu hỏi

C.2 QuestionBank.cpp

```

1 // =====
2 // QuestionBank.cpp - Hoàn thiện các hàm I/O
3 // Tác giả: VAN | Role: Kỹ sư Dữ liệu
4 // =====
5
6 #include "../include/QuestionBank.h"
7 #include <iostream>
8 #include <fstream>
9 #include <sstream>
10 #include <cctype>
11 using namespace std;
12
13 // Constructor: khởi tạo danh sách rỗng
14 QuestionBank::QuestionBank() {

```

```

15     head = nullptr;
16     tail = nullptr;
17 }
18
19 // Destructor: duyệt toàn bộ danh sách và giải phóng từng node để tránh Memory
    Leak
20 QuestionBank::~QuestionBank() {
21     QuestionNode* current = head;
22
23     while (current != nullptr) {
24         QuestionNode* nextNode = current->next;
25         delete current;
26         current = nextNode;
27     }
28
29     head = nullptr;
30     tail = nullptr;
31 }
32
33 // Duyệt toàn bộ danh sách để kiểm tra id có trùng không
34 bool QuestionBank::isIdExists(int id) {
35     QuestionNode* current = head;
36     while (current != nullptr) {
37         if (current->data.id == id) {
38             return true;
39         }
40         current = current->next;
41     }
42     return false;
43 }
44
45 bool QuestionBank::addQuestion(Question q) {
46     if (isIdExists(q.id)) {
47         cout << "[Canh bao] ID " << q.id << " đã tồn tại trong ngân hàng. Không
    them được.\n";
48         return false;
49     }
50
51     QuestionNode* newNode = new QuestionNode();
52     newNode->data = q;
53     newNode->next = nullptr;
54
55     if (tail == nullptr) {
56         head = newNode;
57         tail = newNode;
58     } else {
59         tail->next = newNode;
60         tail = newNode;
61     }
62     return true;
63 }
64
65 // Xóa node có id trùng khớp khỏi danh sách
66 void QuestionBank::removeQuestion(int id) {
67     // Trường hợp 1: Danh sách rỗng
68     if (head == nullptr) {
69         cout << "[Loi] Danh sách câu hỏi đang rỗng.\n";
70         return;
71     }
72
73     // Trường hợp 2: Node cần xóa chính là HEAD
74     if (head->data.id == id) {
75         QuestionNode* toDelete = head;
76         head = head->next;
77         if (head == nullptr) {
78             tail = nullptr;

```

```

79         }
80         delete toDelete;
81         return;
82     }
83
84     // Trường hợp 3: Node cần xóa nằm ở giữa hoặc cuối
85     QuestionNode* prev = head;
86     QuestionNode* current = head->next;
87
88     while (current != nullptr) {
89         // Tìm thấy node cần xóa tại 'current'
90         if (current->data.id == id) {
91             prev->next = current->next;
92             if (current == tail) {
93                 tail = prev;
94             }
95             delete current;
96             return;
97         }
98         // Chưa tìm thấy
99         prev = current;
100        current = current->next;
101    }
102
103    // Trường hợp 4: Không tìm thấy id trong danh sách
104    cout << "[Canh bao] Không tìm thấy câu hỏi có id = " << id << ".\n";
105 }
106
107 // Đếm và trả về tổng số câu hỏi hiện có trong danh sách
108 int QuestionBank::getQuestionCount() {
109     int count = 0;
110     QuestionNode* current = head;
111     while (current != nullptr) {
112         count++;
113         current = current->next;
114     }
115     return count;
116 }
117
118 // Trả về con trỏ tới Question tại vị trí index (0-indexed)
119 // Trả về nullptr nếu index vượt quá phạm vi
120 Question* QuestionBank::getQuestionAt(int index) {
121     // Nếu index âm
122     if (index < 0) {
123         return nullptr;
124     }
125
126     int currentIndex = 0;
127     QuestionNode* current = head;
128     while (current != nullptr) {
129         if (currentIndex == index) {
130             return &(current->data);
131         }
132         currentIndex++;
133         current = current->next;
134     }
135
136     // Duyệt hết danh sách mà không tìm thấy
137     return nullptr;
138 }
139
140 // loadFromFile
141 // Định dạng mỗi dòng:
142 // id|subject|difficulty|content|A|B|C|D|correct
143 bool QuestionBank::loadFromFile(string filename) {
144     ifstream fin(filename);

```

```

145     if (!fin.is_open()) {
146         cout << "[Loi] Khong the mo file de doc: " << filename << "\n";
147         return false;
148     }
149
150     string line;
151     int lineNum = 0;
152     while (getline(fin, line)) {
153         lineNum++;
154         if (line.empty() || line[0] == '#') continue;
155         istringstream ss(line);
156         string token;
157         Question q;
158
159         // Truong 1: id
160         if (!getline(ss, token, '|')) {
161             cout << "[Canh bao] Dong " << lineNum << ": thieu truong id. Bo qua
.\n";
162             continue;
163         }
164         try {
165             q.id = stoi(token);
166         } catch (...) {
167             cout << "[Canh bao] Dong " << lineNum << ": id khong phai so (\n" <<
token << "\n"). Bo qua.\n";
168             continue;
169         }
170
171         // Truong 2: subject
172         if (!getline(ss, token, '|')) {
173             cout << "[Canh bao] Dong " << lineNum << ": thieu mon hoc. Bo qua.\n
";
174             continue;
175         }
176         q.subject = token;
177
178         // Truong 3: difficulty
179         if (!getline(ss, token, '|')) {
180             cout << "[Canh bao] Dong " << lineNum << ": thieu muc do. Bo qua.\n"
;
181             continue;
182         }
183         q.difficulty = token;
184
185         // Truong 4: content
186         if (!getline(ss, token, '|')) {
187             cout << "[Canh bao] Dong " << lineNum << ": thieu noi dung cau hoi.
Bo qua.\n";
188             continue;
189         }
190         q.content = token;
191
192         // Truong 5-8: 4 dap an A B C D
193         bool answersOk = true;
194         for (int i = 0; i < 4; i++) {
195             if (!getline(ss, token, '|')) {
196                 cout << "[Canh bao] Dong " << lineNum << ": thieu dap an thu "
<< (i + 1) << ". Bo qua.\n";
197                 answersOk = false;
198                 break;
199             }
200             q.answers[i] = token;
201         }
202         if (!answersOk) continue;
203
204         // Truong 9: correct ('A'/'B'/'C'/'D')

```

```

205         if (!getline(ss, token, '|')) {
206             cout << "[Canh bao] Dong " << lineNum << ": thieu dap an dung. Bo
qua.\n";
207             continue;
208         }
209         if (token.empty()) {
210             cout << "[Canh bao] Dong " << lineNum << ": truong dap an dung dang
trong. Bo qua.\n";
211             continue;
212         }
213         char c = toupper((unsigned char)token[0]);
214         if (c < 'A' || c > 'D') {
215             cout << "[Canh bao] Dong " << lineNum
216                 << ": dap an dung '" << token << "' khong hop le (chi chap nhan
A-D). Bo qua.\n";
217             continue;
218         }
219         q.correct = c;
220
221         if (!addQuestion(q)) {
222             cout << "[Canh bao] Dong " << lineNum << ": ID " << q.id << " bi
trung voi cau hoi da co, da bo qua dong nay.\n";
223         }
224     }
225
226     fin.close();
227     cout << "[OK] Da nap " << getQuestionCount() << " cau hoi tu file \"" <<
filename << "\".\n";
228     return true;
229 }
230
231 // Duyet toan bo danh sach, gom cac ten mon hoc khac nhau lai
232 string* QuestionBank::getDistinctSubjects(int& count) {
233     int total = getQuestionCount();
234     if (total == 0) {
235         count = 0;
236         return nullptr;
237     }
238
239     // Mang tam, toi da bang tong so cau hoi (truong hop xau nhat moi cau 1 mon)
240     string* temp = new string[total];
241     int distinctCount = 0;
242
243     QuestionNode* current = head;
244     while (current != nullptr) {
245         bool exists = false;
246         for (int i = 0; i < distinctCount; i++) {
247             if (temp[i] == current->data.subject) {
248                 exists = true;
249                 break;
250             }
251         }
252         if (!exists) {
253             temp[distinctCount++] = current->data.subject;
254         }
255         current = current->next;
256     }
257
258     // Cap lai mang dung kich thuoc thuc te de tra ve cho ben ngoai
259     string* result = new string[distinctCount];
260     for (int i = 0; i < distinctCount; i++) result[i] = temp[i];
261     delete[] temp;
262
263     count = distinctCount;
264     return result;
265 }

```

```

266
267 int QuestionBank::countBySubjectAndDifficulty(const string& subject, const
    string& difficulty) {
268     int count = 0;
269     QuestionNode* current = head;
270     while (current != nullptr) {
271         if (current->data.subject == subject && current->data.difficulty ==
            difficulty) {
272             count++;
273         }
274         current = current->next;
275     }
276     return count;
277 }
278
279 Question* QuestionBank::getQuestionsBySubjectAndDifficulty(const string& subject
    , const string& difficulty, int& count) {
280     count = countBySubjectAndDifficulty(subject, difficulty);
281     if (count == 0) return nullptr;
282
283     Question* result = new Question[count];
284     QuestionNode* current = head;
285     int index = 0;
286     while (current != nullptr) {
287         if (current->data.subject == subject && current->data.difficulty ==
            difficulty) {
288             result[index++] = current->data;
289         }
290         current = current->next;
291     }
292     return result;
293 }
294
295 // saveToFile
296 // Ghi toan bo danh sach xuong file theo dinh dang phan tach '|':
297 // id|subject|difficulty|content|answerA|answerB|answerC|answerD|correct
298 bool QuestionBank::saveToFile(string filename) {
299     ofstream fout(filename);
300     if (!fout.is_open()) {
301         cout << "[Loi] Khong the mo file de ghi: " << filename << "\n";
302         return false;
303     }
304
305     QuestionNode* current = head;
306     int saved = 0;
307     while (current != nullptr) {
308         const Question& q = current->data;
309         fout << q.id << '|'
310             << q.subject << '|'
311             << q.difficulty << '|'
312             << q.content << '|'
313             << q.answers[0] << '|'
314             << q.answers[1] << '|'
315             << q.answers[2] << '|'
316             << q.answers[3] << '|'
317             << q.correct
318             << '\n';
319         saved++;
320         current = current->next;
321     }
322
323     fout.close();
324     cout << "[OK] Da luu " << saved
325         << " cau hoi vao file \"" << filename << "\".\n";
326     return true;
327 }

```



```

328
329 void QuestionBank::printAll()
330 {
331     QuestionNode* current = head;
332
333     while (current != nullptr)
334     {
335         cout << "ID: " << current->data.id << "\n";
336         cout << "Mon hoc: " << current->data.subject << "\n";
337         cout << "Muc do: " << current->data.difficulty << "\n";
338         cout << "Cau hoi: " << current->data.content << "\n";
339
340         cout << "A. " << current->data.answers[0] << "\n";
341         cout << "B. " << current->data.answers[1] << "\n";
342         cout << "C. " << current->data.answers[2] << "\n";
343         cout << "D. " << current->data.answers[3] << "\n";
344
345         cout << "Dap an dung: "
346              << current->data.correct << "\n\n";
347
348         current = current->next;
349     }
350 }
351
352 void QuestionBank::printBySubject(const string& subject)
353 {
354     QuestionNode* current = head;
355
356     while (current != nullptr)
357     {
358         if (current->data.subject == subject)
359         {
360             cout << "ID: " << current->data.id << "\n";
361             cout << "Mon hoc: " << current->data.subject << "\n";
362             cout << "Muc do: " << current->data.difficulty << "\n";
363             cout << "Cau hoi: " << current->data.content << "\n";
364
365             cout << "A. " << current->data.answers[0] << "\n";
366             cout << "B. " << current->data.answers[1] << "\n";
367             cout << "C. " << current->data.answers[2] << "\n";
368             cout << "D. " << current->data.answers[3] << "\n";
369
370             cout << "Dap an dung: " << current->data.correct << "\n\n";
371         }
372         current = current->next;
373     }
374 }
375
376 int QuestionBank::countByDifficulty(
377     const string& difficulty)
378 {
379     int count = 0;
380
381     QuestionNode* current = head;
382
383     while (current != nullptr)
384     {
385         if (current->data.difficulty ==
386             difficulty)
387         {
388             count++;
389         }
390
391         current = current->next;
392     }
393 }

```

```

394     return count;
395 }
396
397 Question* QuestionBank::getQuestionsByDifficulty(
398     const string& difficulty,
399     int& count)
400 {
401     count = countByDifficulty(difficulty);
402
403     if (count == 0)
404         return nullptr;
405
406     Question* result =
407         new Question[count];
408
409     QuestionNode* current = head;
410
411     int index = 0;
412
413     while (current != nullptr)
414     {
415         if (current->data.difficulty ==
416             difficulty)
417         {
418             result[index++] =
419                 current->data;
420         }
421
422         current = current->next;
423     }
424
425     return result;
426 }
427
428 // moi them 16/06/2026
429 Question* QuestionBank::generateRandomSet(int n) {
430     int total = getQuestionCount();
431     if (n > total) {
432         cout << "[Loi] Khong du cau hoi!\n";
433         return nullptr;
434     }
435
436     // Tao mang index roi Fisher-Yates
437     int* indices = new int[total];
438     for (int i = 0; i < total; i++) indices[i] = i;
439     for (int i = total - 1; i > 0; i--) {
440         int j = rand() % (i + 1);
441         int tmp = indices[i]; indices[i] = indices[j]; indices[j] = tmp;
442     }
443
444     // Lay N cau dau
445     Question* result = new Question[n];
446     for (int i = 0; i < n; i++)
447         result[i] = *getQuestionAt(indices[i]);
448
449     delete[] indices;
450     return result;
451 }

```

Listing C.2: QuestionBank.cpp – Cài đặt ngân hàng câu hỏi

C.3 Exam.h

```

1 // =====
2 // Exam.h - Khai bao logic sinh de, xao tron, bo dem gio

```

```

3 // Tac gia: HUNG | Role: Ky su Logic
4 // =====
5
6 #pragma once
7 #include "QuestionBank.h"
8 #include "History.h"
9 #include <string>
10
11 class Exam {
12 public:
13     // Khoi tao: cap phat mang Question[n]
14     Exam(int n, int timeLimitSec);
15     int timeLimitSec;
16     // Destructor: giai phong bo nho dong
17     ~Exam();
18
19     void loadFromBank(Question* questions);
20
21     // Nap ngau nhien cau hoi tu ngan hang vao de thi
22     bool generateExamFromBank(QuestionBank& bank, const std::string& subject);
23
24     // Xao tron thu tu cau hoi va dap an
25     void shuffleExam();
26
27     // In de thi ra console de kiem tra
28     void printExam() const;
29
30     TestRecord startExam(const std::string &username, int timeLimitMin);
31
32 private:
33     // So luong cau hoi trong de
34     int numberOfQuestions;
35
36     // Mang dong luu danh sach cau hoi
37     Question* questionList;
38
39     // Mang dong luu dap an nguoi dung chon
40     char* userAnswers;
41
42     // Xao tron mang dap an cua 1 cau (Fisher-Yates)
43     void shuffleAnswers(std::string arr[], int n);
44
45     // Xao tron mang cau hoi (Fisher-Yates)
46     void shuffleQuestions(Question arr[], int n);
47     // Copy cau hoi tu mang vao exam
48
49     static const int EASY_PERCENT = 40;
50     static const int MEDIUM_PERCENT = 40;
51     static const int HARD_PERCENT = 20;
52 };

```

Listing C.3: Exam.h – Khai báo logic sinh đề, xáo trộn, bộ đếm giờ

C.4 Exam.cpp

```

1 // =====
2 // Exam.cpp - Trien khai logic sinh de, xao tron, bo dem gio
3 // Tac gia: HUNG | Role: Ky su Logic / Thuat toan
4 // =====
5
6 #include "Exam.h"
7 #include <iostream>
8 #include <limits>
9 #include <ctime>
10 #include <cctype>

```

```

11 using namespace std;
12
13 // =====
14 // CONSTRUCTOR / DESTRUCTOR
15 // =====
16
17 // Constructor: Cap phat dong mang Question[N]
18 Exam::Exam(int n, int t)
19 {
20     numberOfQuestions = n;
21     timeLimitSec = t;
22
23     questionList = new Question[numberOfQuestions];
24     userAnswers = new char[numberOfQuestions];
25
26     for (int i = 0; i < numberOfQuestions; i++)
27         userAnswers[i] = '-';
28 }
29
30 // Destructor: Giai phong bo nho dong
31 Exam::~Exam() {
32     delete[] questionList;
33     delete[] userAnswers;
34 }
35
36 // =====
37 // NAP CAU HOI VAO DE THI
38 // =====
39
40 // Nap tu mang Question[] (dung sau generateRandomSet)
41 void Exam::loadFromBank(Question* questions) {
42     for (int i = 0; i < numberOfQuestions; i++)
43         questionList[i] = questions[i];
44 }
45
46 // Nap cau hoi tu QuestionBank vao Exam
47 bool Exam::generateExamFromBank(QuestionBank& bank, const string& subject)
48 {
49     int easyNeed = numberOfQuestions * EASY_PERCENT / 100;
50     int mediumNeed = numberOfQuestions * MEDIUM_PERCENT / 100;
51     int hardNeed = numberOfQuestions - easyNeed - mediumNeed;
52
53     int easySize, mediumSize, hardSize;
54
55     Question* easyQuestions = bank.getQuestionsBySubjectAndDifficulty(subject,
56 "Easy", easySize);
57     Question* mediumQuestions = bank.getQuestionsBySubjectAndDifficulty(subject,
58 "Medium", mediumSize);
59     Question* hardQuestions = bank.getQuestionsBySubjectAndDifficulty(subject,
60 "Hard", hardSize);
61
62     if (easySize < easyNeed || mediumSize < mediumNeed || hardSize < hardNeed)
63     {
64         cout << "[Loi] Ngan hang cau hoi mon " << subject << " khong du de sinh
65 de.\n";
66         delete[] easyQuestions;
67         delete[] mediumQuestions;
68         delete[] hardQuestions;
69         return false;
70     }
71
72     shuffleQuestions(easyQuestions, easySize);
73     shuffleQuestions(mediumQuestions, mediumSize);
74     shuffleQuestions(hardQuestions, hardSize);
75
76     int index = 0;

```

```

73     for (int i = 0; i < easyNeed; i++)    questionList[index++] = easyQuestions[i];
74     for (int i = 0; i < mediumNeed; i++) questionList[index++] = mediumQuestions[i];
75     for (int i = 0; i < hardNeed; i++)    questionList[index++] = hardQuestions[i];
76
77     shuffleQuestions(questionList, numberOfQuestions);
78
79     delete[] easyQuestions;
80     delete[] mediumQuestions;
81     delete[] hardQuestions;
82     return true;
83 }
84
85 // =====
86 //  THUAT TOAN XAO TRON (Fisher-Yates)
87 //  =====
88
89 // Xao tron mang chuoai (Ap dung cho mang answers[4])
90 void Exam::shuffleAnswers(string arr[], int n)
91 {
92     for (int i = n - 1; i > 0; --i) {
93         int j = rand() % (i + 1);
94
95         string temp = arr[i];
96         arr[i] = arr[j];
97         arr[j] = temp;
98     }
99 }
100
101 // Xao tron thu tu cac cau hoi trong de
102 void Exam::shuffleQuestions(Question arr[], int n)
103 {
104     for (int i = n - 1; i > 0; --i) {
105         int j = rand() % (i + 1);
106
107         Question temp = arr[i];
108         arr[i] = arr[j];
109         arr[j] = temp;
110     }
111 }
112
113 // Ap dung toan bo: xao cau hoi + xao dap an tung cau,
114 // dong thoi cap nhat lai truong 'correct' sau khi xao dap an.
115 void Exam::shuffleExam() {
116
117     // 1. Xao thu tu cau hoi: Shuffle(questions, qCount)
118     shuffleQuestions(questionList, numberOfQuestions);
119
120     // 2. Voi moi cau, xao dap an va cap nhat dap an dung
121     for (int i = 0; i < numberOfQuestions; ++i) {
122         Question& q = questionList[i]; // Lay cau hoi hien tai
123
124         // Ghi nho noi dung dap an dung truoc khi xao
125         int originalCorrectIndex = q.correct - 'A';
126         string correctText = q.answers[originalCorrectIndex];
127
128         // Xao dap an rieng tung cau: Shuffle(q.answers, q.answerCount)
129         // Voi cau hoi trac nghiem co dinh la 4 dap an
130         shuffleAnswers(q.answers, 4);
131
132         // Cap nhat lai correctIndex sau khi xao
133         for (int k = 0; k < 4; ++k) {
134             if (q.answers[k] == correctText) {
135                 q.correct = static_cast<char>('A' + k);

```

```

136         break;
137     }
138 }
139 }
140 }
141
142 // =====
143 // IN DE THI (dung de debug / kiem tra)
144 // =====
145
146 void Exam::printExam() const {
147     for (int i = 0; i < numberOfQuestions; ++i) {
148         cout << "Cau hoi " << (i + 1) << " (ID: " << questionList[i].id << ") ["
149         << questionList[i].difficulty << "] : ";
150         cout << " A. " << questionList[i].answers[0] << "\n";
151         cout << " B. " << questionList[i].answers[1] << "\n";
152         cout << " C. " << questionList[i].answers[2] << "\n";
153         cout << " D. " << questionList[i].answers[3] << "\n";
154         cout << "=> Dap an dung: " << questionList[i].correct << "\n\n";
155     }
156 }
157
158 // =====
159 // CHAY BAI THI
160 // =====
161
162 // Hien thi tung cau hoi, nhan dap an, dem gio nguoc,
163 // tu dong thu bai khi het gio, tinh diem thang 10.
164 TestRecord Exam::startExam(const std::string& username, int timeLimitSec_) {
165     cout << "\n== BAT DAU LAM BAI ==\n";
166     cout << "Thoi gian: " << timeLimitSec_ << " giay\n";
167     cout << "Nhan 'S' de nop bai som.\n\n";
168
169     TestRecord record;
170     record.studentName = username;
171     record.correctCount = 0;
172     record.totalCount = numberOfQuestions;
173
174     // Khoi tao mang ket qua tung cau
175     bool* result = new bool[numberOfQuestions];
176     for (int i = 0; i < numberOfQuestions; i++) {
177         userAnswers[i] = '-';
178         result[i] = false;
179     }
180
181     time_t startTime = time(nullptr);
182     bool submitted = false; // true khi nop som hoac het gio
183
184     for (int i = 0; i < numberOfQuestions && !submitted; i++) {
185         // -- Kiem tra gio TRUOC khi hien thi cau hoi -----
186         long elapsed = (long)difftime(time(nullptr), startTime);
187         long remaining = timeLimitSec_ - elapsed;
188
189         if (remaining <= 0) {
190             cout << "\n\n*** HET GIO! He thong tu dong thu bai. ***\n";
191             submitted = true;
192             break;
193         }
194
195         cout << "\n[Thoi gian con lai: " << remaining << " giay]\n";
196
197         // -- Hien thi cau hoi -----
198         Question& q = questionList[i];
199         cout << "-----\n";
200         cout << "Cau " << (i + 1) << "/" << numberOfQuestions << "\n";

```

```

201     cout << q.content << "\n";
202     cout << "A. " << q.answers[0] << "\n";
203     cout << "B. " << q.answers[1] << "\n";
204     cout << "C. " << q.answers[2] << "\n";
205     cout << "D. " << q.answers[3] << "\n";
206     cout << "(Nhập S để nộp bài ngay)\n";
207
208     // -- Nhận đáp án từ người dùng -----
209     char chosen = '-';
210     while (true) {
211         cout << "Chọn đáp án (A/B/C/D hoặc S=nộp bài): ";
212         string input;
213         if (!(cin >> input)) break;
214         if (input.empty()) continue;
215
216         char c = static_cast<char>(toupper(static_cast<unsigned char>(input
[0])));
217
218         if (c == 'S') {
219             cout << "\nBạn đã chọn nộp bài sớm.\n";
220             submitted = true;
221             break;
222         }
223         if (c >= 'A' && c <= 'D') {
224             chosen = c;
225             break;
226         }
227         cout << "Nhập không hợp lệ! Vui lòng nhập A/B/C/D hoặc S.\n";
228     }
229
230     if (submitted) break;
231
232     // -- Kiểm tra giới hạn SAU KHI nhập xong -----
233     elapsed = (long)diffTime(time(nullptr), startTime);
234     remaining = timeLimitSec_ - elapsed;
235     if (remaining <= 0) {
236         cout << "\n\n*** HẾT GIỚI HẠN TRONG LÚC LÀM CÂU NÀY! Hệ thống tự động thu
bài. ***\n";
237         submitted = true;
238     }
239
240     // -- Ghi nhận đáp án và chấm điểm câu này -----
241     userAnswers[i] = chosen;
242     if (chosen != '-' && chosen == q.correct) {
243         result[i] = true;
244         record.correctCount++;
245     }
246 }
247
248     // -- Tính thời gian thực tế đã dùng -----
249     long usedSec = (long)diffTime(time(nullptr), startTime);
250     if (usedSec > timeLimitSec_) usedSec = timeLimitSec_;
251     cout << "\nThời gian làm bài: " << usedSec << " giây\n";
252
253     // -- Tính điểm -----
254     record.score = (record.totalCount > 0)
255         ? (double)record.correctCount / record.totalCount * 10.0
256         : 0.0;
257
258     // -- Ghi timestamp -----
259     time_t nowTime = time(nullptr);
260     tm* localTm = localtime(&nowTime);
261     char buf[32];
262     strftime(buf, sizeof(buf), "%Y-%m-%d %H:%M:%S", localTm);
263     record.datetime = buf;
264

```

```

265 // -- Hien thi chi tiet ket qua tung cau -----
266 cout << "\n\n=====\\n";
267 cout << "CHI TIET KET QUA BAI THI\\n";
268 cout << "=====\\n";
269
270 for (int i = 0; i < numberOfQuestions; i++) {
271     cout << "\\nCau " << (i + 1) << ":\\n";
272     cout << questionList[i].content << "\\n";
273
274     cout << "Ban chon : " << userAnswers[i];
275     if (userAnswers[i] >= 'A' && userAnswers[i] <= 'D')
276         cout << ". " << questionList[i].answers[userAnswers[i] - 'A'];
277     cout << "\\n";
278
279     cout << "Dap an dung: " << questionList[i].correct << ". "
280         << questionList[i].answers[questionList[i].correct - 'A'] << "\\n";
281
282     if (userAnswers[i] == '-')
283         cout << "[CHUA TRA LOI]\\n";
284     else if (result[i])
285         cout << "[DUNG]\\n";
286     else
287         cout << "[SAI]\\n";
288 }
289
290 cout << "\\n=====\\n";
291 cout << "Tong so cau dung: " << record.correctCount
292     << "/" << record.totalCount << "\\n";
293 cout << "Diem so: " << record.score << "/10\\n";
294 cout << "=====\\n";
295
296 delete[] result;
297 return record;
298 }

```

Listing C.4: Exam.cpp – Cài đặt logic sinh đề, xáo trộn, bộ đếm giờ

C.5 History.h

```

1 // =====
2 // History.h - Khai bao struct TestRecord & HistoryManager
3 // Tac gia: VAN | Role: Ky su Du lieu
4 // =====
5
6 #ifndef HISTORY_H
7 #define HISTORY_H
8
9 #include <string>
10 using namespace std;
11
12 // Struct luu thong tin mot lan thi cua thi sinh
13 struct TestRecord {
14     string studentName;
15     string subject;
16     int correctCount;
17     int totalCount;
18     double score;
19     string datetime; // Ngay gio thi (dang chuoi, vi du: "2025-06-09
20     14:30:00")
21 };
22
23 // Node cua danh sach lien ket don chua mot ban ghi lich su
24 struct HistoryNode {
25     TestRecord data;
26     HistoryNode* next;

```



```

26 };
27
28 // Class quản lý lịch sử thi bằng danh sách liên kết đơn
29 class HistoryManager {
30 private:
31     HistoryNode* head;
32     HistoryNode* tail;
33
34 public:
35     // Constructor: khởi tạo danh sách rỗng
36     HistoryManager();
37
38     // Destructor: giải phóng toàn bộ bộ nhớ
39     ~HistoryManager();
40
41     // Thêm một bản ghi lịch sử vào cuối danh sách
42     void addRecord(TestRecord r);
43
44     void printAll();
45     void printByUser(const string& username);
46
47     // Ghi toàn bộ danh sách ra file
48     void saveToFile(const string& filename);
49
50     // Lưu thông tin từ file history.text vào danh sách môc mới
51     void loadFromFile(const string& filename);
52 };
53
54 void printAll();
55 #endif // HISTORY_H

```

Listing C.5: History.h – Khai báo struct TestRecord và HistoryManager

C.6 History.cpp

```

1 // =====
2 // History.cpp - Hoàn thiện các hàm I/O
3 // Tác giả: VAN | Role: Kỹ sư Dữ liệu
4 // =====
5
6 #include "../include/History.h"
7 #include <iostream>
8 #include <fstream>
9 #include <sstream>
10 using namespace std;
11
12 // Constructor: khởi tạo danh sách rỗng
13 HistoryManager::HistoryManager() {
14     head = nullptr;
15     tail = nullptr;
16 }
17
18 // Destructor: duyệt toàn bộ danh sách và giải phóng từng node
19 HistoryManager::~HistoryManager() {
20     HistoryNode* current = head;
21     while (current != nullptr) {
22         HistoryNode* nextNode = current->next;
23         delete current;
24         current = nextNode;
25     }
26     head = nullptr;
27     tail = nullptr;
28 }
29
30 // Thêm bản ghi mới vào cuối danh sách

```

```

31 void HistoryManager::addRecord(TestRecord r) {
32     HistoryNode* newNode = new HistoryNode();
33     newNode->data = r;
34     newNode->next = nullptr;
35     if (tail == nullptr) {
36         // Truong hop: Danh sach rong
37         head = newNode;
38         tail = newNode;
39     } else {
40         // Truong hop con lai
41         tail->next = newNode;
42         tail = newNode;
43     }
44 }
45
46 // Ghi noi toan bo danh sach lich su vao file text
47 // Dinh dang moi dong: studentName|correctCount|totalCount|score|datetime
48 void HistoryManager::saveToFile(const string& filename) {
49     ofstream outFile(filename, ios::app);
50     if (!outFile.is_open()) {
51         cerr << "[Loi] Khong the mo file: " << filename << "\n";
52         return;
53     }
54     HistoryNode* current = head;
55     while (current != nullptr) {
56         const TestRecord& rec = current->data;
57         outFile << rec.studentName << "|"
58             << rec.subject << "|"
59             << rec.correctCount << "|"
60             << rec.totalCount << "|"
61             << rec.score << "|"
62             << rec.datetime << "\n";
63
64         current = current->next;
65     }
66     outFile.close();
67     cout << "[OK] Da ghi lich su vao file: " << filename << "\n";
68 }
69
70 // Luu thong tin tu file history.text vao danh sach moc noi
71 void HistoryManager::loadFromFile(const string& filename) {
72     ifstream fin(filename);
73     if (!fin.is_open()) return;
74     string line;
75     while (getline(fin, line)) {
76         if (line.empty()) continue;
77         istringstream ss(line);
78         string token;
79         TestRecord r;
80
81         getline(ss, r.studentName, '|');
82         getline(ss, r.subject, '|');
83         getline(ss, token, '|');
84         r.correctCount = stoi(token);
85         getline(ss, token, '|');
86         r.totalCount = stoi(token);
87         getline(ss, token, '|');
88         r.score = stod(token);
89         getline(ss, r.datetime, '|');
90         addRecord(r);
91     }
92     fin.close();
93 }
94
95 void HistoryManager::printAll()
96 {

```

```

97     if (head == nullptr)
98     {
99         cout << "Chua co lich su thi.\n";
100        return;
101    }
102
103    HistoryNode* current = head;
104
105    while (current != nullptr)
106    {
107        cout << "Sinh vien: "
108             << current->data.studentName
109             << "\n";
110
111        cout << "Mon hoc: "
112             << current->data.subject
113             << "\n";
114
115        cout << "So cau dung: "
116             << current->data.correctCount
117             << "/"
118             << current->data.totalCount
119             << "\n";
120
121        cout << "Diem: "
122             << current->data.score
123             << "\n";
124
125        cout << "Thoi gian: "
126             << current->data.datetime
127             << "\n";
128
129        cout << "-----\n";
130
131        current = current->next;
132    }
133 }
134
135 void HistoryManager::printByUser(const string& username)
136 {
137     bool found = false;
138
139     HistoryNode* current = head;
140
141     while (current != nullptr)
142     {
143         if (current->data.studentName == username)
144         {
145             found = true;
146
147             cout << "Mon hoc: "
148                  << current->data.subject
149                  << "\n";
150
151             cout << "So cau dung: "
152                  << current->data.correctCount
153                  << "/"
154                  << current->data.totalCount
155                  << "\n";
156
157             cout << "Diem: "
158                  << current->data.score
159                  << "\n";
160
161             cout << "Thoi gian: "
162                  << current->data.datetime

```

```

163         << "\n";
164
165         cout << "-----\n";
166     }
167
168     current = current->next;
169 }
170
171 if (!found)
172 {
173     cout << "Khong tim thay lich su thi cua sinh vien "
174         << username
175         << ".\n";
176 }
177 }

```

Listing C.6: History.cpp – Cài đặt quản lý lịch sử thi

C.7 Menu.h

```

1  // =====
2  //  Menu.h - Khai bao cac ham ve giao dien va dieu huong
3  //  Tac gia: THANH | Role: System Architect / UI / QA
4  //  =====
5
6  #pragma once
7  #include <string>
8
9  // -- Thông tin người dùng đang đăng nhập -----
10 struct LoginSession {
11     std::string username;
12     bool        isAdmin;
13 };
14
15 // -- Kết quả đăng nhập -----
16 enum class LoginResult {
17     Success,
18     InvalidCredentials,
19     Exit
20 };
21
22 // -- Các mã màu ANSI cho Terminal -----
23 namespace Color {
24     const std::string RESET    = "\033[0m";
25     const std::string BOLD     = "\033[1m";
26     const std::string RED      = "\033[31m";
27     const std::string GREEN    = "\033[32m";
28     const std::string YELLOW   = "\033[33m";
29     const std::string BLUE     = "\033[34m";
30     const std::string MAGENTA  = "\033[35m";
31     const std::string CYAN     = "\033[36m";
32     const std::string WHITE    = "\033[37m";
33     const std::string BG_BLUE  = "\033[44m";
34 }
35
36 // -- Hàm tiện ích Terminal -----
37 void clearScreen();
38 void pauseScreen();
39 void printLine(char c = '-', int width = 60);
40 void printCentered(const std::string& text, int width = 60);
41
42 // -- Màn hình chào mừng -----
43 void showWelcomeBanner();
44
45 // -- Đăng nhập / Đăng xuất -----

```

```

46 LoginResult showLoginScreen(LoginSession& session);
47
48 // -- Menu chính Admin -----
49 void showAdminMenu(LoginSession& session);
50
51 // -- Menu chính Sinh vien -----
52 void showStudentMenu(LoginSession& session);
53
54 // -- Input Validation -----
55 // Doc so nguyen trong doan [min, max], bay loi hoan toan
56 int getValidInt(const std::string& prompt, int minVal, int maxVal);
57
58 // Doc chuoi khong rong
59 std::string getValidString(const std::string& prompt, int maxLen = 50);
60
61 // Doc lua chon Y/N
62 bool getYesNo(const std::string& prompt);

```

Listing C.7: Menu.h – Khai báo các hàm vẽ giao diện và điều hướng

C.8 Menu.cpp

```

1 // =====
2 // Menu.cpp - Trien khai giao dien Menu va dieu huong
3 // Tac gia: THANH | Role: System Architect / UI / QA
4 // =====
5
6 #include "../include/Menu.h"
7 #include "../include/QuestionBank.h"
8 #include "../include/Exam.h"
9 #include "../include/History.h"
10
11 #include <iostream>
12 #include <limits>
13 #include <iomanip>
14 #include <cstdlib>
15 #include <cctype>
16
17 using namespace std;
18
19 // =====
20 // HAM TIEN ICH TERMINAL
21 // =====
22
23 void clearScreen() {
24 #ifdef _WIN32
25     system("cls");
26 #else
27     system("clear");
28 #endif
29 }
30
31 void pauseScreen() {
32     cout << "\nNhan Enter de tiep tục...";
33     cin.clear();
34     cin.ignore(numeric_limits<streamsize>::max(), '\n');
35     cin.get();
36 }
37
38 void printLine(char c, int width) {
39     cout << string(width, c) << "\n";
40 }
41
42 void printCentered(const string& text, int width) {
43     int len = static_cast<int>(text.length());

```

```

44     if (len >= width) {
45         cout << text << "\n";
46         return;
47     }
48     int padding = (width - len) / 2;
49     cout << string(padding, ' ') << text << "\n";
50 }
51
52 // =====
53 // INPUT VALIDATION
54 // =====
55
56 int getValidInt(const string& prompt, int minVal, int maxVal) {
57     int value;
58     while (true) {
59         cout << prompt;
60         if (cin >> value) {
61             if (value >= minVal && value <= maxVal) {
62                 cin.ignore(numeric_limits<streamsize>::max(), '\n');
63                 return value;
64             }
65             cout << Color::RED << "Gia tri phai trong khoang ["
66                  << minVal << ", " << maxVal << "]\n" << Color::RESET;
67         } else {
68             if (cin.eof()) {
69                 cout << "\n[He thong] Het du lieu dau vao. Thoat chuong trinh.\n";
70                 exit(0);
71             }
72             cout << Color::RED << "Vui long nhap mot so nguyen hop le!\n" <<
Color::RESET;
73             cin.clear();
74         }
75         cin.ignore(numeric_limits<streamsize>::max(), '\n');
76     }
77 }
78
79 string getValidString(const string& prompt, int maxLen) {
80     string input;
81     while (true) {
82         cout << prompt;
83         if (!getline(cin, input)) {
84             cout << "\n[He thong] Het du lieu dau vao. Thoat chuong trinh.\n";
85             exit(0);
86         }
87
88         if (!input.empty() && static_cast<int>(input.length()) <= maxLen) {
89             return input;
90         }
91
92         if (input.empty()) {
93             cout << Color::RED << "Chuoi khong duoc de trong!\n" << Color::RESET;
94         } else {
95             cout << Color::RED << "Chuoi qua dai (toi da " << maxLen << " ky tu)
!\n" << Color::RESET;
96         }
97     }
98 }
99
100 bool getYesNo(const string& prompt) {
101     while (true) {
102         cout << prompt << " (Y/N): ";
103         string input;
104         if (!getline(cin, input)) {
105             cout << "\n[He thong] Het du lieu dau vao. Thoat chuong trinh.\n";

```

```

106         exit(0);
107     }
108
109     if (!input.empty()) {
110         char c = static_cast<char>(toupper(static_cast<unsigned char>(input
[0])));
111         if (c == 'Y') return true;
112         if (c == 'N') return false;
113     }
114     cout << Color::RED << "Vui long nhap Y hoac N!\n" << Color::RESET;
115 }
116 }
117
118 // Tinh so cau hoi toi da co the sinh de duoc, dua tren dung cong thuc chia ti
le 40% Easy - 40% Medium - 20% Hard
119 int getMaxNumberQuest(int nEasy, int nMedium, int nHard)
120 {
121     int total = nEasy + nMedium + nHard;
122
123     // Thu N tu lon -> nho. N hop le dau tien tim duoc chinh la so cau toi da.
124     for (int n = total; n >= 1; n--) {
125         int easyNeed = n * 40 / 100;
126         int mediumNeed = n * 40 / 100;
127         int hardNeed = n - easyNeed - mediumNeed;
128
129         if (easyNeed <= nEasy && mediumNeed <= nMedium && hardNeed <= nHard) {
130             return n;
131         }
132     }
133
134     return 0; // Khong co N nao hop le (vi du thieu han 1 muc do)
135 }
136
137 // =====
138 //  MAN HINH CHAO MUNG
139 // =====
140
141 void showWelcomeBanner() {
142     clearScreen();
143     cout << Color::CYAN << Color::BOLD;
144     printLine('=');
145     printCentered("HE THONG THI TRAC NGHIEM MON TIENG ANH");
146     printLine('=');
147     cout << Color::RESET;
148     pauseScreen();
149 }
150
151 // =====
152 //  DANG NHAP
153 // =====
154
155 LoginResult showLoginScreen(LoginSession& session) {
156     clearScreen();
157     cout << Color::CYAN << Color::BOLD;
158     printLine('=');
159     printCentered("DANG NHAP HE THONG");
160     printLine('=');
161     cout << Color::RESET << "\n";
162
163     cout << "(Go 'exit' o ten dang nhap de thoat chuong trinh)\n\n";
164
165     string username = getValidString("Ten dang nhap: ", 50);
166
167     string lowerUsername = username;
168     for (char& c : lowerUsername) c = static_cast<char>(tolower(static_cast<
unsigned char>(c)));

```

```

169     if (lowerUsername == "exit") {
170         return LoginResult::Exit;
171     }
172
173     string password = getValidString("Mat khau: ", 50);
174
175     // Tai khoan Admin
176     if (username == "admin" && password == "admin123") {
177         session.username = username;
178         session.isAdmin = true;
179         return LoginResult::Success;
180     }
181
182     // Tai khoan Sinh vien: password = "sv" + username
183     if (password == ("sv" + username)) {
184         session.username = username;
185         session.isAdmin = false;
186         return LoginResult::Success;
187     }
188
189     cout << "\n" << Color::RED << "Sai ten dang nhap hoac mat khau!\n" << Color
::RESET;
190     pauseScreen();
191     return LoginResult::InvalidCredentials;
192 }
193
194 // =====
195 // MENU ADMIN
196 // =====
197
198 void showAdminMenu(LoginSession& session) {
199     const string QUESTIONS_FILE = "data/questions.txt";
200     const string HISTORY_FILE = "data/history.txt";
201
202     QuestionBank bank;
203     bank.loadFromFile(QUESTIONS_FILE);
204
205     bool running = true;
206     while (running) {
207         clearScreen();
208         cout << Color::MAGENTA << Color::BOLD;
209         printLine('=');
210         printCentered("MENU QUAN TRI VIEN (" + session.username + ")");
211         printLine('=');
212         cout << Color::RESET;
213
214         cout << "1. Xem danh sach cau hoi\n";
215         cout << "2. Them cau hoi moi\n";
216         cout << "3. Xoa cau hoi theo ID\n";
217         cout << "4. Xem lich su thi cua tat ca sinh vien\n";
218         cout << "5. Dang xuat\n";
219         printLine();
220
221         int choice = getValidInt("Chon chuc nang (1-5): ", 1, 5);
222
223         switch (choice) {
224             case 1: {
225                 clearScreen();
226                 cout << Color::BOLD;
227                 printCentered("DANH SACH CAU HOI");
228                 cout << Color::RESET;
229                 printLine();
230
231                 if (bank.getQuestionCount() == 0) {
232                     cout << "Ngan hang cau hoi dang rong.\n";
233                     pauseScreen();

```



```

234         break;
235     }
236
237     int subjectCount = 0;
238     string* subjects = bank.getDistinctSubjects(subjectCount);
239
240     cout << "Danh sach cac mon hoc:\n";
241     for (int i = 0; i < subjectCount; i++) {
242         cout << "- " << subjects[i] << "\n";
243     }
244
245     string subject;
246     while (true){
247         cout << "\nNhap mon hoc muon xem cau hoi (Nhap ALL neu muon
in het tat ca): ";
248         getline(cin, subject);
249
250         if (subject == "all" || subject == "ALL" || subject == "All"
)
251         {
252             bank.printAll();
253             pauseScreen();
254             break;
255         }
256
257         bool found = false;
258         for (int i = 0; i < subjectCount; i++)
259         {
260             if (subjects[i] == subject)
261             {
262                 found = true;
263                 break;
264             }
265         }
266
267         if (found)
268         {
269             bank.printBySubject(subject);
270             pauseScreen();
271             break;
272         }
273
274         cout << Color::RED
275             << "\n[LOI] Khong ton tai mon hoc \"" << subject << "
\n!\n"
276             << "Vui long nhap lai.\n"
277             << Color::RESET;
278     }
279     break;
280 }
281
282 case 2: {
283     clearScreen();
284     cout << Color::BOLD;
285     printCentered("THEM CAU HOI MOI");
286     cout << Color::RESET;
287     printLine();
288
289     Question q;
290     while (true) {
291         q.id = getValidInt("Nhap ID cau hoi (so nguyen duong): ", 1,
1000000);
292         if (bank.isIdExists(q.id)) {
293             cout << Color::RED << "[Loi] ID " << q.id
294                 << " da ton tai! Vui long nhap mot ID khac.\n" <<
Color::RESET;

```

```

295         } else {
296             break;
297         }
298     }
299
300     q.subject = getValidString("Mon hoc: ", 50);
301
302     while (true) {
303         string diff = getValidString("Muc do (Easy / Medium / Hard):
", 10);
304         if (!diff.empty()) {
305             diff[0] = toupper((unsigned char)diff[0]);
306             for (int i = 1; i < (int)diff.size(); i++)
307                 diff[i] = tolower((unsigned char)diff[i]);
308         }
309         if (diff == "Easy" || diff == "Medium" || diff == "Hard") {
310             q.difficulty = diff;
311             break;
312         }
313         cout << Color::RED << "Chi chap nhan: Easy, Medium, hoac
Hard!\n" << Color::RESET;
314     }
315
316     q.content = getValidString("Noi dung cau hoi: ", 200);
317     q.answers[0] = getValidString("Dap an A: ", 100);
318     q.answers[1] = getValidString("Dap an B: ", 100);
319     q.answers[2] = getValidString("Dap an C: ", 100);
320     q.answers[3] = getValidString("Dap an D: ", 100);
321
322     while (true) {
323         string cAns = getValidString("Dap an dung (A/B/C/D): ", 5);
324         char up = toupper((unsigned char)cAns[0]);
325         if (up == 'A' || up == 'B' || up == 'C' || up == 'D') {
326             q.correct = up;
327             break;
328         }
329         cout << Color::RED << "Vui long nhap A, B, C hoac D!\n" <<
Color::RESET;
330     }
331
332     if (bank.addQuestion(q)) {
333         bank.saveToFile(QUESTIONS_FILE);
334         cout << Color::GREEN << "\n[OK] Da them cau hoi thanh cong!\n
" << Color::RESET;
335     } else {
336         cout << Color::RED << "\n[Loi] Khong the them cau hoi (ID bi
trung).\n" << Color::RESET;
337     }
338     pauseScreen();
339     break;
340 }
341
342 case 3: {
343     clearScreen();
344     cout << Color::BOLD;
345     printCentered("XOA CAU HOI");
346     cout << Color::RESET;
347     printLine();
348
349     if (bank.getQuestionCount() == 0) {
350         cout << "Ngan hang cau hoi dang rong.\n";
351     } else {
352         bank.printAll();
353         printLine();
354         int id = getValidInt("Nhap ID cau hoi can xoa: ", 1,
1000000);

```

```

355         bank.removeQuestion(id);
356         bank.saveToFile(QUESTIONS_FILE);
357     }
358     pauseScreen();
359     break;
360 }
361
362 case 4: {
363     HistoryManager history;
364     history.loadFromFile(HISTORY_FILE);
365
366     clearScreen();
367     cout << Color::BOLD;
368     printCentered("LICH SU THI CUA SINH VIEN");
369     cout << Color::RESET;
370     printLine();
371     history.printAll();
372     pauseScreen();
373     break;
374 }
375
376 case 5:
377     running = false;
378     break;
379 }
380 }
381 }
382
383 // =====
384 // MENU SINH VIEN
385 // =====
386
387 void showStudentMenu(LoginSession& session) {
388     const string QUESTIONS_FILE = "data/questions.txt";
389     const string HISTORY_FILE = "data/history.txt";
390
391     bool running = true;
392     while (running) {
393         clearScreen();
394         cout << Color::BLUE << Color::BOLD;
395         printLine('=');
396         printCentered("MENU SINH VIEN (" + session.username + ")");
397         printLine('=');
398         cout << Color::RESET;
399
400         cout << "1. Bat dau lam bai thi\n";
401         cout << "2. Xem lich su thi cua ban\n";
402         cout << "3. Dang xuat\n";
403         printLine();
404
405         int choice = getValidInt("Chon chuc nang (1-3): ", 1, 3);
406
407         switch (choice) {
408             case 1: {
409                 QuestionBank bank;
410                 bank.loadFromFile(QUESTIONS_FILE);
411
412                 if (bank.getQuestionCount() == 0) {
413                     cout << Color::RED << "\nNgan hang cau hoi rong!\n" << Color
::RESET;
414                     pauseScreen();
415                     break;
416                 }
417
418                 // -- Cho sinh vien chon mon hoc -----
419                 int subjectCount = 0;

```

```

420         string* subjects = bank.getDistinctSubjects(subjectCount);
421
422         clearScreen();
423         cout << Color::BOLD;
424         printCentered("CHON MON HOC DE THI");
425         cout << Color::RESET;
426         printLine();
427
428         for (int i = 0; i < subjectCount; i++) {
429             cout << (i + 1) << ". " << subjects[i] << "\n";
430         }
431         printLine();
432
433         int subjectChoice = getValidInt("Chon mon hoc (1-" + to_string(
subjectCount) + "): ", 1, subjectCount);
434
435         string selectedSubject = subjects[subjectChoice - 1];
436         delete[] subjects;
437
438         int nEasy    = bank.countBySubjectAndDifficulty(selectedSubject,
"Easy");
439         int nMedium  = bank.countBySubjectAndDifficulty(selectedSubject,
"Medium");
440         int nHard    = bank.countBySubjectAndDifficulty(selectedSubject,
"Hard");
441
442         clearScreen();
443         cout << Color::BOLD;
444         printCentered("BAT DAU LAM BAI THI - MON " + selectedSubject);
445         cout << Color::RESET;
446         printLine();
447
448         int maxQ = getMaxNumberQuest(nEasy, nMedium, nHard);
449
450         if (maxQ < 1) {
451             cout << Color::RED
452                 << "[Loi] Mon " << selectedSubject
453                 << " chua du cau hoi theo ca 3 muc do de sinh de!\n"
454                 << Color::RESET;
455             pauseScreen();
456             break;
457         }
458
459         int numQ = getValidInt("Nhap so cau hoi muon thi (1- " +
to_string(maxQ) + "): ", 1, maxQ);
460         int timeLimit = getValidInt("Nhap thoi gian lam bai (giay, toi
thieu 10): ", 10, 36000);
461         if (!getYesNo("\nBat dau lam bai?")) break;
462
463         // -- Sinh de CHI TU mon hoc da chon -----
464         Exam exam(numQ, timeLimit);
465         if (!exam.generateExamFromBank(bank, selectedSubject)) {
466             cout << Color::RED
467                 << "[Loi] Khong the sinh de thi (ngan hang khong du theo
ti le 40-40-20).\n"
468                 << Color::RESET;
469             pauseScreen();
470             break;
471         }
472         exam.shuffleExam();
473
474         clearScreen();
475         TestRecord record = exam.startExam(session.username, exam.
timeLimitSec);
476         record.subject = selectedSubject;
477

```

```
478         clearScreen();
479         cout << Color::GREEN << Color::BOLD;
480         printLine('=');
481         printCentered("KET QUA BAI THI");
482         printLine('=');
483         cout << Color::RESET;
484
485         cout << "Sinh vien  : " << record.studentName << "\n";
486         cout << "Mon hoc    : " << record.subject << "\n";
487         cout << "So cau dung: " << record.correctCount << "/" << record.
totalCount << "\n";
488         cout << fixed << setprecision(2);
489         cout << "Diem so    : " << record.score << " / 10\n";
490         cout << "Thoi gian : " << record.datetime << "\n";
491
492         HistoryManager history;
493         history.addRecord(record);
494         history.saveToFile(HISTORY_FILE);
495
496         pauseScreen();
497         break;
498     }
499
500     case 2: {
501         HistoryManager history;
502         history.loadFromFile(HISTORY_FILE);
503
504         clearScreen();
505         cout << Color::BOLD;
506         printCentered("LICH SU LAM BAI THI CUA BAN");
507         cout << Color::RESET;
508         printLine();
509         cout << fixed << setprecision(2);
510         history.printByUser(session.username);
511         pauseScreen();
512         break;
513     }
514
515     case 3:
516         running = false;
517         break;
518 }
519 }
520 }
```

Listing C.8: Menu.cpp – Cài đặt giao diện Menu và điều hướng

Phụ lục D

Hướng dẫn cài đặt và sử dụng chương trình

C.1. Yêu cầu môi trường

- Trình biên dịch hỗ trợ chuẩn C++17 trở lên: **g++** (khuyến nghị MinGW UCRT64 trên Windows, hoặc GCC mặc định trên Linux/macOS).
- Visual Studio Code (tùy chọn) kèm extension C/C++ của Microsoft để biên dịch và debug thuận tiện thông qua `tasks.json` có sẵn trong dự án.

C.2. Biên dịch chương trình

Mở Terminal tại thư mục gốc của dự án (`KY_THUAT_LAP_TRINH/`) và chạy lệnh:

```
1 g++ -std=c++17 -g -Wall -I./include \  
2   src/main.cpp src/Menu.cpp src/Exam.cpp \  
3   src/History.cpp src/QuestionBank.cpp \  
4   -o quiz_program.exe
```

Listing D.1: Biên dịch thủ công bằng g++

Nếu sử dụng Visual Studio Code với cấu hình `.vscode/tasks.json` có sẵn, người dùng chỉ cần nhấn tổ hợp phím **Ctrl + Shift + B** để chạy tác vụ *Build Quiz Program*.

C.3. Chạy chương trình

Sau khi biên dịch thành công, file thực thi `quiz_program.exe` được tạo tại thư mục gốc của dự án. Chạy chương trình bằng lệnh:

```
1 ./quiz_program.exe
```

Listing D.2: Chạy chương trình

Lưu ý chương trình cần được chạy từ đúng thư mục gốc để các đường dẫn tương đối đến `data/questions.txt` và `data/history.txt` hoạt động chính xác.

C.4. Tài khoản mặc định

Vai trò	Username	Password
Admin	admin	admin123
Sinh viên	(bất kỳ, ví dụ hung)	sv + username (ví dụ svhung)

Bảng D.1: Tài khoản mặc định để đăng nhập chương trình

C.5. Thoát chương trình

Tại màn hình đăng nhập, gõ **exit** vào ô tên đăng nhập để kết thúc chương trình một cách an toàn. Trong quá trình làm bài thi, sinh viên có thể nhấn **S** bất kỳ lúc nào để nộp bài sớm thay vì phải hoàn thành toàn bộ số câu đã chọn.